



ITI Summer Project

Chirp Spread Spectrum (CSS) PHY Transmitter

Team 2

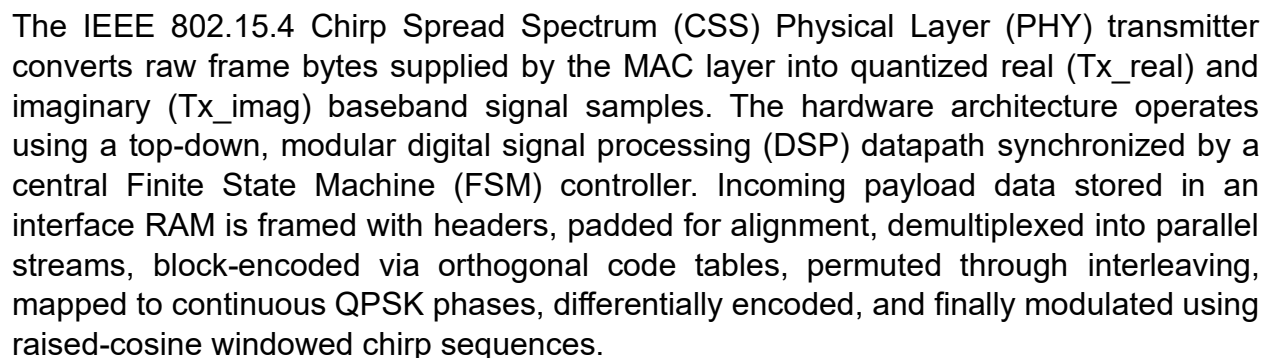
Table of Contents

1	introducction	4
2	design specifications	4
2.1	Block Diagram & Architecture:	4
2.2	Design Breakdown:	5
2.2.1	Design Top Wrapper:.....	5
2.2.2	System Controller (Controller):.....	8
2.2.3	CSS transmitter frontend:	13
2.2.4	Payload RAM:	17
2.2.5	PHR and PSDU Zero Padding:	18
2.2.6	DEMUX:	21
2.2.7	Serial to Parallel Converters:	22
2.2.8	Interface:	23
2.2.9	Symbol Mappers (I / Q):.....	24
2.2.10	inetrleaver and PPDU top wrapper:	26
2.2.11	Interleaver (I / Q):	28
2.2.12	interleaver stage:.....	29
2.2.13	Preamble and SFD ROM:.....	32
2.2.14	Form PPDU:	33
2.2.15	Parallel to serial convertor:.....	36
2.2.16	Symbol Mapping and Differential Encoding	38
2.2.17	Representation of QPSK and DQPSK symbols	38
2.2.18	QPSK Mapper.....	39
2.2.19	DQPSK Encoder	39
2.2.20	transmitter to data path top wrapper:.....	41
2.2.21	Fixed-Point Analysis and word length selection:.....	43
2.2.22	Module map — CSK modulation chain	46
2.2.23	CSS Symbol Generator (css_symbol_generator)	47
2.2.24	chirp_rom:	50
2.2.25	csk_generator:	61
2.2.26	complex_multiplier:	66
3	verification Methodology	68
3.1	Integration Testbench	69
3.1.1	Test Vectors	69
3.2	Chirp Sequence Fixed-Point Resolution	69
3.3	Simulation and Capture.....	69
3.4	Comparison Script	70
4	Accuracy Report: Fixed-Point vs. Floating-Point	71

4.1	Chirp Quantizer Resolution Sweep	71
4.2	Full-Chain Fixed-Point vs. Floating-Point MSE	72
5	Debugging Log: Issues Found and Fixed	74
6	Behavioral (RTL) Simulation Evidence	75
7	Future improvements	77
8	synthesis and Implementation	77
8.1	ASIC FLOW:	77
8.1.1	Stage 1 — Logic Synthesis.....	77
8.1.2	Stage 2 — Design Library Setup	78
8.1.3	Stage 3 — Floorplanning	78
8.1.4	Stage 4 — Power Plan	79
8.1.5	Stage 6 — Clock Tree Synthesis (CTS)	83
8.1.6	Final Outputs	87
8.1.7	Overall Status — Flow Complete	87
8.2	FPGA FLOW:	88
8.2.1	FPGA type:.....	88
8.2.2	Constraint file:	88
8.2.3	Resource Utilization & timings:.....	95
8.2.4	Results:	97
9	Summary.....	98
10	References	98
11	Appendix.....	99
11.1	Testbenches:	99

The IEEE 802.15.4 Chirp Spread Spectrum (CSS) Physical Layer (PHY) transmitter converts raw frame bytes supplied by the MAC layer into quantized real (Tx_real) and imaginary (Tx_imag) baseband signal samples. The hardware architecture operates using a top-down, modular digital signal processing (DSP) datapath synchronized by a central Finite State Machine (FSM) controller. Incoming payload data stored in an interface RAM is framed with headers, padded for alignment, demultiplexed into parallel streams, block-encoded via orthogonal code tables, permuted through interleaving, mapped to continuous QPSK phases, differentially encoded, and finally modulated using raised-cosine windowed chirp sequences.

2.1 Block Diagram & Architecture:



2.2 Design Breakdown:

2.2.1 Design Top Wrapper:

2.2.1.1 Purpose:

The ``css_tx_top`` module serves as the top-level structural wrapper for the complete IEEE 802.15.4 Chirp Spread Spectrum (CSS) PHY transmitter, integrating the full processing pipeline from raw memory-stored payload bytes to baseband complex I/Q output signals. It coordinates data flow across four primary sub-systems: reading payload bytes from a dedicated RAM block, passing them through ``css_tx_frontend`` for zero-padding, I/Q demuxing, and symbol mapping; using a central ``controller`` to manage pipeline flow, pacing, and rate-mismatch buffering; streaming I and Q codewords into ``interleaver_ppdu_top`` for rate-dependent interleaving, preamble insertion, and serialization; and finally driving ``tx_datapath_top`` to execute QPSK mapping, DQPSK differential encoding, and Chirp Sequential Keying (CSK) modulation to emit valid complex baseband samples (``tx_real``, ``tx_imag``).

2.2.1.2 Interface:

Port	Direction	Description
clk	input	System clock
reset	input	Synchronous, active-high reset
payload_we	input	Payload RAM write enable (MAC writes the PSDU before start)
payload_waddr	input [ADDR_WIDTH-1:0]	Payload RAM write address
payload_wdata	input [DATA_WIDTH-1:0]	Payload RAM write data
start_tx	input	1-cycle pulse: begin transmitting a packet
payload_length	input [6:0]	PSDU length in bytes
chirp_index	input [1:0]	CSK sequence select (m = 1-4 as 0-3)
tx_done	output	Whole PPDU fully transmitted
tx_real	output signed [OUT_WIDTH-1:0]	Final transmitted baseband real (I) sample
tx_imag	output signed [OUT_WIDTH-1:0]	Final transmitted baseband imaginary (Q) sample
tx_valid	output	tx_real/tx_imag valid this cycle
fifo_overflow	output	Status flag from the controller's chip FIFO; should never assert

2.2.1.3 RTL –css_tx_top.v:

```
`timescale 1ns/1ps

module css_tx_top #(
    parameter          DATA_RATE          = 0, //0 --> 1M | 1--> 250 kbps
    parameter integer  N_IN                = 3, //3 --> 1M | 6 --> 250 kbps
    parameter integer  M                  = 4, //4 --> 1M | 32 --> 250 kbps
    parameter integer  GROUP_SIZE         = 6, //6 --> 1M | 24 --> 250 kbps
    parameter integer  PREAMBLE_TOTAL_BITS = 48, //48 --> 1M | 96 --> 250 kbps
    parameter [15:0]   SFD                = 16'h749C, //16'h749C --> 1M | 16'h7A23 --> 250 kbps
    parameter integer  DATA_WIDTH        = 8,
    parameter integer  ADDR_WIDTH         = 7,
    parameter integer  NUM_SYMBOLS_WIDTH  = 12,
    parameter integer  ROM_WIDTH          = 5,
    parameter integer  OUT_WIDTH          = 6
)(
    input  wire          clk,
    input  wire          reset,

    input  wire          payload_we,
    input  wire [ADDR_WIDTH-1:0] payload_waddr,
    input  wire [DATA_WIDTH-1:0] payload_wdata,

    input  wire          start_tx,          // 1-cycle pulse
    input  wire [6:0]    payload_length,    // PSDU length in bytes
    input  wire [1:0]    chirp_index,       // CSK sequence select (m=1-4 as 0-3)
    output wire          tx_done,

    output wire signed [OUT_WIDTH-1:0] tx_real,
    output wire signed [OUT_WIDTH-1:0] tx_imag,
    output wire          tx_valid,

    output wire          fifo_overflow
);

wire [ADDR_WIDTH-1:0] payload_rd_addr;
wire [DATA_WIDTH-1:0] payload_rd_data;
wire [6:0] payload_length_reg ;

payload_ram #(
    .DATA_WIDTH (DATA_WIDTH),
    .ADDR_WIDTH (ADDR_WIDTH)
) u_payload_ram (
    .clk      (clk),
    .we       (payload_we),
    .waddr    (payload_waddr),
    .din      (payload_wdata),
    .raddr    (payload_rd_addr),
    .dout     (payload_rd_data)
);

wire          fe_load, fe_enable, fe_frame_done;
wire [M-1:0]  fe_i_codeword, fe_q_codeword;
wire          fe_i_codeword_valid, fe_q_codeword_valid;
wire [15:0]   padded_total_bits;
css_tx_frontend #(
    .GROUP_SIZE (GROUP_SIZE),
    .N_IN       (N_IN),
    .M_OUT      (M),
    .DATA_WIDTH (DATA_WIDTH),
    .ADDR_WIDTH (ADDR_WIDTH),
    .DATA_RATE  (DATA_RATE)
) u_frontend (
    .clk          (clk),
    .reset        (reset),
    .load         (fe_load),
    .enable       (fe_enable),
    .payload_length_reg (payload_length_reg),
```

```

        .payload_rd_data      (payload_rd_data),
        .payload_rd_addr     (payload_rd_addr),
        .frame_done          (fe_frame_done),
        .i_codeword           (fe_i_codeword),
        .i_codeword_valid     (fe_i_codeword_valid),
        .q_codeword           (fe_q_codeword),
        .q_codeword_valid     (fe_q_codeword_valid),
        .padded_total_bits    (padded_total_bits)
    );

    wire                                pf_start, pf_last_codeword;
    wire [M-1:0]                       pf_i_codeword, pf_q_codeword;
    wire                                pf_i_codeword_valid, pf_q_codeword_valid;
    wire                                pf_req_next_symbol, pf_chip_valid;
    wire                                pf_i_bit, pf_q_bit;
    wire                                dp_start, dp_symbol_req, dp_qpsk_valid;
    wire                                dp_i_bit, dp_q_bit;
    wire [NUM_SYMBOLS_WIDTH-1:0] num_symbols;

    controller #(
        .N_IN              (N_IN),
        .M                  (M),
        .GROUP_SIZE        (GROUP_SIZE),
        .PREAMBLE_TOTAL_BITS (PREAMBLE_TOTAL_BITS),
        .NUM_SYMBOLS_WIDTH  (NUM_SYMBOLS_WIDTH)
    ) u_controller (
        .clk                (clk),
        .reset              (reset),
        .start_tx           (start_tx),
        .payload_length      (payload_length),
        .payload_length_reg  (payload_length_reg),
        .num_symbols         (num_symbols),

        .fe_load            (fe_load),
        .fe_enable          (fe_enable),
        .fe_i_codeword       (fe_i_codeword),
        .fe_i_codeword_valid (fe_i_codeword_valid),
        .fe_q_codeword       (fe_q_codeword),
        .fe_q_codeword_valid (fe_q_codeword_valid),
        .fe_frame_done       (fe_frame_done),

        .pf_start           (pf_start),
        .pf_i_codeword       (pf_i_codeword),
        .pf_i_codeword_valid (pf_i_codeword_valid),
        .pf_q_codeword       (pf_q_codeword),
        .pf_q_codeword_valid (pf_q_codeword_valid),
        .pf_last_codeword    (pf_last_codeword),
        .pf_req_next_symbol  (pf_req_next_symbol),
        .pf_i_bit            (pf_i_bit),
        .pf_q_bit            (pf_q_bit),
        .pf_chip_valid       (pf_chip_valid),

        .dp_start           (dp_start),
        .dp_symbol_req       (dp_symbol_req),
        .dp_i_bit            (dp_i_bit),
        .dp_q_bit            (dp_q_bit),
        .dp_qpsk_valid       (dp_qpsk_valid),
        .padded_total_bits   (padded_total_bits),
        .fifo_overflow        (fifo_overflow)
    );

    interleaver_ppdu_top #(
        .M                  (M),
        .DATA_RATE           (DATA_RATE),
        .PREAMBLE_TOTAL_BITS (PREAMBLE_TOTAL_BITS),
        .SFD                 (SFD)
    ) u_interleaver_ppdu (
        .clk                (clk),
        .reset              (reset),

```

```

.start          (pf_start),
.i_codeword_in  (pf_i_codeword),
.i_codeword_valid (pf_i_codeword_valid),
.q_codeword_in  (pf_q_codeword),
.q_codeword_valid (pf_q_codeword_valid),
.last_codeword  (pf_last_codeword),
.req_next_symbol (pf_req_next_symbol),
.i_bit          (pf_i_bit),
.q_bit          (pf_q_bit),
.chip_valid     (pf_chip_valid)
);

tx_datapath_top #(
    .NUM_SYMBOLS_WIDTH (NUM_SYMBOLS_WIDTH),
    .ROM_WIDTH          (ROM_WIDTH),
    .OUT_WIDTH          (OUT_WIDTH)
) u_tx_datapath (
    .clk      (clk),
    .reset    (reset),
    .start    (dp_start),
    .chirp_index (chirp_index),
    .num_symbols (num_symbols),
    .tx_done    (tx_done),
    .i_bit      (dp_i_bit),
    .q_bit      (dp_q_bit),
    .qpsk_valid (dp_qpsk_valid),
    .symbol_req (dp_symbol_req),
    .tx_real    (tx_real),
    .tx_imag    (tx_imag),
    .tx_valid   (tx_valid)
);

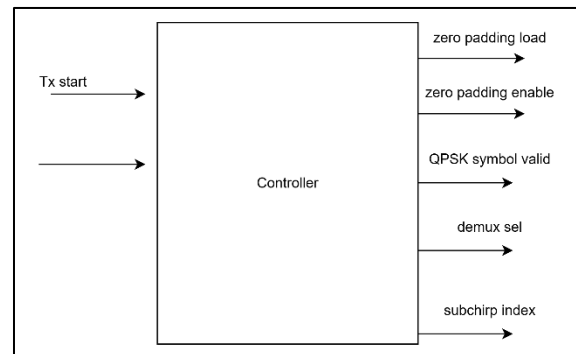
endmodule

```

2.2.2 System Controller (Controller):

2.2.2.1 Purpose:

The module operates as an FSM managing packet transmission lifecycle. Upon receiving the Tx start pulse, it generates zero padding load to initialize header insertion and byte reading from payload memory. It maintains active execution via zero padding enable while toggling demux sel to demultiplex bits into even and odd paths. Throughout execution, it generates QPSK symbol valid strobes to push mapped symbols into the differential DQPSK encoder, configures subchirp index for baseband chirp synthesis, and terminates operations when zero padding frame done is asserted.



2.2.2.2 Interface:

Port	Direction	Description
clk	input	System clock
reset	input	Synchronous, active-high reset
start_tx	input	1-cycle pulse: begin a PPDU
payload_length	input [6:0]	Payload length in bytes
num_symbols	output [NUM_SYMBOLS_WIDTH-1:0]	Total symbols for this packet, computed combinationaly at start
payload_length_reg	output [6:0]	Registered copy of payload_length, latched at start_tx
fe_load	output	Load pulse to css_tx_frontend (= start_tx)
fe_enable	output	Enable to css_tx_frontend (asserted while the codeword buffer has room)
fe_i_codeword	input [M-1:0]	I codeword from css_tx_frontend
fe_i_codeword_valid	input	Valid for fe_i_codeword
fe_q_codeword	input [M-1:0]	Q codeword from css_tx_frontend
fe_q_codeword_valid	input	Valid for fe_q_codeword
fe_frame_done	input	Frame-done flag from css_tx_frontend
padded_total_bits	input [15:0]	Padded bit count from css_tx_frontend

2.2.2.3 RTL — controller.v:

```
//=====
`timescale 1ns/1ps

module controller #(
    parameter integer N_IN          = 3,    // 3 (1Mbps) / 6 (250kbps)
    parameter integer M            = 4,    // 4 (1Mbps) / 32 (250kbps)
    parameter integer GROUP_SIZE   = 6,    // 6 (1Mbps) / 24 (250kbps)
    parameter integer PREAMBLE_TOTAL_BITS = 48, // 48 (1Mbps) / 96 (250kbps)
    parameter integer NUM_SYMBOLS_WIDTH = 12,
    parameter integer FIFO_DEPTH    = 256,
    parameter integer FIFO_AW       = 8    // $clog2(FIFO_DEPTH)
)(
    input wire      clk,
    input wire      reset,

    // packet-level control
    input wire      start_tx,              // 1-cycle pulse: begin a PPDU
    input wire [6:0] payload_length,       // payload length in bytes
    output wire [NUM_SYMBOLS_WIDTH-1:0] num_symbols, // total symbols for this packet
    output reg [6:0] payload_length_reg,

    // to / from css_tx_frontend
    output wire      fe_load,
    output wire      fe_enable,
    input wire [M-1:0] fe_i_codeword,
    input wire      fe_i_codeword_valid,
    input wire [M-1:0] fe_q_codeword,
    input wire      fe_q_codeword_valid,
    input wire      fe_frame_done,
    input wire [15:0] padded_total_bits,

    // to / from interleaver_ppdu_top
    output wire      pf_start,
    output reg [M-1:0] pf_i_codeword,
    output reg      pf_i_codeword_valid,
    output reg [M-1:0] pf_q_codeword,
    output reg      pf_q_codeword_valid,
    output reg      pf_last_codeword,
    input wire      pf_req_next_symbol,
    input wire      pf_i_bit,
    input wire      pf_q_bit,
    input wire      pf_chip_valid,

    // to / from tx_datapath_top
    output wire      dp_start,
    input wire      dp_symbol_req,
    output reg      dp_i_bit,
    output reg      dp_q_bit,
    output reg      dp_qpsk_valid,

    // status
    output reg      fifo_overflow // verification aid; should never assert
);

localparam integer PHR_BITS = 12;

wire [15:0] total_bits_now      = PHR_BITS + ({9'd0, payload_length} << 3);
wire [15:0] remainder_now      = total_bits_now % GROUP_SIZE;
wire [15:0] pad_bits_now       = GROUP_SIZE - remainder_now;
wire [15:0] padded_total_bits_now = total_bits_now + pad_bits_now;

wire [15:0] codeword_pairs     = padded_total_bits_now / (2*N_IN);
wire [15:0] payload_chips      = codeword_pairs * M;
wire [15:0] total_symbols      = PREAMBLE_TOTAL_BITS + payload_chips;
```

```

assign num_symbols = total_symbols[NUM_SYMBOLS_WIDTH-1:0];

assign fe_load = start_tx;
assign pf_start = start_tx;
assign dp_start = start_tx;

localparam integer CW_DEPTH = 4;

reg [M-1:0] cw_i    [0:CW_DEPTH-1];
reg [M-1:0] cw_q    [0:CW_DEPTH-1];
reg          cw_last[0:CW_DEPTH-1];
reg [2:0]    cw_wptr, cw_rptr, cw_count;

wire cw_full  = (cw_count == CW_DEPTH);
wire cw_empty = (cw_count == 3'd0);

// Frontend advances only while there is room to accept what it makes,
// and only while the packet is live and the frontend has not finished.
reg fe_done_latched;
assign fe_enable = !cw_full ;

always @(posedge clk) begin
    if (reset)          fe_done_latched <= 1'b0;
    else if (start_tx)   fe_done_latched <= 1'b0;
    else if (fe_frame_done && fe_enable) fe_done_latched <= 1'b1;
end

reg [1:0]          chip_mem [0:FIFO_DEPTH-1];
reg [FIFO_AW-1:0] chip_wptr, chip_rptr;
reg [FIFO_AW:0]    chip_count;

wire chip_fifo_empty = (chip_count == 0);
wire [FIFO_AW:0] chip_space = FIFO_DEPTH[FIFO_AW:0] - chip_count;

// Release a codeword pair only when the FIFO can absorb the whole
// codeword the p2s shifters will then emit back-to-back.
wire chip_room_for_codeword = (chip_space > M);

reg pf_req_pending;

reg [M-1:0] hold_i, hold_q;
reg          have_i, have_q;
reg          hold_last;

wire i_avail = have_i || fe_i_codeword_valid;
wire q_avail = have_q || fe_q_codeword_valid;

wire [M-1:0] push_i = have_i ? hold_i : fe_i_codeword;
wire [M-1:0] push_q = have_q ? hold_q : fe_q_codeword;
wire          push_last = hold_last || fe_frame_done || fe_done_latched;

wire cw_push = i_avail && q_avail && !cw_full;
wire cw_pop  = pf_req_pending && !cw_empty && chip_room_for_codeword;

always @(posedge clk) begin
    if (reset) begin
        have_i <= 1'b0; have_q <= 1'b0; hold_last <= 1'b0;
        hold_i <= {M{1'b0}}; hold_q <= {M{1'b0}};
        payload_length_reg <= 7'd0;
    end
    else if (start_tx) begin
        have_i <= 1'b0; have_q <= 1'b0; hold_last <= 1'b0;
        hold_i <= {M{1'b0}}; hold_q <= {M{1'b0}};
        payload_length_reg <= payload_length;
    end
    else if (cw_push) begin
        have_i <= 1'b0; have_q <= 1'b0; hold_last <= 1'b0;
    end
    else begin
        if (fe_i_codeword_valid) begin hold_i <= fe_i_codeword; have_i <= 1'b1; end
        if (fe_q_codeword_valid) begin hold_q <= fe_q_codeword; have_q <= 1'b1; end
        if (fe_frame_done || fe_done_latched) hold_last <= 1'b1;
    end
end

```

```

end
end

always @(posedge clk) begin
    if (reset )      pf_req_pending <= 1'b0;
    else if (start_tx) pf_req_pending <= 1'b0;
    else if (pf_req_next_symbol) pf_req_pending <= 1'b1;
    else if (cw_pop)      pf_req_pending <= 1'b0;
end

integer idx ;
always @(posedge clk) begin
    if (reset) begin
        cw_wptr <= 3'd0;  cw_rptr <= 3'd0;  cw_count <= 3'd0;
        pf_i_codeword <= {M{1'b0}};
        pf_q_codeword <= {M{1'b0}};
        pf_i_codeword_valid <= 1'b0;
        pf_q_codeword_valid <= 1'b0;
        pf_last_codeword <= 1'b0;
        for (idx = 0; idx < CW_DEPTH; idx = idx + 1) begin
            cw_i[idx] <= {M{1'b0}};
            cw_q[idx] <= {M{1'b0}};
            cw_last[idx] <= 1'b0;
        end
    end
    else if (start_tx) begin
        cw_wptr <= 3'd0;  cw_rptr <= 3'd0;  cw_count <= 3'd0;
        pf_i_codeword <= {M{1'b0}};
        pf_q_codeword <= {M{1'b0}};
        pf_i_codeword_valid <= 1'b0;
        pf_q_codeword_valid <= 1'b0;
        pf_last_codeword <= 1'b0;
    end
    else begin
        pf_i_codeword_valid <= 1'b0;
        pf_q_codeword_valid <= 1'b0;

        if (cw_push) begin
            cw_i [cw_wptr[1:0]] <= push_i;
            cw_q [cw_wptr[1:0]] <= push_q;
            cw_last[cw_wptr[1:0]] <= push_last;
            cw_wptr <= cw_wptr + 3'd1;
        end

        if (cw_pop) begin
            pf_i_codeword <= cw_i [cw_rptr[1:0]];
            pf_q_codeword <= cw_q [cw_rptr[1:0]];
            pf_last_codeword <= cw_last[cw_rptr[1:0]];
            pf_i_codeword_valid <= 1'b1;
            pf_q_codeword_valid <= 1'b1;
            cw_rptr <= cw_rptr + 3'd1;
        end

        case ({cw_push, cw_pop})
            2'b10: cw_count <= cw_count + 3'd1;
            2'b01: cw_count <= cw_count - 3'd1;
            default: cw_count <= cw_count; // 00 and 11 leave it unchanged
        endcase
    end
end

wire chip_push = pf_chip_valid;
wire chip_pop  = dp_symbol_req && !chip_fifo_empty;

always @(posedge clk) begin
    if (reset) begin
        chip_wptr <= {FIFO_AW{1'b0}};
        chip_rptr <= {FIFO_AW{1'b0}};
        chip_count <= {(FIFO_AW+1){1'b0}};
        dp_i_bit <= 1'b0;
        dp_q_bit <= 1'b0;
    end
end

```

```

        dp_qpsk_valid <= 1'b0;
        fifo_overflow <= 1'b0;
    end else if (start_tx) begin
        chip_wptra <= {FIFO_AW{1'b0}};
        chip_rptra <= {FIFO_AW{1'b0}};
        chip_count <= {(FIFO_AW+1){1'b0}};
        dp_i_bit <= 1'b0;
        dp_q_bit <= 1'b0;
        dp_qpsk_valid <= 1'b0;
        fifo_overflow <= 1'b0;
    end else begin
        dp_qpsk_valid <= 1'b0;

        if (chip_push && (chip_count == FIFO_DEPTH))
            fifo_overflow <= 1'b1; // sticky; verification aid only

        if (chip_push && !chip_pop) begin
            chip_mem[chip_wptra] <= {pf_i_bit, pf_q_bit};
            chip_wptra <= chip_wptra + 1'b1;
            chip_count <= chip_count + 1'b1;
        end else if (!chip_push && chip_pop) begin
            {dp_i_bit, dp_q_bit} <= chip_mem[chip_rptra];
            dp_qpsk_valid <= 1'b1;
            chip_rptra <= chip_rptra + 1'b1;
            chip_count <= chip_count - 1'b1;
        end else if (chip_push && chip_pop) begin
            chip_mem[chip_wptra] <= {pf_i_bit, pf_q_bit};
            chip_wptra <= chip_wptra + 1'b1;
            {dp_i_bit, dp_q_bit} <= chip_mem[chip_rptra];
            dp_qpsk_valid <= 1'b1;
            chip_rptra <= chip_rptra + 1'b1;
            // count unchanged
        end
    end
end
end
endmodule

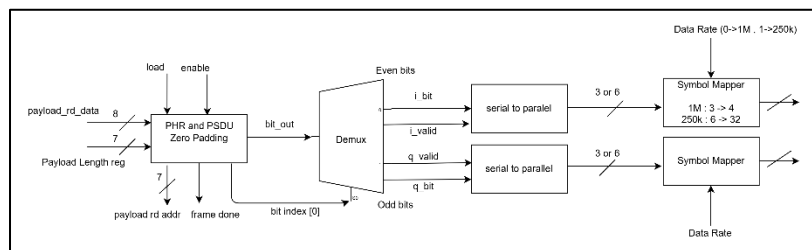
```

2.2.3 CSS transmitter frontend:

2.2.3.1 Purpose:

The `css_tx_frontend` module serves as the top-level transmitter frontend for the CSS PHY, driving the initial data-processing pipeline that transforms raw payload bytes from memory into mapped I

and Q channel codewords. It begins by passing payload bytes through a `zero_padding` block to attach any required padding bits based on frame length, streaming out individual bits alongside address read controls. These serial bits are split into In-phase (I) and Quadrature (Q) streams via a `demux_iq` module, with alignment registers (`sel_d`, `valid_d`) compensating for memory lookup delays. Each stream is then converted into `Nin` - bit groups using dual `serial_to_parallel` modules before feeding parallel `symbol_mapper` blocks. Ultimately, these mappers translate the bit groups into `Mout` - bit I and Q



codewords (i_codeword, q_codeword) ready for downstream interleaving and frame assembly.

2.2.3.2 Interface:

Port	Direction	Description
clk	input	System clock
reset	input	Synchronous, active-high reset
load	input	Pulse: reload for a new PPDU (forwarded to zero_padding)
enable	input	Advance the padded bitstream by 1 each cycle while high
payload_length_reg	input [6:0]	PHR+PSDU bit-count basis, forwarded to zero_padding
payload_rd_data	input [DATA_WIDTH-1:0]	Payload byte read from payload_ram
payload_rd_addr	output [ADDR_WIDTH-1:0]	Read address into payload_ram
frame_done	output	High when the padded frame's last bit has been reached
i_codeword	output [M_OUT-1:0]	Symbol-mapped codeword, I path
i_codeword_valid	output	Valid for i_codeword
q_codeword	output [M_OUT-1:0]	Symbol-mapped codeword, Q path
q_codeword_valid	output	Valid for q_codeword
padded_total_bits	output [15:0]	Total padded bit count for the current frame

2.2.3.3 RTL –css_tx_frontend.v:

```
module css_tx_frontend #(
    parameter GROUP_SIZE = 6,                // 6 for 1Mbps, 24 for 250kbps
    parameter N_IN       = 3,                // 3 for 1Mbps, 6 for 250kbps
    parameter M_OUT      = 4,                // 4 for 1Mbps, 32 for 250kbps
    parameter DATA_WIDTH = 8,
    parameter ADDR_WIDTH = 7,
    parameter DATA_RATE = 1'b0
) (
    input wire clk,
    input wire reset,
    input wire load,
    input wire enable,
    input wire [6:0] payload_length_reg,
    input wire [DATA_WIDTH-1:0] payload_rd_data,

    output wire [ADDR_WIDTH-1:0] payload_rd_addr,
    output wire frame_done,

    // Symbol Mapper Outputs
    output wire [M_OUT-1:0] i_codeword,
    output wire i_codeword_valid,
    output wire [M_OUT-1:0] q_codeword,
    output wire q_codeword_valid,
    output wire [15:0] padded_total_bits
);

// 1. Zero Padding Block
wire zp_bit_index_sel;
wire zp_bit_out;
wire [1:0] zp_state;

zero_padding #(
    .GROUP_SIZE(GROUP_SIZE),
    .DATA_WIDTH(DATA_WIDTH),
    .ADDR_WIDTH(ADDR_WIDTH)
) u_zero_padding (
    .clk (clk),
    .reset (reset),
    .load (load),
    .enable (enable),
    .payload_length_reg (payload_length_reg),
    .padded_total_bits (padded_total_bits),
    .payload_rd_data (payload_rd_data),
    .payload_rd_addr (payload_rd_addr),
    .bit_index_sel (zp_bit_index_sel),
    .bit_out (zp_bit_out),
    .state (zp_state),
    .frame_done (frame_done)
);

// Pipeline Alignment Logic:
// Align bit_index[0] and enable with payload_bit_in 1-cycle register delay
reg sel_d;
reg valid_d;

always @(posedge clk) begin
    if (reset) begin
        sel_d <= 1'b0;
        valid_d <= 1'b0;
    end else begin
        sel_d <= zp_bit_index_sel;
    end
end
```

```

        valid_d <= enable && (zp_state != 2'b00);
    end
end

// 2. Demux I/Q Block
wire i_bit, q_bit;
wire i_valid_raw, q_valid_raw;

demux_iq u_demux_iq (
    .bit_in  (zp_bit_out),
    .sel     (sel_d),
    .i_bit   (i_bit),
    .q_bit   (q_bit),
    .i_valid (i_valid_raw),
    .q_valid (q_valid_raw)
);

wire i_valid = valid_d && i_valid_raw;
wire q_valid = valid_d && q_valid_raw;

// 3. Serial to Parallel Blocks (I & Q Paths)
wire [N_IN-1:0] i_group, q_group;
wire            i_group_valid, q_group_valid;

serial_to_parallel #(N(N_IN)) u_s2p_i (
    .clk      (clk),
    .reset    (reset),
    .start    (load),
    .valid_in (i_valid),
    .bit_in   (i_bit),
    .data_out (i_group),
    .valid_out (i_group_valid)
);

serial_to_parallel #(N(N_IN)) u_s2p_q (
    .clk      (clk),
    .reset    (reset),
    .start    (load),
    .valid_in (q_valid),
    .bit_in   (q_bit),
    .data_out (q_group),
    .valid_out (q_group_valid)
);

// 4. Symbol Mapper Blocks (I & Q Paths)
symbol_mapper #(
    .N_IN    (N_IN),
    .M_OUT   (M_OUT),
    .DATA_RATE(DATA_RATE)
) u_mapper_i (
    .clk      (clk),
    .group_in (i_group),
    .group_valid (i_group_valid),
    .codeword_out (i_codeword),
    .codeword_valid (i_codeword_valid)
);

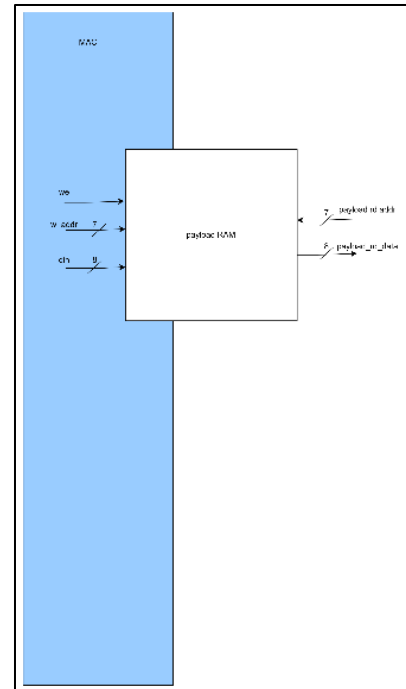
symbol_mapper #(
    .N_IN    (N_IN),
    .M_OUT   (M_OUT),
    .DATA_RATE(DATA_RATE)
) u_mapper_q (
    .clk      (clk),
    .group_in (q_group),
    .group_valid (q_group_valid),
    .codeword_out (q_codeword),
    .codeword_valid (q_codeword_valid)
);
endmodule

```

2.2.4 Payload RAM:

2.2.4.1 Purpose:

Serves as the primary shared memory buffer between the host MAC layer and the PHY hardware pipeline, supporting asynchronous pre-loading of payload data bytes via standard memory controls prior to transmission initiation and outputting stored payload bytes sequentially during active transmission cycles. The central Controller implements the FSM that orchestrates overall packet transmission flow upon assertion of the Tx_start control pulse, generating control handshaking signals across individual sub-modules while managing state transitions across preamble generation, header insertion, payload encoding, chirp sequencing, and completion reporting. Also, it supports asynchronous pre-loading of payload data bytes via standard memory controls (payload_w_data, w_addr, we) prior to transmission initiation and outputs stored payload bytes (payload_r_data) sequentially to the framing logic during active transmission cycles.



2.2.4.2 Interface:

Port	Direction	Description
clk	input	System clock
we	input	Write enable
waddr	input [ADDR_WIDTH-1:0]	Write address
din	input [DATA_WIDTH-1:0]	Write data byte
raddr	input [ADDR_WIDTH-1:0]	Read address (driven by the Framer)
dout	output [DATA_WIDTH-1:0]	Read data byte (registered, to the Framer)

2.2.4.3 RTL – payload_ram.v:

```
module payload_ram #(
    parameter DATA_WIDTH = 8,
    parameter ADDR_WIDTH = 7 // 2^7 = 128 bytes maximum PSDU capacity
)(
    input wire clk,

    // Write Interface (Payload Loading)
    input wire we, // Write enable
    input wire [ADDR_WIDTH-1:0] waddr, // Write address
    input wire [DATA_WIDTH-1:0] din, // Write data byte

    // Read Interface (Connected to Framer)
    input wire [ADDR_WIDTH-1:0] raddr, // Read address (ram_addr from Framer)
    output reg [DATA_WIDTH-1:0] dout // Read data byte (ram_din to Framer)
);

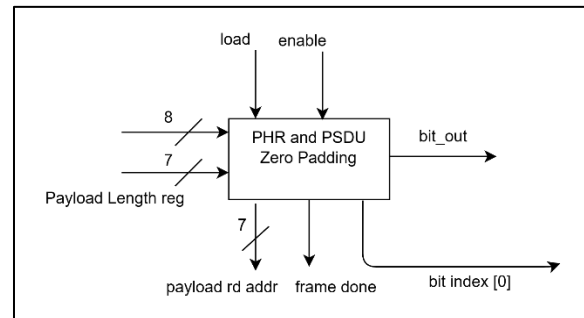
// 128 x 8-bit RAM Memory Array
reg [DATA_WIDTH-1:0] ram [0:(1<<ADDR_WIDTH)-1];

// Synchronous Memory Read & Write Operations
always @(posedge clk) begin
    if (we) begin
        ram[waddr] <= din;
    end
    dout <= ram[raddr];
end
endmodule
```

2.2.5 PHR and PSDU Zero Padding:

2.2.5.1 Purpose:

The PHR and PSDU Zero Padding module generates the PHY Header from input frame length information, prepends it to the raw payload bitstream, and appends trailing zero-padding bits to align the total frame length with required CSS symbol block boundaries. Following framing, the Demux de-interleaves the unified serial bitstream into separate In-phase and Quadrature processing channels with associated data valid indicators. These bitwise streams pass into the Serial to Parallel converters, which group the incoming bits into parallel symbol vectors required to index downstream block encoders.



This module receives payload bytes and length configurations to construct the transmitter's framed bitstream. It prepends the PHR header, outputs the payload bits, and appends zero-padding bits to satisfy the symbol alignment requirements of the CSS specification ($N = 6$ for 1 Mbps, $N = 24$ for 250 kbps). It outputs a continuous serial stream (`bit_out`), tracks the bit count using an internal index generator, manages memory fetch addresses via `payload rd addr`, and asserts `frame done` upon transmission completion.

2.2.5.2 Interface:

Port	Direction	Description
clk	input	System clock
reset	input	Synchronous, active-high reset
load	input	Pulse: reload bit_index to 0 for a new PPDU
enable	input	Advance bit_index by 1 each cycle while high
payload_length_reg	input [6:0]	PHR(12) + PSDU(8×payloadLength) bit count
padded_total_bits	output [15:0]	total_bits + pad_bits (rounded up to a multiple of GROUP_SIZE)
payload_rd_data	input [DATA_WIDTH-1:0]	Payload byte read from payload_ram
payload_rd_addr	output [ADDR_WIDTH-1:0]	Read address into payload_ram
bit_index_sel	output	LSB of bit_index (0-based position within the padded stream)
bit_out	output	Payload bit, or 0 inside the padding region
state	output [1:0]	Current FSM state (IDLE/PHR/PSDU), exposed to CSS TX frontend
frame_done	output	High when bit_index is on the last bit of the padded frame

2.2.5.3 RTL –zero_padding.v:

```

module zero_padding #(
    parameter GROUP_SIZE = 6 ,      // 6 for 1 Mbps, 24 for 250 kbps
    parameter DATA_WIDTH = 8 ,
    parameter ADDR_WIDTH = 7
) (
    input wire      clk,
    input wire      reset,          // synchronous, active-high
    input wire      load,           // pulse: reload bit_index to 0 for a new PPDU
    input wire      enable,         // advance bit_index by 1 each cycle while high

    input wire [6:0] payload_length_reg, // PHR(12) + PSDU(8*payloadLength) bit count
    output wire [15:0] padded_total_bits, // total_bits + pad_bits (multiple of GROUP_SIZE)

    input wire [DATA_WIDTH-1:0] payload_rd_data,
    output reg [ADDR_WIDTH-1:0] payload_rd_addr,

    output wire bit_index_sel,      // 0-based position within the padded stream --
                                    // value of PHR/PSDU bit `bit_index` (only meaningful
                                    // when bit_index < total_bits; framer/RAM supplies it)

    output wire      bit_out,       // payload bit, or 0 inside the padding region

```

```

output reg [1:0] state, //output for css tx frontend
output wire frame_done // high when bit_index is on the last bit of the padded frame
);

//PHR(12) + PSDU(8*payloadLength) bit count
reg [15:0] bit_index ;
wire [4:0] pad_bits; // 0 .. GROUP_SIZE-1 zero bits appended //max no of 0's is 23 for the
250k mode
wire [15:0] total_bits ;
assign total_bits = 16'd12 + ({9'd0, payload_length_reg} << 3); //12 bit phy header + (no bytes * 8 )

wire [15:0] remainder;
// assign remainder = total_bits - (GROUP_SIZE * (total_bits / GROUP_SIZE));
assign remainder = total_bits % GROUP_SIZE ; //?synthesizable

assign pad_bits = GROUP_SIZE - remainder;

assign padded_total_bits = total_bits + pad_bits;

// bit_index sequencing: registered counter, walks
// 0 .. padded_total_bits-1, one step per cycle while enabled.

reg payload_bit_in ;
reg [2:0] word_idx ;
wire [11:0] phy_header ;

assign phy_header = {3'b000 ,2'b00 , payload_length_reg} ; //0->6 payload length , 7->8 not used , 9-
>11 reserved

localparam [1:0] IDLE = 2'b00,
                PHR = 2'b01,
                PSDU = 2'b11;

reg [1:0] next_state ;

always @(posedge clk) begin
    if (reset) begin
        state <= IDLE ;
    end
    else begin
        state <= next_state ;
    end
end

always @(*) begin
    next_state = state ;
    case(state)
        IDLE: begin
            if (load) begin
                next_state = PHR ;
            end
        end
        PHR: begin
            if (bit_index == 'd11) begin
                next_state = PSDU ;
            end
        end
        PSDU: begin
            if (frame_done) begin
                next_state = IDLE ;
            end
        end
        default: begin
            next_state = state ;
        end
    endcase
endcase

```

```

end

always @(posedge clk) begin
    if (reset) begin
        bit_index <= 16'd0;
        payload_bit_in <= 1'b0 ;
        payload_rd_addr <= 'b0 ;
        word_idx <= 1'b0 ;
    end
    else begin
        case (state)
            IDLE: begin
                bit_index <= 16'd0;
                payload_bit_in <= 1'b0 ;
                payload_rd_addr <= 'b0 ;
                word_idx <= 1'b0 ;
            end

            PHR: begin
                if(enable) begin
                    bit_index <= bit_index + 16'd1;
                    payload_bit_in <= phy_header[bit_index] ;
                end
            end

            PSDU: begin
                if (enable && !frame_done) begin
                    bit_index <= bit_index + 16'd1;
                    word_idx <= word_idx + 1'b1 ;
                    payload_bit_in <= payload_rd_data[word_idx];
                    if(word_idx == 3'd6 ) begin // end of byte
                        payload_rd_addr <= payload_rd_addr + 1'b1 ;
                    end
                end
            end
        endcase
    end
end

assign frame_done = (bit_index == (padded_total_bits - 16'd1)); //? -1 ?

reg [15:0] bit_index_d ;
always @(posedge clk) begin
    if (reset)
        bit_index_d <= 16'd0;
    else
        bit_index_d <= bit_index;
end

assign bit_out = (bit_index_d < total_bits) ? payload_bit_in : 1'b0;

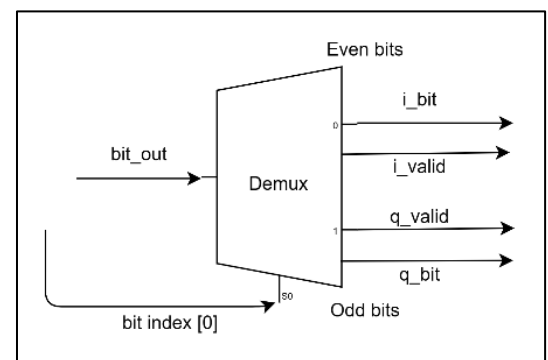
assign bit_index_sel = bit_index[0];
endmodule

```

2.2.6 DEMUX:

2.2.6.1 Purpose:

The 1-to-2 demultiplexer uses the least significant bit of the frame bit counter (bit index [0]) as its select line (S0). Even-indexed bits (S0 = 0) are routed to the upper path as i_bit with an associated i_valid strobe, while odd-indexed bits (S0 = 1) are routed to the lower path as q_bit with q_valid.



2.2.6.2 Interface:

Port	Direction	Description
bit_in	input	Padded-stream bit value at bit_index (from zero_padding)
sel	input	Bit index [0], selects I vs Q path
i_bit	output	= bit_in, present only when i_valid
q_bit	output	= bit_in, present only when q_valid
i_valid	output	1 when bit_index is even (bit belongs to I path)
q_valid	output	1 when bit_index is odd (bit belongs to Q path)

2.2.6.3 RTL –demux_iq.v:

```
`timescale 1ns/1ps

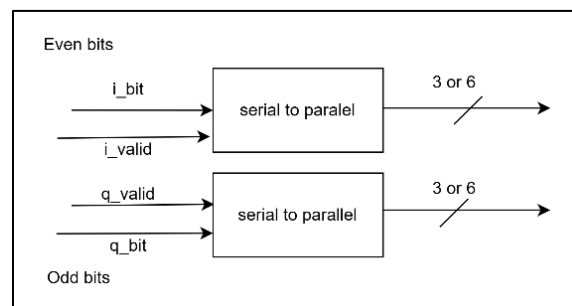
module demux_iq (
    input wire bit_in,
    input wire sel,           //bit index [0]
    output wire i_bit,        // = bit_in, present only when i_valid
    output wire q_bit,        // = bit_in, present only when q_valid
    output wire i_valid,      // 1 when bit_index is even (this bit belongs to I path)
    output wire q_valid       // 1 when bit_index is odd (this bit belongs to Q path)
);
    assign i_valid = ~sel;
    assign q_valid = sel;

    assign i_bit = bit_in;
    assign q_bit = bit_in;
endmodule
```

2.2.7 Serial to Parallel Converters:

2.2.7.1 Purpose:

Two identical units collect the de-interleaved serial bitstreams (i_bit and q_bit) under the control of their respective valid flags. Once a complete symbol's worth of bits is accumulated (3 bits for 1 Mbps mode or 6 bits for 250 kbps mode), the module outputs the parallel symbol vector to feed the downstream Walsh–Hadamard Symbol Mappers.



2.2.8 Interface:

Port	Direction	Description
clk	input	System clock
reset	input	Synchronous, active-high reset
start	input	Pulse to clear counters for a new frame
valid_in	input	High when bit_in is valid
bit_in	input	Serial input bit
data_out	output [N-1:0]	N-bit parallel output word
valid_out	output	Active for 1 cycle when a full N-bit word is ready

2.2.8.1 RTL –serial_to_parallel.v:

```
module serial_to_parallel #(
    parameter N = 3                                // Parallel word width (3 for 1Mbps sub-chirp, 6 for 250kbps)
) (
    input wire      clk,
    input wire      reset,                          // Synchronous active-high reset
    input wire      start,                          //?maybe i would need to delay this 1 cycle // Pulse to clear
counters for a new frame (e.g., connected to 'load')
    input wire      valid_in,                       // High when bit_in is valid (e.g., enable & i_valid / q_valid)
    input wire      bit_in,                         // Serial input bit

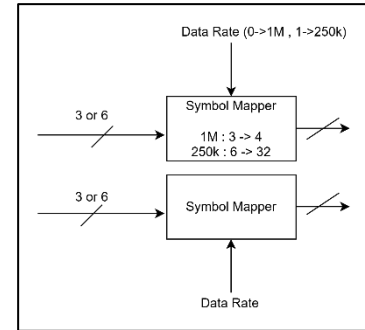
    output reg [N-1:0] data_out,                    // N-bit parallel output
    output reg      valid_out                       // Active for 1 cycle when a full N-bit word is ready
);

reg [$clog2(N):0] bit_cnt;
reg [N-1:0]      shift_reg;
always @(posedge clk) begin
    if (reset) begin
        bit_cnt  <= 'd0;
        shift_reg <= 'd0;
        data_out  <= 'd0;
        valid_out <= 1'b0;
    end
    else if (start) begin
        bit_cnt  <= 'd0;
        shift_reg <= 'd0;
        data_out  <= 'd0;
        valid_out <= 1'b0;
    end
    else begin
        valid_out <= 1'b0;
        if (valid_in) begin
            // Shift incoming bit in from MSB side (LSB first)
            ////////////////raghad fix 5////////////////////////
            //shift_reg <= {bit_in, shift_reg[N-1:1]};
            shift_reg <= {shift_reg[N-2:0], bit_in};
            if (bit_cnt == N - 1) begin
                bit_cnt  <= 'd0;
                //data_out <= {bit_in, shift_reg[N-1:1]};
                data_out  <= {shift_reg[N-2:0], bit_in};
                valid_out <= 1'b1;
            end
            else begin
                bit_cnt  <= bit_cnt + 1'b1;
            end
        end
    end
end
end
endmodule
```

2.2.9 Symbol Mappers (I / Q):

2.2.9.1 Purpose:

Two parallel ROM-based modules perform orthogonal Walsh–Hadamard block encoding independently on the In-phase and Quadrature paths. Depending on the Data Rate configuration, each unit maps an input vector of 3 bits into a 4-bit codeword (1 Mbps mode) or an input vector of 6 bits into a 32-bit codeword (250 kbps mode) in accordance with standard lookup tables.



2.2.9.2 Interface:

Port	Direction	Description
clk	input	System clock
group_in	input [N_IN-1:0]	n-bit input group (binary index) from serializer
group_valid	input	Input valid from serializer
codeword_out	output [M_OUT-1:0]	m-bit codeword ("chips") looked up from the ROM table
codeword_valid	output	Output valid, to interleaver

2.2.9.3 RTL –symbol_mapper.v:

```
`timescale 1ns/1ps

module symbol_mapper #(
    parameter DATA_RATE = 1'b0,          // 0 = 1Mbps, 1 = 250kbps
    parameter N_IN       = 3,              // input group width (3 or 6)
    parameter M_OUT      = 4,              // output codeword width (4 or 32)
    parameter NUM_WORDS  = (1 << N_IN)    // 8 or 64
) (
    input          clk ,
    input wire [N_IN-1:0] group_in, // n-bit input group (binary index)
    input wire      group_valid , //input from serilizer
    output reg [M_OUT-1:0] codeword_out ,
    output reg      codeword_valid // output to interleaver
);

    reg [M_OUT-1:0] rom ;

    generate
        if (DATA_RATE == 1'b0) begin : gen_rom_1mbps
            always @(*) begin
                case (group_in)
                    3'd0: rom = 4'b1111; // +1 +1 +1 +1
                    3'd1: rom = 4'b0101; // +1 -1 +1 -1
                    3'd2: rom = 4'b0011; // +1 +1 -1 -1
                    3'd3: rom = 4'b1001; // +1 -1 -1 +1
                    3'd4: rom = 4'b0000; // -1 -1 -1 -1
                    3'd5: rom = 4'b1010; // -1 +1 -1 +1
                    3'd6: rom = 4'b1100; // -1 -1 +1 +1
                    3'd7: rom = 4'b0110; // -1 +1 +1 -1
                    default: rom = 4'b0000;
                endcase
            end
        end
    end
```

```

end
else begin : gen_rom_250kbps
  always @(*) begin
    case (group_in)
      6'd0: rom = 32'b00000000000000000000000000000000;
      6'd1: rom = 32'b00000100000100000100000100000101;
      6'd2: rom = 32'b00001000001000001000001000001010;
      6'd3: rom = 32'b00001100001100001100001100001111;
      6'd4: rom = 32'b00010000010000010000010000010000;
      6'd5: rom = 32'b00010100010100010100010100010101;
      6'd6: rom = 32'b00011000011000011000011000011010;
      6'd7: rom = 32'b00011100011100011100011100011111;
      6'd8: rom = 32'b00100000100000100000100000100000;
      6'd9: rom = 32'b00100100100100100100100100100101;
      6'd10: rom = 32'b00101000101000101000101000101010;
      6'd11: rom = 32'b00101100101100101100101100101111;
      6'd12: rom = 32'b00110000110000110000110000110000;
      6'd13: rom = 32'b00110100110100110100110100110101;
      6'd14: rom = 32'b00111000111000111000111000111010;
      6'd15: rom = 32'b00111100111100111100111100111111;
      6'd16: rom = 32'b01000001000001000001000001000000;
      6'd17: rom = 32'b01000101000101000101000101000101;
      6'd18: rom = 32'b01001001001001001001001001001010;
      6'd19: rom = 32'b01001101001101001101001101001111;
      6'd20: rom = 32'b01010001010001010001010001010000;
      6'd21: rom = 32'b01010101010101010101010101010101;
      6'd22: rom = 32'b01011001011001011001011001011010;
      6'd23: rom = 32'b01011101011101011101011101011111;
      6'd24: rom = 32'b01100001100001100001100001100000;
      6'd25: rom = 32'b01100101100101100101100101100101;
      6'd26: rom = 32'b01101001101001101001101001101010;
      6'd27: rom = 32'b01101101101101101101101101101111;
      6'd28: rom = 32'b01110001110001110001110001110000;
      6'd29: rom = 32'b01110101110101110101110101110101;
      6'd30: rom = 32'b01111001111001111001111001111010;
      6'd31: rom = 32'b01111101111101111101111101111111;
      6'd32: rom = 32'b10000010000010000010000010000000;
      6'd33: rom = 32'b10000110000110000110000110000101;
      6'd34: rom = 32'b10001010001010001010001010001010;
      6'd35: rom = 32'b10001110001110001110001110001111;
      6'd36: rom = 32'b10010010010010010010010010010000;
      6'd37: rom = 32'b10010110010110010110010110010101;
      6'd38: rom = 32'b10011010011010011010011010011010;
      6'd39: rom = 32'b10011110011110011110011110011111;
      6'd40: rom = 32'b10100010100010100010100010100000;
      6'd41: rom = 32'b10100110100110100110100110100101;
      6'd42: rom = 32'b10101010101010101010101010101010;
      6'd43: rom = 32'b10101110101110101110101110101111;
      6'd44: rom = 32'b10110010110010110010110010110000;
      6'd45: rom = 32'b10110110110110110110110110110101;
      6'd46: rom = 32'b10111010111010111010111010111010;
      6'd47: rom = 32'b10111110111110111110111110111111;
      6'd48: rom = 32'b11000011000011000011000011000000;
      6'd49: rom = 32'b1100011000111000111000111000101;
      6'd50: rom = 32'b11001011001011001011001011001010;
      6'd51: rom = 32'b11001110011110011110011110011111;
      6'd52: rom = 32'b11010011010011010011010011010000;
      6'd53: rom = 32'b1101011010111010111010111010101;
      6'd54: rom = 32'b11011011011011011011011011011010;
      6'd55: rom = 32'b11011110111101111101111101111111;
      6'd56: rom = 32'b11100011100011100011100011100000;
      6'd57: rom = 32'b11100111100111100111100111100101;
      6'd58: rom = 32'b11101011101011101011101011101010;
      6'd59: rom = 32'b11101111011111011111011111011111;
      6'd60: rom = 32'b11110011110011110011110011110000;
      6'd61: rom = 32'b11110111110111110111110111110101;
      6'd62: rom = 32'b11111011111011111011111011111010;
      6'd63: rom = 32'b11111111111111111111111111111111;
    end
  end
end

```

```

        default: rom = 32'b0;
    endcase
    end
end
endgenerate

always @(posedge clk) begin
    if(group_valid) begin
        codeword_out <= rom;
        codeword_valid <= 1'b1 ;
    end
    else begin
        codeword_valid <= 1'b0 ;
    end
end
end

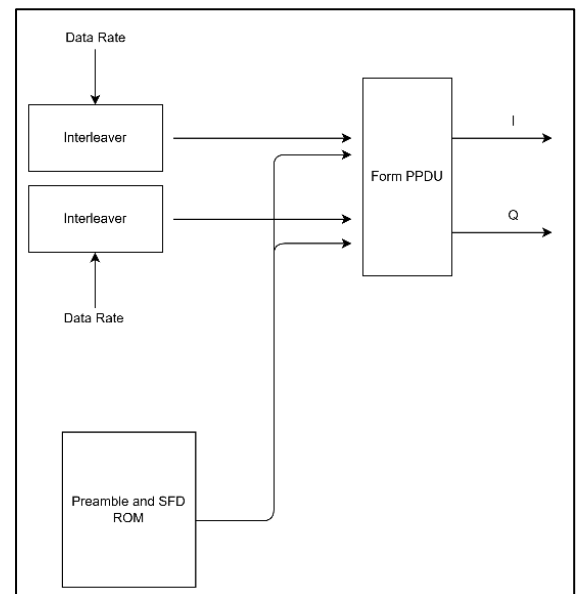
endmodule

```

2.2.10 inetrleaver and PPDU top wrapper:

2.2.10.1 Purpose:

The `interleaver_ppdu_top` module serves as a top-level structural wrapper that integrates the dual-channel bit interleaving pipeline with the packet formatting logic in the CSS PHY transmitter. It instantiates two parallel `interleaver_stage` modules to handle In-phase (I) and Quadrature (Q) codeword processing, seamlessly applying or bypassing rate-dependent bit interleaving based on the selected `DATA_RATE`. The processed I and Q codewords are then routed directly into the `ppdu_former` sub-block, which manages preamble insertion, handshakes via `req_next_symbol` to pull upcoming payload codewords, and serializes the complete physical frame into synchronous I and Q chip streams (`i_bit`, `q_bit`) accompanied by an active `chip_valid` strobe.



2.2.10.2 Interface:

Port	Direction	Description
clk	input	System clock
reset	input	Synchronous, active-high reset
start	input	Pulse: begin a new PPDU
i_codeword_in	input [M-1:0]	Upstream I-path codeword
i_codeword_valid	input	Valid for i_codeword_in
q_codeword_in	input [M-1:0]	Upstream Q-path codeword
q_codeword_valid	input	Valid for q_codeword_in

last_codeword	input	Marks the final codeword pair of the frame
req_next_symbol	output	Handshake pulse to upstream controller, requesting the next codeword pair
i_bit	output	Serial output chip (I)
q_bit	output	Serial output chip (Q)
chip_valid	output	Active during preamble and valid payload chips

2.2.10.3 RTL –interleaver_ppdu_top.v:

```
`timescale 1ns/1ps

module interleaver_ppdu_top #(
    parameter integer M                = 4,      // Codeword width: 4 (1Mbps) or 32 (250kbps)
    parameter      DATA_RATE          = 1'b0,   // 0 = 1Mbps (bypass), 1 = 250kbps (interleave)
    parameter integer PREAMBLE_TOTAL_BITS = 48,   // Preamble+SFD bit count: 48 (1Mbps) or 96 (250kbps)
    parameter [15:0] SFD                = 16'h749C
)(
    input wire      clk,
    input wire      reset,
    input wire      start,

    // Upstream Codeword Inputs (I and Q paths)
    input wire [M-1:0] i_codeword_in,
    input wire          i_codeword_valid,
    input wire [M-1:0] q_codeword_in,
    input wire          q_codeword_valid,
    input wire          last_codeword,

    // Pipeline Handshake & Serial Output
    output wire      req_next_symbol, // Handshake pulse to upstream controller
    output wire      i_bit,           // Serial output chip (I)
    output wire      q_bit,           // Serial output chip (Q)
    output wire      chip_valid       // Active during preamble and valid payload chips
);

// Signals between interleaver stages and PPDU former
wire [M-1:0] i_stage_out;
wire          i_stage_valid;
wire [M-1:0] q_stage_out;
wire          q_stage_valid;

// I-Channel Interleaver Stage
interleaver_stage #(
    .M_OUT      (M),
    .DATA_RATE  (DATA_RATE)
) u_interleaver_stage_i (
    .clk         (clk),
    .reset       (reset),
    .codeword_in (i_codeword_in),
    .codeword_valid (i_codeword_valid),
    .data_out    (i_stage_out),
    .data_valid  (i_stage_valid)
);

// Q-Channel Interleaver Stage
interleaver_stage #(
    .M_OUT      (M),
    .DATA_RATE  (DATA_RATE)
) u_interleaver_stage_q (
    .clk         (clk),
```

```

        .reset          (reset),
        .codeword_in    (q_codeword_in),
        .codeword_valid (q_codeword_valid),
        .data_out       (q_stage_out),
        .data_valid     (q_stage_valid)
    );

    // PPDU Former (Preamble ROM + Parallel-to-Serial Muxing)

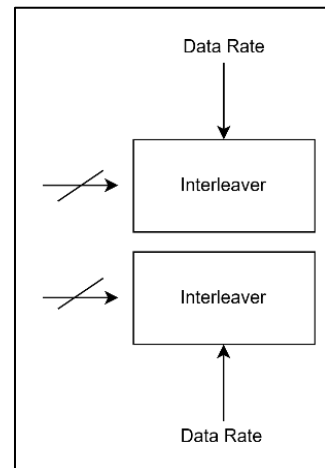
    ppdu_former #(
        .M              (M),
        .PREAMBLE_TOTAL_BITS(PREAMBLE_TOTAL_BITS)
    ) u_ppdu_former (
        .clk             (clk),
        .reset           (reset),
        .start           (start),
        .i_codeword      (i_stage_out),
        .i_codeword_valid (i_stage_valid),
        .q_codeword      (q_stage_out),
        .q_codeword_valid (q_stage_valid),
        .last_codeword   (last_codeword),
        .req_next_symbol (req_next_symbol),
        .i_bit           (i_bit),
        .q_bit           (q_bit),
        .chip_valid      (chip_valid)
    );
endmodule

```

2.2.11 Interleaver (I/Q):

2.2.11.1 Purpose:

Dual processing blocks that apply a fixed bit-group permutation to the encoded Walsh–Hadamard codewords to provide immunity against channel burst errors. This block actively reorders 4-bit sub-blocks when operating in 250 kbps mode and is bypassed when configured for 1 Mbps mode.



2.2.11.2 Interface:

Port	Direction	Description
data_in	input [63:0]	Two concatenated 32-bit codewords, viewed as 16 groups of 4 bits (group i = bits [63-4i -: 4])
data_out	output [63:0]	Bit-interleaved output; groups reordered per the fixed PERM table

2.2.11.3 RTL –interleaver.v:

```

`timescale 1ns/1ps

module interleaver (
    input wire [63:0] data_in,          // group i occupies bits [63-4*i -: 4], i=0..15 (G0 = MSB group)
    output wire [63:0] data_out
);

```

```

// PERM[i] = which input group feeds output position i
function [3:0] perm_of; input [3:0] i;
begin
    case (i)
        4'd0: perm_of = 4'd0;
        4'd1: perm_of = 4'd13;
        4'd2: perm_of = 4'd2;
        4'd3: perm_of = 4'd15;
        4'd4: perm_of = 4'd4;
        4'd5: perm_of = 4'd9;
        4'd6: perm_of = 4'd6;
        4'd7: perm_of = 4'd11;
        4'd8: perm_of = 4'd8;
        4'd9: perm_of = 4'd5;
        4'd10: perm_of = 4'd10;
        4'd11: perm_of = 4'd7;
        4'd12: perm_of = 4'd12;
        4'd13: perm_of = 4'd1;
        4'd14: perm_of = 4'd14;
        default: perm_of = 4'd3;
    endcase
end
endfunction

genvar gi;
generate
    for (gi = 0; gi < 16; gi = gi + 1) begin : g_perm
        // group gi of data_out (bits [63-4*gi -: 4]) = group perm_of(gi) of data_in
        wire [3:0] src_idx = perm_of(gi[3:0]);
        assign data_out[63-4*gi -: 4] = data_in[63-4*src_idx -: 4];
    end
endgenerate
endmodule

```

2.2.12 interleaver stage:

2.2.12.1 Purpose:

The interleaver_stage module reconciles rate-dependent bit interleaving requirements within the CSS PHY transmitter datapath by unifying the module's interface across different operating modes. When configured for 1 Mbps operation (DATA_RATE = 0), bit interleaving is bypassed entirely, registering incoming 4-bit codewords straight to the output with a single-cycle pipeline delay. For 250 kbps operation (DATA_RATE = 1), the module operates a two-state FSM that buffers two consecutive 32-bit codewords into a combined 64-bit block to feed the underlying interleaver permutation logic, streaming the resulting two permuted 32-bit halves back-to-back over two consecutive cycles. This abstraction allows the downstream PPDU former and serializer to interact with a uniform "one codeword in, one codeword out" interface regardless of the selected data rate.

2.2.12.2 Interface:

Port	Direction	Description
clk	input	System clock
reset	input	Synchronous, active-high reset
codeword_in	input [M_OUT-1:0]	Codeword in from symbol_mapper
codeword_valid	input	Valid for codeword_in
data_out	output [M_OUT-1:0]	Codeword out (bypassed at 1Mbps, interleaved at 250kbps)
data_valid	output	Valid for data_out, to Form PPDU / parallel_to_serial

2.2.12.3 RTL –interleaver_stage.v:

```

`timescale 1ns/1ps

module interleaver_stage #(
    parameter integer M_OUT      = 4,      // 4 (1Mbps) or 32 (250kbps)
    parameter DATA_RATE = 1'b0 // 0 = 1Mbps (interleaver elaborated out entirely), 1 = 250kbps
)(interleaver_active)
(
    input wire      clk,
    input wire      reset,

    input wire [M_OUT-1:0] codeword_in,
    input wire      codeword_valid,    // from symbol_mapper

    output reg [M_OUT-1:0] data_out,
    output reg      data_valid        // to Form PPDU / parallel_to_serial
);

generate
    if (DATA_RATE == 1'b0) begin : g_1mbps_bypass

        always @(posedge clk) begin
            if (reset) begin
                data_out  <= {M_OUT{1'b0}};
                data_valid <= 1'b0;
            end else begin
                data_out  <= codeword_in;
                data_valid <= codeword_valid;
            end
        end

    end

    else begin : g_250k_interleave
        // 2-State FSM Definitions
        localparam S_WAIT_FIRST = 1'b0; // Waiting for codeword N
        localparam S_WAIT_SECOND = 1'b1; // First codeword buffered, waiting for codeword N+1

        reg      state;
    end
endgenerate

```

```

reg [M_OUT-1:0]  cw0;
reg              pending_second;
reg [M_OUT-1:0]  second_half;

wire [2*M_OUT-1:0] pair_in = {cw0, codeword_in}; // {older codeword, newer codeword}
wire [2*M_OUT-1:0] pair_out;

interleaver u_interleaver (
    .data_in      (pair_in),
    .data_out     (pair_out)
);

always @(posedge clk) begin
    if (reset) begin
        state      <= S_WAIT_FIRST;
        cw0        <= {M_OUT{1'b0}};
        second_half <= {M_OUT{1'b0}};
        pending_second <= 1'b0;
        data_out    <= {M_OUT{1'b0}};
        data_valid  <= 1'b0;
    end else begin
        data_valid <= 1'b0;

        if (pending_second) begin
            // Stream out second half of the pair (computed previous cycle)
            data_out    <= second_half;
            data_valid  <= 1'b1;
            pending_second <= 1'b0;
        end else if (codeword_valid) begin
            case (state)
                S_WAIT_FIRST: begin
                    // Buffer codeword N and wait for N+1
                    cw0    <= codeword_in;
                    state <= S_WAIT_SECOND;
                end

                S_WAIT_SECOND: begin
                    // Codeword N+1 arrived: pair processed combinationaly this cycle
                    data_out    <= pair_out[2*M_OUT-1 -: M_OUT];
                    second_half <= pair_out[M_OUT-1:0];
                    data_valid  <= 1'b1;
                    pending_second <= 1'b1;
                    state      <= S_WAIT_FIRST;
                end
            endcase
        end
    end
end

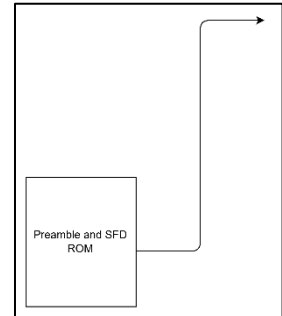
endgenerate
endmodule

```

2.2.13 Preamble and SFD ROM:

2.2.13.1 Purpose :

A dedicated Look-Up Table storing the predefined Synchronization Header (SHR) patterns, including the chip sequence preamble and the Start-of-Frame Delimiter (SFD). It streams these fixed sequences to the PPDU assembly unit at the start of frame generation.



2.2.13.2 Interface:

Port	Direction	Description
addr	input [7:0]	Bit address, 0 to TOTAL_BITS-1
bit_out	output	Preamble/SFD bit at the given address (constant ROM, all-1s preamble + SFD pattern)

2.2.13.3 RTL –preamble_sdf_ROM.v:

```
`timescale 1ns/1ps

module preamble_sdf_rom #(
    parameter TOTAL_BITS = 48,                // 48 for 1Mbps, 96 for 250kbps
    parameter [15:0] SFD = 16'h749C          // 16'h749C for 1Mbps | 16'h7A23 for 250kbps
) (
    input wire [7:0] addr,                    // 0 to TOTAL_BITS-1
    output wire      bit_out
);

    localparam PREAMBLE_LEN = TOTAL_BITS - 16;

    // Concatenate preamble (all 1s) and SFD into a single constant vector
    localparam [TOTAL_BITS-1:0] BITS = { {PREAMBLE_LEN{1'b1}}, SFD };

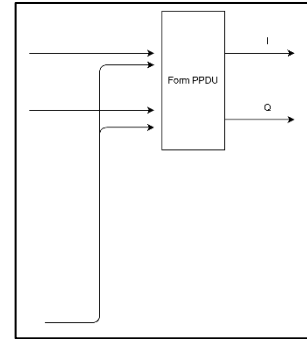
    // Index from MSB to LSB based on input address
    assign bit_out = BITS[TOTAL_BITS - 1 - addr] ; //!

endmodule
```

2.2.14 Form PPDU:

2.2.14.1 Purpose:

The frame construction unit merges the synchronization sequences from the Preamble and SFD ROM with the encoded, interleaved PHY Header (PHR) and PSDU payload streams. It outputs the fully structured Physical Protocol Data Unit (PPDU) bit sequence in proper frame order.



2.2.14.2 Interface:

Port	Direction	Description
clk	input	System clock
reset	input	Synchronous, active-high reset
start	input	Pulse: begin a new PPDU (start by reading the preamble)
i_codeword	input [M-1:0]	Interleaved/bypassed I-path codeword
i_codeword_valid	input	Valid for i_codeword
q_codeword	input [M-1:0]	Interleaved/bypassed Q-path codeword
q_codeword_valid	input	Valid for q_codeword
last_codeword	input	Asserted by the Controller alongside the final valid pulse (forwarded from zero_padding's frame_done)
req_next_symbol	output	Pulse: ask Controller to advance the pipeline for the next codeword pair
i_bit	output	Serial output chip, I path (preamble bit or payload chip)
q_bit	output	Serial output chip, Q path (preamble bit or payload chip)
chip_valid	output	1 while i_bit/q_bit carry a real chip this cycle

2.2.14.3 RTL –ppdu_former.v:

```
`timescale 1ns/1ps

module ppdu_former #(
    parameter integer M = 4, // 4 (1Mbps) / 32 (250kbps)
    parameter integer PREAMBLE_TOTAL_BITS = 48, // 48 (1Mbps) / 96 (250kbps)
    parameter PREAMBLE_MEMFILE = "../vectors/preamble_sfd_1mbps.mem"
)(
    input wire clk,
    input wire reset,
    input wire start, // pulse: begin a new PPDU (go read the preamble first)

    // interleaved/bypassed codeword streams from interleaver_stage
    input wire [M-1:0] i_codeword,
    input wire i_codeword_valid,
    input wire [M-1:0] q_codeword,
    input wire q_codeword_valid,
    input wire last_codeword, // Controller asserts this alongside the final valid pulse

    output reg req_next_symbol, // pulse: ask Controller to advance the pipeline for the next
codeword pair

    output wire i_bit,
    output wire q_bit,
    output wire chip_valid // 1 while i_bit/q_bit carry a real chip this cycle
);

reg [7:0] preamble_addr;
wire preamble_bit;

preamble_sfd_rom #(
    .TOTAL_BITS (PREAMBLE_TOTAL_BITS)
) u_preamble_sfd (
    .addr (preamble_addr),
    .bit_out (preamble_bit)
);

wire payload_load ;

wire i_payload_bit, q_payload_bit;
wire i_payload_valid, q_payload_valid;
wire i_done, q_done;

parallel_to_serial #(.M(M)) u_p2s_i (
    .clk (clk),
    .reset (reset),
    .load (payload_load),
    .data_in (i_codeword),
    .data_in_valid (i_codeword_valid),
    .bit_out (i_payload_bit),
    .bit_valid (i_payload_valid),
    .done (i_done)
);

parallel_to_serial #(.M(M)) u_p2s_q (
    .clk (clk),
    .reset (reset),
    .load (payload_load),
    .data_in (q_codeword),
    .data_in_valid (q_codeword_valid),
    .bit_out (q_payload_bit),
    .bit_valid (q_payload_valid),
    .done (q_done)
);

localparam [1:0] S_IDLE = 2'd0,
S_PREAMBLE = 2'd1,
S_PAYLOAD_WAIT = 2'd2,
```

```

        S_PAYLOAD_SHIFT= 2'd3;

reg [1:0] state;
reg      pending_last;    // latches last_codeword for the pair currently shifting

assign payload_load = (state == S_PAYLOAD_WAIT) && i_codeword_valid && q_codeword_valid;

always @(posedge clk) begin
    if (reset) begin
        state          <= S_IDLE;
        preamble_addr   <= 8'd0;
        req_next_symbol <= 1'b0;
        pending_last    <= 1'b0;
    end else begin
        req_next_symbol <= 1'b0;

        case (state)

            S_IDLE: begin
                if (start) begin
                    preamble_addr <= 8'd0;
                    state         <= S_PREAMBLE;
                end
            end

            // Rule: drain the preamble/SFD ROM first, one chip/cycle
            // on both I and Q, before touching any payload data.
            S_PREAMBLE: begin
                if (preamble_addr == PREAMBLE_TOTAL_BITS - 1) begin
                    req_next_symbol <= 1'b1;    // ask for the first codeword pair
                    state          <= S_PAYLOAD_WAIT;
                end else begin
                    preamble_addr <= preamble_addr + 8'd1;
                end
            end

            // Then start with the interleaved (or bypassed) data.
            S_PAYLOAD_WAIT: begin
                if (i_codeword_valid && q_codeword_valid) begin
                    pending_last <= last_codeword;
                    state        <= S_PAYLOAD_SHIFT;
                end
            end

            S_PAYLOAD_SHIFT: begin
                if (i_done && q_done) begin
                    if (pending_last) begin
                        state <= S_IDLE;
                    end else begin
                        req_next_symbol <= 1'b1;
                        state          <= S_PAYLOAD_WAIT;
                    end
                end
            end

            default: state <= S_IDLE;
        endcase
    end
end

// Output chip mux: preamble bit while in S_PREAMBLE, payload chip
// (post interleave/bypass) once in the payload phase.

wire in_preamble = (state == S_PREAMBLE);

assign i_bit      = in_preamble ? preamble_bit : i_payload_bit;
assign q_bit      = in_preamble ? preamble_bit : q_payload_bit;
assign chip_valid = in_preamble || (i_payload_valid && q_payload_valid);
endmodule

```

2.2.15 Parallel to serial convertor:

2.2.15.1 Purpose:

Within the PPDU former, the `parallel\_to\_serial` converter acts as a critical interface that transforms parallel M -bit codewords into a continuous, single-bit serial stream of chips. When fed with M -bit codewords ($M=4$ for 1 Mbps or $M=32$ for 250 kbps) from the upstream interleaver stage, the module captures the data and shifts it out MSB-first at a rate of one chip per clock cycle. This serialized output directly feeds the downstream QPSK mapper to enable precise chip-by-chip phase encoding across both In-phase (I) and Quadrature (Q) paths. Additionally, the module manages execution handshaking by asserting a `bit\_valid` flag while shifting is active and firing a single-cycle `done` strobe on the final chip to signal completion to the controlling logic.

2.2.15.2 Interface:

Port	Direction	Description
clk	input	System clock
reset	input	Synchronous, active-high reset
load	input	Pulse: capture data_in this cycle (must coincide with data_in_valid)
data_in	input [M-1:0]	M-bit codeword to shift out
data_in_valid	input	Valid for data_in at the load pulse
bit_out	output	One chip per cycle while busy
bit_valid	output	High while a captured codeword is being shifted out
done	output	1-cycle pulse on the last chip of this codeword

2.2.15.3 RTL –parallel_to_serial.v:

```
`timescale 1ns/1ps

module parallel_to_serial #(
    parameter integer M = 4 ,           // codeword width: 4 (1Mbps) or 32 (250kbps)
    parameter cnt_width = $clog2(M+1)
)(
    input wire      clk,
    input wire      reset,

    input wire      load,               // pulse: capture data_in this cycle (must coincide with
data_in_valid)
    input wire [M-1:0] data_in,
    input wire      data_in_valid,

    output reg      bit_out,            // one chip per cycle while busy
    output reg      bit_valid,          // high while a captured codeword is being shifted out
    output reg      done                // 1-cycle pulse on the last chip of this codeword
);

    reg [M-1:0]      shift_reg;
    reg [cnt_width-1:0] cnt;           // counts remaining chips after the first
    reg              busy;

    always @(posedge clk) begin
        if (reset) begin
```

```

shift_reg <= {M{1'b0}};
cnt      <= {cnt_width{1'b0}};
busy     <= 1'b0;
bit_out  <= 1'b0;
bit_valid <= 1'b0;
done     <= 1'b0;
end else begin
done <= 1'b0;

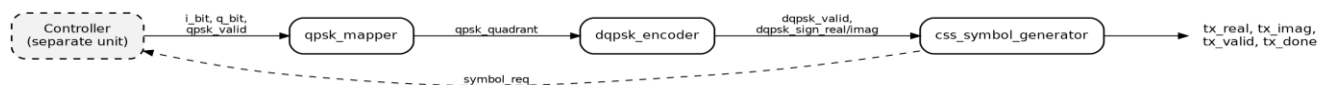
if (load && data_in_valid) begin
// First chip is available immediately (MSB), remaining M-1
// chips shift out over the next M-1 cycles.

//*****
//Raghad Fix 4
//shift_reg <= data_in;
//cnt      <= {cnt_width{1'b0}} ;
shift_reg <= data_in >> 1;
cnt      <= { {(cnt_width-1){1'b0}}, 1'b1 }; // count chips EMITTED
//*****
busy     <= 1'b1;
bit_out  <= data_in[0];
bit_valid <= 1'b1;
end else if (busy) begin
//if (cnt == (M[$clog2(M+1)-1:0] - 1'b1)) begin //raghad fix
if (cnt == M[cnt_width-1:0]) begin
busy     <= 1'b0;
bit_valid <= 1'b0;
done     <= 1'b1;
end else begin
shift_reg <= shift_reg >> 1;
cnt      <= cnt + 1'b1;
bit_out  <= shift_reg[0];
bit_valid <= 1'b1;
end
end else begin
bit_valid <= 1'b0;
end
end
end
endmodule

```

2.2.15.4 Module map — symbol mapping and differential encoding

Module	File	Role
QPSK Mapper	qpsk_mapper.v	Combinational lookup: one (I,Q) bit pair to one QPSK rotation quadrant.
DQPSK Differential Encoder	dqpsk_encoder.v	4-symbol delay line and rotate: implements $S(n) = X(n) \cdot S(n-4)$ with no multiplier hardware.
Datapath top wrapper	tx_datapath_top.v	Integrates qpsk_mapper and dqpsk_encoder with css_symbol_generator, and presents the symbol-request interface to the Controller.



2.2.16 Symbol Mapping and Differential Encoding

This section covers the QPSK Mapper and the DQPSK Differential Encoder, together with the wrapper that integrates them with `css_symbol_generator` (Sections 3-7) to form the complete modulation datapath, ending at the transmitted (`tx_real`, `tx_imag`) output. `css_symbol_generator` is reused unchanged.

2.2.17 Representation of QPSK and DQPSK symbols

One question determines the structure of every module in this section and is therefore settled first: how a QPSK or DQPSK symbol is represented as a hardware signal.

The QPSK Mapper output is a rotation value, one of $\{+1, +j, -1, -j\}$, rather than a general complex point. This is why the architecture figure represents this step as a multiplexer rather than an arithmetic block: it is a four-entry lookup indexed by the (I, Q) bits. This is confirmed against the reference formula $\text{QPSK_symbol} = ((I+Q) - j(I-Q))/2$, which always evaluates to one of those four axis points.

The DQPSK differential encoder, $S(n) = X(n) \cdot S(n-4)$, rotates the previous symbol by that amount. The standard's initial reference value is $S(0)=S(1)=S(2)=S(3)=\exp(j\pi/4)$ -- a 45-degree point where the real and imaginary parts are equal. Since every subsequent rotation is a multiple of 90 degrees, every $S(n)$ stays on that same diagonal forever: real and imaginary parts are always exactly +1 or -1 (the $1/\sqrt{2}$ scale factor is dropped deliberately, matching how `complex_multiplier.v` already treats its DQPSK input - only the phase matters for the chirp modulation stage).

Consequently the rotation requires no multiplier hardware: rotating a (`sign_real`, `sign_imag`) pair by 0, 90, 180 or 270 degrees reduces to a swap and a conditional negation.

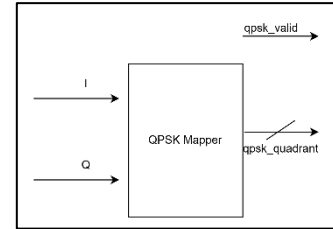
Quadrant (from mapper)	New (<code>sign_real</code> , <code>sign_imag</code>)
0 (x 1)	(<code>sign_real</code> , <code>sign_imag</code>) -- unchanged
1 (x +j)	(- <code>sign_imag</code> , <code>sign_real</code>)
2 (x -1)	(- <code>sign_real</code> , - <code>sign_imag</code>)
3 (x -j)	(<code>sign_imag</code> , - <code>sign_real</code>)

This maintains the multiplier-free approach established in `complex_multiplier.v` (Section 5) consistently across the whole modulation chain, and the encoder output connects directly to `css_symbol_generator` without modification to that module.

2.2.18 QPSK Mapper

2.2.18.1 Purpose:

converts one (I,Q) bit pair into the QPSK rotation quadrant the DQPSK encoder needs. Purely combinational.



I	Q	QPSK symbol	Quadrant code
0	0	+1	00
1	0	+j	01
1	1	-1	10
0	1	-j	11

2.2.18.2 Interface

Signal	Direction	Width	Description
i_bit	IN	1	0 = +1, 1 = -1 (project-wide sign convention)
q_bit	IN	1	0 = +1, 1 = -1
qpsk_quadrant	OUT	2	Rotation count x90 degrees: 00=+1, 01=+j, 10=-1, 11=-j

2.2.18.3 RTL -- qpsk_mapper.v

```

`timescale 1ns/1ps

module qpsk_mapper (
    input wire    i_bit,           // 0 = +1, 1 = -1 (project-wide sign convention)
    input wire    q_bit,           // 0 = +1, 1 = -1
    output reg [1:0] qpsk_quadrant // 00=+1, 01=+j, 10=-1, 11=-j (rotation count x90 deg)
);

    always @(*) begin
        case ({i_bit, q_bit})
            2'b00: qpsk_quadrant = 2'd2; // -1
            2'b10: qpsk_quadrant = 2'd3; // -j
            2'b11: qpsk_quadrant = 2'd0; // +1
            2'b01: qpsk_quadrant = 2'd1; // +j
            default: qpsk_quadrant = 2'd0;
        endcase
    end
endmodule

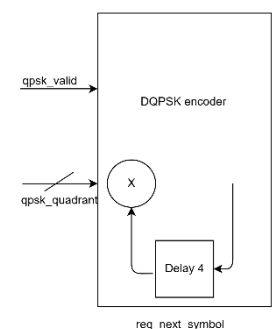
```

2.2.19 DQPSK Encoder

2.2.19.1 Purpose:

the "4 FF DELAY" + first "X" block of the architecture figure. Implements $S(n) = X(n) \cdot S(n-4)$ as a 4-deep delay line of previously-produced sign pairs plus the rotate logic from Section 8.1 -- no real multiplier.

- reset re-initializes all 4 delay slots to the standard's stated $S(0..3)$ initial condition: sign_real=0, sign_imag=0 (both "+1"), representing $\exp(j \cdot \pi/4)$.



- qpsk_valid is expected to be a single-cycle pulse with qpsk_quadrant valid the same cycle; dqpsk_valid pulses exactly one cycle later with the new sign pair.
- No back-pressure: a new qpsk_valid pulse is assumed not to arrive before the previous one's dqpsk_valid has been consumed downstream -- this module produces exactly one output per input, one cycle later, unconditionally.

2.2.19.2 Interface

Signal	Direction	Width	Description
clk, reset	IN	1 each	System clock; synchronous, active-high reset
qpsk_valid	IN	1	qpsk_quadrant is valid this cycle
qpsk_quadrant	IN	2	Rotation to apply, from qpsk_mapper.v
dqpsk_valid	OUT	1	New S(n) is valid this cycle (1-cycle pulse)
dqpsk_sign_real	OUT	1	S(n) real sign: 0 = +1, 1 = -1
dqpsk_sign_imag	OUT	1	S(n) imag sign: 0 = +1, 1 = -1

2.2.19.3 RTL -- dqpsk_encoder.v

```
`timescale 1ns/1ps

module dqpsk_encoder (
    input wire      clk,
    input wire      reset,          // synchronous, active-high -- reloads the
                                    // S(0..3) initial condition into the delay line

    input wire      qpsk_valid,     // qpsk_quadrant is valid this cycle
    input wire [1:0] qpsk_quadrant, // rotation to apply, from qpsk_mapper.v

    output reg      dqpsk_valid,     // new S(n) is valid this cycle (1-cycle pulse)
    output reg      dqpsk_sign_real, // S(n) real sign: 0 = +1, 1 = -1
    output reg      dqpsk_sign_imag // S(n) imag sign: 0 = +1, 1 = -1
);

// 4-deep delay line of previously-produced (sign_real, sign_imag)
// pairs -- slot 0 is the OLDEST (this is S(n-4)); slot 3 is the most
// recently produced symbol.

reg sr_q [0:3];
reg si_q [0:3];
integer k;

// Rotate S(n-4) = (sr_q[0], si_q[0]) by qpsk_quadrant -- see header
// comment for the derivation:
// quadrant 0 (x 1): (sr, si)
// quadrant 1 (x +j): (-si, sr)
// quadrant 2 (x -1): (-sr, -si)
// quadrant 3 (x -j): (si, -sr)
// Sign bits use 0=+1,1=-1, so "negate" is just a bit flip (XOR 1).

reg new_sr, new_si;
always @(*) begin
    case (qpsk_quadrant)
        2'd0: begin new_sr = sr_q[0];      new_si = si_q[0];      end
        2'd1: begin new_sr = ~si_q[0];     new_si = sr_q[0];      end
        2'd2: begin new_sr = ~sr_q[0];     new_si = ~si_q[0];     end
        2'd3: begin new_sr = si_q[0];      new_si = ~sr_q[0];     end
        default: begin new_sr = sr_q[0];   new_si = si_q[0];     end
    endcase
end

always @(posedge clk) begin
    if (reset) begin
        for (k = 0; k < 4; k = k + 1) begin
            sr_q[k] <= 1'b0; // both +1 -> the exp(j*pi/4) reference point
            si_q[k] <= 1'b0;
        end
    end
end
```

```

        end
        dqpsk_valid      <= 1'b0;
        dqpsk_sign_real <= 1'b0;
        dqpsk_sign_imag <= 1'b0;
    end else begin
        dqpsk_valid <= 1'b0; // default; overridden below
        if (qpsk_valid) begin
            // shift the delay line: drop slot0 (just consumed),
            // push the new symbol in at slot3
            sr_q[0] <= sr_q[1]; si_q[0] <= si_q[1];
            sr_q[1] <= sr_q[2]; si_q[1] <= si_q[2];
            sr_q[2] <= sr_q[3]; si_q[2] <= si_q[3];
            sr_q[3] <= new_sr; si_q[3] <= new_si;

            dqpsk_valid      <= 1'b1;
            dqpsk_sign_real <= new_sr;
            dqpsk_sign_imag <= new_si;
        end
    end
end
endmodule

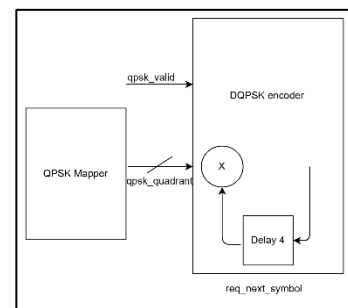
```

2.2.20 transmitter to data path top wrapper:

2.2.20.1 purpose:

This wrapper integrates qpsk_mapper and dqpsk_encoder with css_symbol_generator to form the complete modulation datapath, and presents the symbol-request interface to the Controller.

The Controller is deliberately not instantiated inside this module. Its output, one (I, Q) bit pair per requested symbol, is taken directly as an input, which keeps the modulation datapath independent of the Controller implementation and allows each to be verified separately.



2.2.20.2 Interface protocol

The datapath is request-driven rather than free-running. css_symbol_generator asserts next_symbol_req, which this module exposes as symbol_req; the Controller responds by presenting one (I, Q) bit pair on i_bit and q_bit with qpsk_valid asserted for a single cycle. The datapath consumes exactly one symbol per request, so no flow control beyond this handshake is required.

num_symbols is the total symbol count for the complete packet, comprising the preamble, the SFD and the payload, and is supplied by the Controller. It must be an exact multiple of four, since four DQPSK symbols are mapped onto the four subcarriers of one chirp sequence (Section 4). The preamble and SFD contribute 48 symbols at 1 Mbps and 96 symbols at 250 kbps, both multiples of four; the payload symbol count must therefore also be a multiple of four for this constraint to hold, which is a requirement on the upstream padding arithmetic.

2.2.20.3 Interface

Signal	Direction	Width	Description
clk, reset	IN	1 each	System clock; synchronous, active-high reset
start	IN	1	Begin a new PPDU (single-cycle pulse)
chirp_index	IN	2	CSK chirp sequence select, m = 1-4 encoded as 0-3
num_symbols	IN	NUM_SYMBOLS_WIDTH	Total symbol count for the whole packet; must be a multiple of 4
i_bit	IN	1	I bit from the Controller; 0 = +1, 1 = -1
q_bit	IN	1	Q bit from the Controller; 0 = +1, 1 = -1
qpsk_valid	IN	1	i_bit and q_bit are valid this cycle (single-cycle pulse)
symbol_req	OUT	1	Requests the next (I, Q) bit pair from the Controller
tx_done	OUT	1	Whole PPDU fully transmitted (single-cycle pulse)
tx_real, tx_imag	OUT	OUT_WIDTH (signed)	Final transmitted I/Q sample
tx_valid	OUT	1	High exactly when tx_real and tx_imag hold a valid sample

2.2.20.4 RTL -- tx_datapath_top.v

```

`timescale 1ns/1ps

module tx_datapath_top #(
    parameter NUM_SYMBOLS_WIDTH = 12, // must match css_symbol_generator's parameter
    parameter ROM_WIDTH         = 5,
    parameter OUT_WIDTH         = 6
) (
    input wire          clk,
    input wire          reset,

    input wire          start,           // begin a new PPDU
    input wire [1:0]    chirp_index,     // CSK chirp sequence select (m=1-4)
    input wire [NUM_SYMBOLS_WIDTH-1:0] num_symbols, // total symbols for the whole packet
                                           // (preamble+SFD+payload), multiple of 4

    output wire         tx_done,         // whole PPDU fully transmitted

    input wire          i_bit,           // 0 = +1, 1 = -1
    input wire          q_bit,           // 0 = +1, 1 = -1
    input wire          qpsk_valid,      // i_bit/q_bit valid this cycle
    output wire         symbol_req,      // requests the next (i_bit, q_bit) pair

    output wire signed [OUT_WIDTH-1:0] tx_real,
    output wire signed [OUT_WIDTH-1:0] tx_imag,
    output wire         tx_valid
);

    wire [1:0] qpsk_quadrant;

    qpsk_mapper u_qpsk_mapper (
        .i_bit      (i_bit),
        .q_bit      (q_bit),
        .qpsk_quadrant (qpsk_quadrant)
    );

    wire dqpsk_valid_w, dqpsk_sign_real_w, dqpsk_sign_imag_w;

    dqpsk_encoder u_dqpsk_encoder (
        .clk      (clk),

```

```

        .reset          (reset),
        .qpsk_valid     (qpsk_valid),
        .qpsk_quadrant  (qpsk_quadrant),
        .dqpsk_valid    (dqpsk_valid_w),
        .dqpsk_sign_real (dqpsk_sign_real_w),
        .dqpsk_sign_imag (dqpsk_sign_imag_w)
    );

    css_symbol_generator #(
        .NUM_SYMBOLS_WIDTH (NUM_SYMBOLS_WIDTH),
        .ROM_WIDTH          (ROM_WIDTH),
        .OUT_WIDTH          (OUT_WIDTH)
    ) u_css_symbol_generator (
        .clk                (clk),
        .reset              (reset),
        .start              (start),
        .chirp_index        (chirp_index),
        .num_symbols        (num_symbols),
        .dqpsk_valid        (dqpsk_valid_w),
        .dqpsk_sign_real    (dqpsk_sign_real_w),
        .dqpsk_sign_imag    (dqpsk_sign_imag_w),
        .next_symbol_req    (symbol_req),
        .done               (tx_done),
        .tx_real            (tx_real),
        .tx_imag            (tx_imag),
        .tx_valid           (tx_valid)
    );
endmodule

```

2.2.21 Fixed-Point Analysis and word length selection:

2.2.21.1 Objective

The purpose of this analysis is to determine a suitable fixed-point word length for the chirp samples stored in the FPGA ROM. The original MATLAB chirp sequence is floating-point and is used as the reference. Candidate word lengths are swept, and the quantization error is measured using normalized Mean Square Error (MSE).

2.2.21.2 Fixed-point quantization method

For a candidate word length R , the project uses the scale factor:

$$\text{Scale} = 2^{(R-1)} - 1$$

The floating-point chirp samples are multiplied by this scale factor. The hardware quantization rule uses floor:

$$\text{fixedValue} = \text{floor}(\text{floatingValue} \times \text{Scale})$$

Example: for a floating-point sample of 0.87 and $R = 6$, the scale factor is 31, so $0.87 \times 31 = 26.97$ and the stored ROM value is $\text{floor}(26.97) = 26$. The FPGA operates directly on this integer; it never reconstructs 0.87 during normal operation.

2.2.21.3 Why both round and floor are evaluated

The MSE sweep evaluates two quantization methods — round and floor — which are not two different fixed-point formats, but two different ways of converting the scaled floating-point value into an integer.

Method	Operation	Meaning
Round	<code>round(exactValue)</code>	Selects the nearest integer.
Floor	<code>floor(exactValue)</code>	Selects the next lower integer (toward -infinity) — this is the actual hardware-facing rule.

The distinction matters for negative values: floor always moves toward negative infinity, while round selects the nearest integer, so the two methods can produce different ROM values and different MSE results.

2.2.21.4 MSE calculation

For each candidate R, the floating-point samples are first scaled:

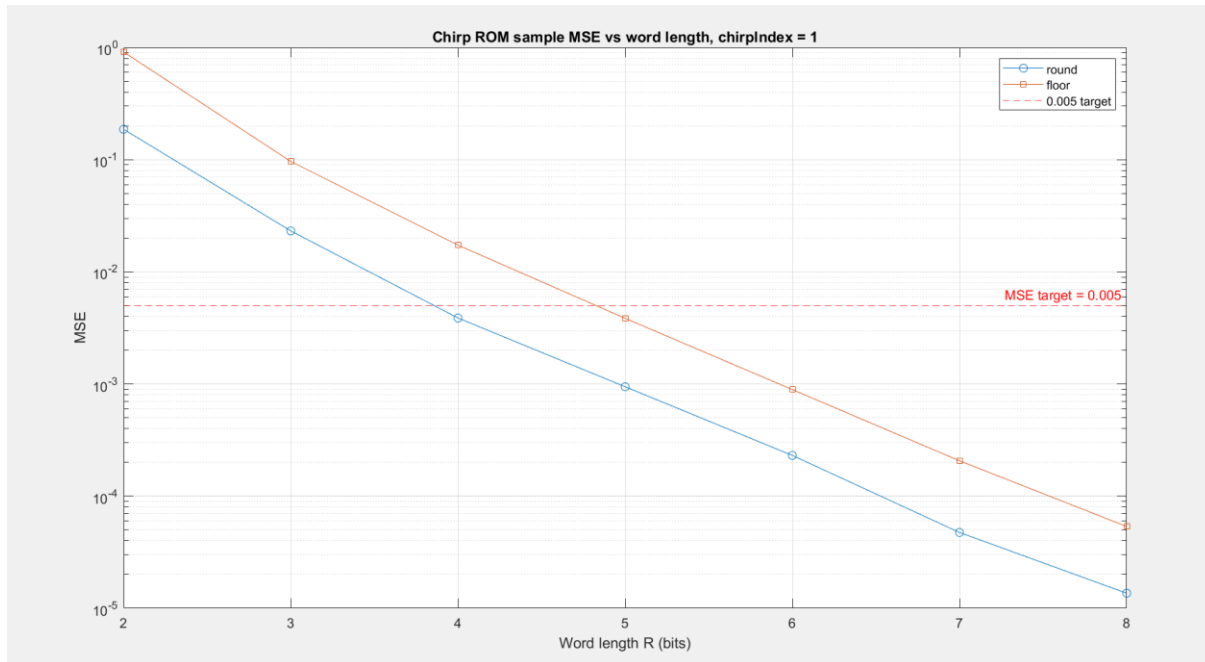
`exactValue = chirpSequence x (2(R-1) - 1)`

MSE is then computed between `exactValue` and its integer-quantized version, normalized by the energy of `exactValue`:

`MSE = sum(|exactValue - fixedValue|^2) / sum(|exactValue|^2)`

Target: $MSE < 0.005$ (IEEE 802.15.4a, Section 6.5a.5.1). The sweep is run for $R = 2$ through 8 bits, across all 4 chirp indices.

Results — `chirpIndex = 1`



R (bits)	MSE (round)	MSE (floor)
2	0.18658	0.91193
3	0.023155	0.096403
4	0.0038757	0.01729
5	0.00094463	0.0038504
6	0.00023074	0.00089133
7	4.7419e-05	0.00020668
8	1.3629e-05	5.3594e-05

2.2.21.5 Results — worst case across all 4 chirp indices

The sweep was repeated for chirpIndex = 1 through 4, and the worst-case MSE across all four was taken for each candidate R:

R (bits)	Worst-case MSE (floor)	Worst-case MSE (round)
2	0.91193	0.18658
3	0.096403	0.023155
4	0.01729	0.0038757
5	0.0038504	0.00094463
6	0.00089133	0.00023074
7	0.00020668	4.7419e-05
8	5.3594e-05	1.3629e-05

2.2.21.6 Final decision

Item	Value
MSE requirement	MSE < 0.005
Smallest R satisfying the requirement (worst case, floor)	5 bits (MSE = 0.0038504)

Chosen ROM word length	R = 5 bits
------------------------	------------

2.2.21.7 Justification:

The final word length is selected using the hardware-accurate quantization rule (floor), evaluated as the worst case across all 4 chirp indices, with the round results included as an additional cross-check. R = 5 clears the 0.005 target with roughly 25% margin on the worst-case floor curve.

The DQPSK symbol components are always exactly +1 or -1 (never fractional — see Section 5.1), so no separate quantization analysis is needed for them; the chirp ROM samples are the only signal that needed this analysis.

2.2.21.8 Output word-length derivation (complex multiplier)

With R = 5, ROM samples are 5-bit signed (range -16..15). The complex multiplier's output is a sum or difference of two such ROM values (see Section 5), so the worst-case magnitude is $15 + 15 = 30$. Representing ± 30 in two's complement needs 6 bits (range -32..31, comfortably covering ± 30) — 5 bits would only reach ± 16 , which is not enough. The complex multiplier and top-level wrapper therefore use a 6-bit signed output, not 7 — the 7-bit figure sometimes seen in early drafts of this design assumed R = 6, which was superseded once R = 5 was confirmed here.

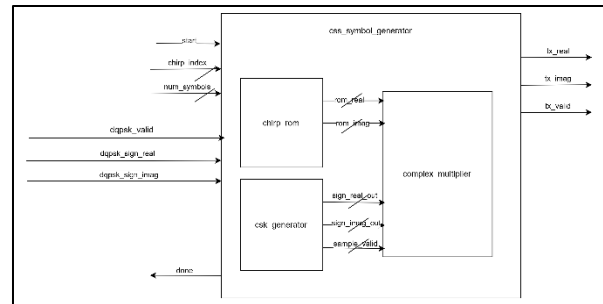
2.2.22 Module map — CSK modulation chain

Module	File	Role
CSS Symbol / Chirp Sequence ROM	chirp_rom.v	Stores the 4 pre-computed, fixed-point chirp sequences; returns one complex sample per address.
CSK Generator	csk_generator.v	Sequencing / control FSM: walks the ROM address bus in step with the DQPSK symbol stream, inserts inter-sequence gaps.
Complex Multiplier	complex_multiplier.v	Combines a ROM sample with a DQPSK sign pair to produce the final modulated I/Q sample.
Top-level wrapper	css_symbol_generator.v	Integrates the three modules above, compensating for each one's timing.

2.2.23 CSS Symbol Generator (css_symbol_generator)

2.2.23.1 Purpose:

Ties `chirp_rom` + `csk_generator` + `complex_multiplier` together into the single block that Task 2's top-level controller / DQPSK encoder instantiates. Corresponds to the "CSS Symbols ROM" + Controller + multiplier chain in Figure 3-3 of the thesis document. This module's only job is integration: wiring the three already-built submodules together correctly, including compensating for the timing each one introduces on its own.



2.2.23.2 Integration issues solved here

`chirp_rom`'s one-cycle read latency vs. `csk_generator`'s combinational valid

`chirp_rom` is a synchronous, registered ROM: the sample it returns on (`rom_real`, `rom_imag`) corresponds to whatever address was on (`chirp_index`, `subchirp_sel`, `sample_addr`) ONE CYCLE EARLIER. `csk_generator`'s `sample_valid` is combinational — asserted the same cycle an address is issued, with zero latency. Feeding `sample_valid` directly to the multiplier would flag the WRONG cycle as valid.

Fix: a single register, `sample_valid_d1`, delays `sample_valid` by exactly one cycle so it lines up with the ROM's registered output.

2.2.23.3 Why `sign_real_out`/`sign_imag_out` do NOT need the same delay

`csk_generator`'s `sign_real_out`/`sign_imag_out` are registered (set inside the `S_STREAM` case body), so they update to match whatever address was just issued starting the NEXT cycle — i.e. they already carry exactly one cycle of latency, by construction. They therefore already land in step with `chirp_rom`'s registered output for the same address, with no extra delay needed. Delaying them again would misalign them by a cycle in the other direction — this is called out explicitly in the module comments to prevent this being reintroduced during later modification.

2.2.23.4 `chirp_index` mid-packet drift

`csk_generator` latches `chirp_index` into its own internal `chirp_index_reg` at the start pulse and is not affected by the input changing mid-packet — but `chirp_rom` has no such latch and uses whatever is on its `chirp_index` input every cycle. If the top-level `chirp_index` input were fed to `chirp_rom` directly and something upstream changed it mid-packet (e.g. preparing the next packet's setting while this one is still streaming), `chirp_rom` would silently address the WRONG chirp sequence for the remainder of the packet.

Fix: `chirp_index` is latched a second time, independently, inside the wrapper itself, on the same start pulse — and that latched copy, not the live input, is what feeds `chirp_rom`. This makes the module correct regardless of how the caller drives its `chirp_index` input afterward. Section 7.2 describes the regression test written specifically to prove this fix works.

2.2.23.5 Pipeline timing summary

Cycle	What happens
N	Address (chirp_index_latched, subchirp_sel, sample_addr) issued; sample_valid asserted combinationally
N+1	rom_real/rom_imag respond to the cycle-N address; sign_real_out/sign_imag_out ALSO now reflect cycle N's sign (already registered inside csk_generator); sample_valid_d1 now equals cycle N's sample_valid
N+2	tx_real/tx_imag/tx_valid reflect complex_multiplier's registered response to cycle N+1's inputs

Total pipeline latency, address-issued to final Tx sample: 2 cycles. done is passed straight through from csk_generator with no extra delay: the corrected FSM always runs a gap (at least 10 cycles) after the final chirp sequence before asserting done, which is far longer than this module's 2-cycle pipeline drain time, so by the time done pulses every real sample has already appeared on (tx_real, tx_imag, tx_valid).

2.2.23.6 Interface

Signal	Direction	Width	Description
clk, reset	IN	1 each	System clock; synchronous, active-high reset
start	IN	1	Begin generating for this packet
chirp_index	IN	2	Selected chirp sequence, latched at start (see Section 6.2.3)
num_symbols	IN	NUM_SYMBOLS_WIDTH	Total DQPSK symbols in this packet, latched at start
dqpsk_valid, dqpsk_sign_real/imag	IN	1 each	DQPSK symbol handshake, passed through to csk_generator
next_symbol_req	OUT	1	Pulses to request the next DQPSK symbol
done	OUT	1	Packet modulation complete (one-cycle pulse)
tx_real, tx_imag	OUT	OUT_WIDTH (signed)	Final modulated I/Q output
tx_valid	OUT	1	High exactly when tx_real/tx_imag hold a real sample

2.2.23.7 RTL — css_symbol_generator.v

```

module css_symbol_generator #(
    parameter NUM_SYMBOLS_WIDTH = 12,
    parameter ROM_WIDTH         = 5,
    parameter OUT_WIDTH         = 6
) (
    input wire          clk,
    input wire          reset,           // synchronous, active-high

    input wire          start,
    input wire [1:0]    chirp_index,
    input wire [NUM_SYMBOLS_WIDTH-1:0] num_symbols, // total DQPSK symbols in this packet, latched at start

    input wire          dqpsk_valid,     // new DQPSK symbol available

```

```

input wire          dqpsk_sign_real, // 0 = +1, 1 = -1
input wire          dqpsk_sign_imag, // 0 = +1, 1 = -1

output wire          next_symbol_req, // pulses to request the next DQPSK symbol
output wire          done,

output wire signed [OUT_WIDTH-1:0] tx_real,          // final modulated I output
output wire signed [OUT_WIDTH-1:0] tx_imag,          // final modulated Q output
output wire          tx_valid                        // high exactly when tx_real/tx_imag hold a real
sample
);

reg [1:0] chirp_index_latched;
always @(posedge clk) begin
    if (reset)
        chirp_index_latched <= 2'd0;
    else if (start)
        chirp_index_latched <= chirp_index;
end

--
wire [1:0] subchirp_sel;
wire [5:0] sample_addr;
wire      sign_real_out;
wire      sign_imag_out;
wire      sample_valid;
wire      gap_active;

csk_generator #(
    .NUM_SYMBOLS_WIDTH (NUM_SYMBOLS_WIDTH)
) u_csk_generator (
    .clk          (clk),
    .reset        (reset),
    .start        (start),
    .chirp_index  (chirp_index),
    .num_symbols  (num_symbols),
    .dqpsk_valid  (dqpsk_valid),
    .dqpsk_sign_real (dqpsk_sign_real),
    .dqpsk_sign_imag (dqpsk_sign_imag),
    .subchirp_sel (subchirp_sel),
    .sample_addr  (sample_addr),
    .sign_real_out (sign_real_out),
    .sign_imag_out (sign_imag_out),
    .sample_valid  (sample_valid),
    .next_symbol_req (next_symbol_req),
    .done          (done),
    .gap_active    (gap_active)
);

wire signed [ROM_WIDTH-1:0] rom_real;
wire signed [ROM_WIDTH-1:0] rom_imag;

chirp_rom u_chirp_rom (
    .clk          (clk),
    .chirp_index  (chirp_index_latched),
    .subchirp_sel (subchirp_sel),
    .sample_addr  (sample_addr),
    .rom_real     (rom_real),
    .rom_imag     (rom_imag)
);

reg sample_valid_d1;
reg gap_active_d1;
always @(posedge clk) begin
    if (reset) begin
        sample_valid_d1 <= 1'b0;
        gap_active_d1   <= 1'b0;
    end
    else begin

```

```

        sample_valid_d1 <= sample_valid;
        gap_active_d1   <= gap_active;
    end
end

wire signed [ROM_WIDTH-1:0] gap_masked_real = gap_active_d1 ? {ROM_WIDTH{1'b0}} : rom_real;
wire signed [ROM_WIDTH-1:0] gap_masked_imag = gap_active_d1 ? {ROM_WIDTH{1'b0}} : rom_imag;
complex_multiplier #(
    .ROM_WIDTH (ROM_WIDTH),
    .OUT_WIDTH (OUT_WIDTH)
) u_complex_multiplier (
    .clk      (clk),
    .reset    (reset),
    //.rom_real (rom_real),
    //.rom_imag (rom_imag),
    .rom_real (gap_masked_real),
    .rom_imag (gap_masked_imag),
    .sign_real (sign_real_out),
    .sign_imag (sign_imag_out),
    //.valid_in (sample_valid_d1),
    .valid_in  (sample_valid_d1 | gap_active_d1),
    .tx_real   (tx_real),
    .tx_imag   (tx_imag),
    .valid_out (tx_valid)
);

endmodule

```

2.2.24 chirp_rom:

2.2.24.1 Purpose

Stores the 4 pre-computed chirp sequences (chirp_index 0-3, corresponding to the standard's $m = 1-4$) as constant, synthesis-time data. Given an address built from which sequence / which subchirp / which sample, returns the corresponding real and imaginary fixed-point sample. This is the "CSS Symbols ROM" block in Figure 3-3 of the thesis document.

2.2.24.2 Design decisions

Two separate ROMs (rom_real_mem, rom_imag_mem), sharing one address bus — chosen over a single wide ROM because it mirrors how the rest of the design already treats real/imaginary as separate signals, and is easier to verify independently against the MATLAB reference files.

Address scheme: a straight concatenation, $\text{addr} = \{\text{chirp_index}[1:0], \text{subchirp_sel}[1:0], \text{sample_addr}[5:0]\}$ — 10 bits, no arithmetic needed to compute an address. Only 608 of the 1024 addressable locations are meaningful (sample_addr only uses 0-37 of the 6-bit range).

Word length: 5 bits signed per sample ($R = 5$, from Section 2), giving a range of -16 to +15.

Unused-address behavior: for sample_addr > 37 (the 38-63 range that the address scheme wastes), the ROM explicitly outputs 0 rather than an arbitrary don't-care value, keeping simulation and downstream logic predictable.

Read timing: synchronous, registered read — rom_real/rom_imag reflect the address that was present ONE CYCLE EARLIER, not the current cycle's address. This is a standard, expected characteristic of a Block-RAM-style ROM, but it is a critical timing fact the top-level wrapper has to compensate for (Section 6).

ROM contents are loaded from external files (chirp_rom_real.mem, chirp_rom_imag.mem) via \$readmemb, generated from the validated MATLAB chirp sequence generator with the Section 2 quantization rule applied — never hand-typed.

2.2.24.3 Interface

Signal	Direction	Width	Description
clk	IN	1	System clock
chirp_index	IN	2	Which of the 4 chirp sequences (0-3 <-> standard m=1-4)
subchirp_sel	IN	2	Which subchirp within the sequence (k = 0-3)
sample_addr	IN	6	Sample index within the subchirp (0-37 valid; 38-63 -> outputs 0)
rom_real	OUT	5 (signed)	Real part of the addressed chirp sample, registered (1-cycle latency)
rom_imag	OUT	5 (signed)	Imaginary part of the addressed chirp sample, registered (1-cycle latency)

2.2.24.4 RTL — chirp_rom.v

```
// =====
// Precision:    R = 5 bits (Signed 2's Complement: -16 to +15)
// Word Size:    5 bits for rom_real, 5 bits for rom_imag
//
// Address Map:  10 bits total
//                [9:8] = chirp_index  (0 to 3  <-> m = 1 to 4)
//                [7:6] = subchirp_sel (0 to 3  <-> k = 0 to 3)
//                [5:0] = sample_addr  (0 to 37 valid; 38 to 63 unused)
// =====
module chirp_rom (
    input wire      clk,
    input wire [1:0] chirp_index,
    input wire [1:0] subchirp_sel,
    input wire [5:0] sample_addr,
    output reg [4:0] rom_real,
    output reg [4:0] rom_imag
);

    // 10-Bit Address Concatenation

    wire [9:0] addr;
    assign addr = {chirp_index, subchirp_sel, sample_addr};

    // made in this way to be synthesizable
    always @(posedge clk) begin
        case (addr)
            10'd 0: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end

```

```

10'd 1: begin rom_real <= 5'b00000; rom_imag <= 5'b11111; end
10'd 2: begin rom_real <= 5'b11111; rom_imag <= 5'b11101; end
10'd 3: begin rom_real <= 5'b11010; rom_imag <= 5'b00000; end
10'd 4: begin rom_real <= 5'b11110; rom_imag <= 5'b00111; end
10'd 5: begin rom_real <= 5'b01001; rom_imag <= 5'b00101; end
10'd 6: begin rom_real <= 5'b01001; rom_imag <= 5'b10111; end
10'd 7: begin rom_real <= 5'b11011; rom_imag <= 5'b10010; end
10'd 8: begin rom_real <= 5'b10001; rom_imag <= 5'b11110; end
10'd 9: begin rom_real <= 5'b10111; rom_imag <= 5'b01100; end
10'd 10: begin rom_real <= 5'b00110; rom_imag <= 5'b01101; end
10'd 11: begin rom_real <= 5'b01110; rom_imag <= 5'b00001; end
10'd 12: begin rom_real <= 5'b01001; rom_imag <= 5'b10100; end
10'd 13: begin rom_real <= 5'b11101; rom_imag <= 5'b10001; end
10'd 14: begin rom_real <= 5'b10010; rom_imag <= 5'b11000; end
10'd 15: begin rom_real <= 5'b10001; rom_imag <= 5'b00100; end
10'd 16: begin rom_real <= 5'b11000; rom_imag <= 5'b01101; end
10'd 17: begin rom_real <= 5'b00011; rom_imag <= 5'b01110; end
10'd 18: begin rom_real <= 5'b01100; rom_imag <= 5'b01000; end
10'd 19: begin rom_real <= 5'b01111; rom_imag <= 5'b00000; end
10'd 20: begin rom_real <= 5'b01100; rom_imag <= 5'b10111; end
10'd 21: begin rom_real <= 5'b00110; rom_imag <= 5'b10010; end
10'd 22: begin rom_real <= 5'b11110; rom_imag <= 5'b10001; end
10'd 23: begin rom_real <= 5'b11000; rom_imag <= 5'b10011; end
10'd 24: begin rom_real <= 5'b10011; rom_imag <= 5'b10111; end
10'd 25: begin rom_real <= 5'b10001; rom_imag <= 5'b11100; end
10'd 26: begin rom_real <= 5'b10001; rom_imag <= 5'b00001; end
10'd 27: begin rom_real <= 5'b10010; rom_imag <= 5'b00101; end
10'd 28: begin rom_real <= 5'b10011; rom_imag <= 5'b01000; end
10'd 29: begin rom_real <= 5'b10101; rom_imag <= 5'b01010; end
10'd 30: begin rom_real <= 5'b10111; rom_imag <= 5'b01100; end
10'd 31: begin rom_real <= 5'b11000; rom_imag <= 5'b01100; end
10'd 32: begin rom_real <= 5'b11001; rom_imag <= 5'b01011; end
10'd 33: begin rom_real <= 5'b11011; rom_imag <= 5'b01001; end
10'd 34: begin rom_real <= 5'b11100; rom_imag <= 5'b00111; end
10'd 35: begin rom_real <= 5'b11101; rom_imag <= 5'b00100; end
10'd 36: begin rom_real <= 5'b11110; rom_imag <= 5'b00001; end
10'd 37: begin rom_real <= 5'b11111; rom_imag <= 5'b00000; end
10'd 64: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 65: begin rom_real <= 5'b11111; rom_imag <= 5'b00000; end
10'd 66: begin rom_real <= 5'b11110; rom_imag <= 5'b00001; end
10'd 67: begin rom_real <= 5'b11101; rom_imag <= 5'b00100; end
10'd 68: begin rom_real <= 5'b11100; rom_imag <= 5'b00111; end
10'd 69: begin rom_real <= 5'b11011; rom_imag <= 5'b01001; end
10'd 70: begin rom_real <= 5'b11001; rom_imag <= 5'b01011; end
10'd 71: begin rom_real <= 5'b11000; rom_imag <= 5'b01100; end
10'd 72: begin rom_real <= 5'b10111; rom_imag <= 5'b01100; end
10'd 73: begin rom_real <= 5'b10101; rom_imag <= 5'b01010; end
10'd 74: begin rom_real <= 5'b10011; rom_imag <= 5'b01000; end
10'd 75: begin rom_real <= 5'b10010; rom_imag <= 5'b00101; end
10'd 76: begin rom_real <= 5'b10001; rom_imag <= 5'b00001; end
10'd 77: begin rom_real <= 5'b10001; rom_imag <= 5'b11100; end
10'd 78: begin rom_real <= 5'b10011; rom_imag <= 5'b10111; end
10'd 79: begin rom_real <= 5'b11000; rom_imag <= 5'b10011; end
10'd 80: begin rom_real <= 5'b11110; rom_imag <= 5'b10001; end
10'd 81: begin rom_real <= 5'b00110; rom_imag <= 5'b10010; end
10'd 82: begin rom_real <= 5'b01100; rom_imag <= 5'b10111; end
10'd 83: begin rom_real <= 5'b01111; rom_imag <= 5'b00000; end
10'd 84: begin rom_real <= 5'b01100; rom_imag <= 5'b01000; end
10'd 85: begin rom_real <= 5'b00011; rom_imag <= 5'b01110; end
10'd 86: begin rom_real <= 5'b11000; rom_imag <= 5'b01101; end
10'd 87: begin rom_real <= 5'b10001; rom_imag <= 5'b00100; end
10'd 88: begin rom_real <= 5'b10010; rom_imag <= 5'b11000; end
10'd 89: begin rom_real <= 5'b11101; rom_imag <= 5'b10001; end
10'd 90: begin rom_real <= 5'b01001; rom_imag <= 5'b10100; end
10'd 91: begin rom_real <= 5'b01110; rom_imag <= 5'b00001; end
10'd 92: begin rom_real <= 5'b00110; rom_imag <= 5'b01101; end
10'd 93: begin rom_real <= 5'b10111; rom_imag <= 5'b01100; end
10'd 94: begin rom_real <= 5'b10001; rom_imag <= 5'b11110; end
10'd 95: begin rom_real <= 5'b11011; rom_imag <= 5'b10010; end

```

```

10'd 96: begin rom_real <= 5'b01001; rom_imag <= 5'b10111; end
10'd 97: begin rom_real <= 5'b01001; rom_imag <= 5'b00101; end
10'd 98: begin rom_real <= 5'b11110; rom_imag <= 5'b00111; end
10'd 99: begin rom_real <= 5'b11010; rom_imag <= 5'b00000; end
10'd 100: begin rom_real <= 5'b11111; rom_imag <= 5'b11101; end
10'd 101: begin rom_real <= 5'b00000; rom_imag <= 5'b11111; end
10'd 128: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 129: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 130: begin rom_real <= 5'b11111; rom_imag <= 5'b00010; end
10'd 131: begin rom_real <= 5'b11010; rom_imag <= 5'b11111; end
10'd 132: begin rom_real <= 5'b11110; rom_imag <= 5'b11000; end
10'd 133: begin rom_real <= 5'b01001; rom_imag <= 5'b11010; end
10'd 134: begin rom_real <= 5'b01001; rom_imag <= 5'b01000; end
10'd 135: begin rom_real <= 5'b11011; rom_imag <= 5'b01101; end
10'd 136: begin rom_real <= 5'b10001; rom_imag <= 5'b00001; end
10'd 137: begin rom_real <= 5'b10111; rom_imag <= 5'b10011; end
10'd 138: begin rom_real <= 5'b00110; rom_imag <= 5'b10010; end
10'd 139: begin rom_real <= 5'b01110; rom_imag <= 5'b11110; end
10'd 140: begin rom_real <= 5'b01001; rom_imag <= 5'b01011; end
10'd 141: begin rom_real <= 5'b11101; rom_imag <= 5'b01110; end
10'd 142: begin rom_real <= 5'b10010; rom_imag <= 5'b00111; end
10'd 143: begin rom_real <= 5'b10001; rom_imag <= 5'b11011; end
10'd 144: begin rom_real <= 5'b11000; rom_imag <= 5'b10010; end
10'd 145: begin rom_real <= 5'b00011; rom_imag <= 5'b10001; end
10'd 146: begin rom_real <= 5'b01100; rom_imag <= 5'b10111; end
10'd 147: begin rom_real <= 5'b01111; rom_imag <= 5'b00000; end
10'd 148: begin rom_real <= 5'b01100; rom_imag <= 5'b01000; end
10'd 149: begin rom_real <= 5'b00110; rom_imag <= 5'b01101; end
10'd 150: begin rom_real <= 5'b11110; rom_imag <= 5'b01110; end
10'd 151: begin rom_real <= 5'b11000; rom_imag <= 5'b01100; end
10'd 152: begin rom_real <= 5'b10011; rom_imag <= 5'b01000; end
10'd 153: begin rom_real <= 5'b10001; rom_imag <= 5'b00011; end
10'd 154: begin rom_real <= 5'b10001; rom_imag <= 5'b11110; end
10'd 155: begin rom_real <= 5'b10010; rom_imag <= 5'b11010; end
10'd 156: begin rom_real <= 5'b10011; rom_imag <= 5'b10111; end
10'd 157: begin rom_real <= 5'b10101; rom_imag <= 5'b10101; end
10'd 158: begin rom_real <= 5'b10111; rom_imag <= 5'b10011; end
10'd 159: begin rom_real <= 5'b11000; rom_imag <= 5'b10011; end
10'd 160: begin rom_real <= 5'b11001; rom_imag <= 5'b10100; end
10'd 161: begin rom_real <= 5'b11011; rom_imag <= 5'b10110; end
10'd 162: begin rom_real <= 5'b11100; rom_imag <= 5'b11000; end
10'd 163: begin rom_real <= 5'b11101; rom_imag <= 5'b11011; end
10'd 164: begin rom_real <= 5'b11110; rom_imag <= 5'b11110; end
10'd 165: begin rom_real <= 5'b11111; rom_imag <= 5'b11111; end
10'd 192: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 193: begin rom_real <= 5'b11111; rom_imag <= 5'b11111; end
10'd 194: begin rom_real <= 5'b11110; rom_imag <= 5'b11110; end
10'd 195: begin rom_real <= 5'b11101; rom_imag <= 5'b11011; end
10'd 196: begin rom_real <= 5'b11100; rom_imag <= 5'b11000; end
10'd 197: begin rom_real <= 5'b11011; rom_imag <= 5'b10110; end
10'd 198: begin rom_real <= 5'b11001; rom_imag <= 5'b10100; end
10'd 199: begin rom_real <= 5'b11000; rom_imag <= 5'b10011; end
10'd 200: begin rom_real <= 5'b10111; rom_imag <= 5'b10011; end
10'd 201: begin rom_real <= 5'b10101; rom_imag <= 5'b10101; end
10'd 202: begin rom_real <= 5'b10011; rom_imag <= 5'b10111; end
10'd 203: begin rom_real <= 5'b10010; rom_imag <= 5'b11010; end
10'd 204: begin rom_real <= 5'b10001; rom_imag <= 5'b11110; end
10'd 205: begin rom_real <= 5'b10001; rom_imag <= 5'b00011; end
10'd 206: begin rom_real <= 5'b10011; rom_imag <= 5'b01000; end
10'd 207: begin rom_real <= 5'b11000; rom_imag <= 5'b01100; end
10'd 208: begin rom_real <= 5'b11110; rom_imag <= 5'b01110; end
10'd 209: begin rom_real <= 5'b00110; rom_imag <= 5'b01101; end
10'd 210: begin rom_real <= 5'b01100; rom_imag <= 5'b01000; end
10'd 211: begin rom_real <= 5'b01111; rom_imag <= 5'b00000; end
10'd 212: begin rom_real <= 5'b01100; rom_imag <= 5'b10111; end
10'd 213: begin rom_real <= 5'b00011; rom_imag <= 5'b10001; end
10'd 214: begin rom_real <= 5'b11000; rom_imag <= 5'b10010; end
10'd 215: begin rom_real <= 5'b10001; rom_imag <= 5'b11011; end
10'd 216: begin rom_real <= 5'b10010; rom_imag <= 5'b00111; end

```

```

10'd 217: begin rom_real <= 5'b11101; rom_imag <= 5'b01110; end
10'd 218: begin rom_real <= 5'b01001; rom_imag <= 5'b01011; end
10'd 219: begin rom_real <= 5'b01110; rom_imag <= 5'b11110; end
10'd 220: begin rom_real <= 5'b00110; rom_imag <= 5'b10010; end
10'd 221: begin rom_real <= 5'b10111; rom_imag <= 5'b10011; end
10'd 222: begin rom_real <= 5'b10001; rom_imag <= 5'b00001; end
10'd 223: begin rom_real <= 5'b11011; rom_imag <= 5'b01101; end
10'd 224: begin rom_real <= 5'b01001; rom_imag <= 5'b01000; end
10'd 225: begin rom_real <= 5'b01001; rom_imag <= 5'b11010; end
10'd 226: begin rom_real <= 5'b11110; rom_imag <= 5'b11000; end
10'd 227: begin rom_real <= 5'b11010; rom_imag <= 5'b11111; end
10'd 228: begin rom_real <= 5'b11111; rom_imag <= 5'b00010; end
10'd 229: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 256: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 257: begin rom_real <= 5'b11111; rom_imag <= 5'b00000; end
10'd 258: begin rom_real <= 5'b11110; rom_imag <= 5'b00001; end
10'd 259: begin rom_real <= 5'b11101; rom_imag <= 5'b00100; end
10'd 260: begin rom_real <= 5'b11100; rom_imag <= 5'b00111; end
10'd 261: begin rom_real <= 5'b11011; rom_imag <= 5'b01001; end
10'd 262: begin rom_real <= 5'b11001; rom_imag <= 5'b01011; end
10'd 263: begin rom_real <= 5'b11000; rom_imag <= 5'b01100; end
10'd 264: begin rom_real <= 5'b10111; rom_imag <= 5'b01100; end
10'd 265: begin rom_real <= 5'b10101; rom_imag <= 5'b01010; end
10'd 266: begin rom_real <= 5'b10011; rom_imag <= 5'b01000; end
10'd 267: begin rom_real <= 5'b10010; rom_imag <= 5'b00101; end
10'd 268: begin rom_real <= 5'b10001; rom_imag <= 5'b00001; end
10'd 269: begin rom_real <= 5'b10001; rom_imag <= 5'b11100; end
10'd 270: begin rom_real <= 5'b10011; rom_imag <= 5'b10111; end
10'd 271: begin rom_real <= 5'b11000; rom_imag <= 5'b10011; end
10'd 272: begin rom_real <= 5'b11110; rom_imag <= 5'b10001; end
10'd 273: begin rom_real <= 5'b00110; rom_imag <= 5'b10010; end
10'd 274: begin rom_real <= 5'b01100; rom_imag <= 5'b10111; end
10'd 275: begin rom_real <= 5'b01111; rom_imag <= 5'b00000; end
10'd 276: begin rom_real <= 5'b01100; rom_imag <= 5'b01000; end
10'd 277: begin rom_real <= 5'b00011; rom_imag <= 5'b01110; end
10'd 278: begin rom_real <= 5'b11000; rom_imag <= 5'b01101; end
10'd 279: begin rom_real <= 5'b10001; rom_imag <= 5'b00100; end
10'd 280: begin rom_real <= 5'b10010; rom_imag <= 5'b11000; end
10'd 281: begin rom_real <= 5'b11101; rom_imag <= 5'b10001; end
10'd 282: begin rom_real <= 5'b01001; rom_imag <= 5'b10100; end
10'd 283: begin rom_real <= 5'b01110; rom_imag <= 5'b00001; end
10'd 284: begin rom_real <= 5'b00110; rom_imag <= 5'b01101; end
10'd 285: begin rom_real <= 5'b10111; rom_imag <= 5'b01100; end
10'd 286: begin rom_real <= 5'b10001; rom_imag <= 5'b11110; end
10'd 287: begin rom_real <= 5'b11011; rom_imag <= 5'b10010; end
10'd 288: begin rom_real <= 5'b01001; rom_imag <= 5'b10111; end
10'd 289: begin rom_real <= 5'b01001; rom_imag <= 5'b00101; end
10'd 290: begin rom_real <= 5'b11110; rom_imag <= 5'b00111; end
10'd 291: begin rom_real <= 5'b11010; rom_imag <= 5'b00000; end
10'd 292: begin rom_real <= 5'b11111; rom_imag <= 5'b11101; end
10'd 293: begin rom_real <= 5'b00000; rom_imag <= 5'b11111; end
10'd 320: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 321: begin rom_real <= 5'b11111; rom_imag <= 5'b11111; end
10'd 322: begin rom_real <= 5'b11110; rom_imag <= 5'b11110; end
10'd 323: begin rom_real <= 5'b11101; rom_imag <= 5'b11011; end
10'd 324: begin rom_real <= 5'b11100; rom_imag <= 5'b11000; end
10'd 325: begin rom_real <= 5'b11011; rom_imag <= 5'b10110; end
10'd 326: begin rom_real <= 5'b11001; rom_imag <= 5'b10100; end
10'd 327: begin rom_real <= 5'b11000; rom_imag <= 5'b10011; end
10'd 328: begin rom_real <= 5'b10111; rom_imag <= 5'b10011; end
10'd 329: begin rom_real <= 5'b10101; rom_imag <= 5'b10101; end
10'd 330: begin rom_real <= 5'b10011; rom_imag <= 5'b10111; end
10'd 331: begin rom_real <= 5'b10010; rom_imag <= 5'b11010; end
10'd 332: begin rom_real <= 5'b10001; rom_imag <= 5'b11110; end
10'd 333: begin rom_real <= 5'b10001; rom_imag <= 5'b00011; end
10'd 334: begin rom_real <= 5'b10011; rom_imag <= 5'b01000; end
10'd 335: begin rom_real <= 5'b11000; rom_imag <= 5'b01100; end
10'd 336: begin rom_real <= 5'b11110; rom_imag <= 5'b01110; end
10'd 337: begin rom_real <= 5'b00110; rom_imag <= 5'b01101; end

```

```

10'd 338: begin rom_real <= 5'b01100; rom_imag <= 5'b01000; end
10'd 339: begin rom_real <= 5'b01111; rom_imag <= 5'b00000; end
10'd 340: begin rom_real <= 5'b01100; rom_imag <= 5'b10111; end
10'd 341: begin rom_real <= 5'b00011; rom_imag <= 5'b10001; end
10'd 342: begin rom_real <= 5'b11000; rom_imag <= 5'b10010; end
10'd 343: begin rom_real <= 5'b10001; rom_imag <= 5'b11011; end
10'd 344: begin rom_real <= 5'b10010; rom_imag <= 5'b00111; end
10'd 345: begin rom_real <= 5'b11101; rom_imag <= 5'b01110; end
10'd 346: begin rom_real <= 5'b01001; rom_imag <= 5'b01011; end
10'd 347: begin rom_real <= 5'b01110; rom_imag <= 5'b11110; end
10'd 348: begin rom_real <= 5'b00110; rom_imag <= 5'b10010; end
10'd 349: begin rom_real <= 5'b10111; rom_imag <= 5'b10011; end
10'd 350: begin rom_real <= 5'b10001; rom_imag <= 5'b00001; end
10'd 351: begin rom_real <= 5'b11011; rom_imag <= 5'b01101; end
10'd 352: begin rom_real <= 5'b01001; rom_imag <= 5'b01000; end
10'd 353: begin rom_real <= 5'b01001; rom_imag <= 5'b11010; end
10'd 354: begin rom_real <= 5'b11110; rom_imag <= 5'b11000; end
10'd 355: begin rom_real <= 5'b11010; rom_imag <= 5'b11111; end
10'd 356: begin rom_real <= 5'b11111; rom_imag <= 5'b00010; end
10'd 357: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 384: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 385: begin rom_real <= 5'b00000; rom_imag <= 5'b11111; end
10'd 386: begin rom_real <= 5'b11111; rom_imag <= 5'b11101; end
10'd 387: begin rom_real <= 5'b11010; rom_imag <= 5'b00000; end
10'd 388: begin rom_real <= 5'b11110; rom_imag <= 5'b00111; end
10'd 389: begin rom_real <= 5'b01001; rom_imag <= 5'b00101; end
10'd 390: begin rom_real <= 5'b01001; rom_imag <= 5'b10111; end
10'd 391: begin rom_real <= 5'b11011; rom_imag <= 5'b10010; end
10'd 392: begin rom_real <= 5'b10001; rom_imag <= 5'b11110; end
10'd 393: begin rom_real <= 5'b10111; rom_imag <= 5'b01100; end
10'd 394: begin rom_real <= 5'b00110; rom_imag <= 5'b01101; end
10'd 395: begin rom_real <= 5'b01110; rom_imag <= 5'b00001; end
10'd 396: begin rom_real <= 5'b01001; rom_imag <= 5'b10100; end
10'd 397: begin rom_real <= 5'b11101; rom_imag <= 5'b10001; end
10'd 398: begin rom_real <= 5'b10010; rom_imag <= 5'b11000; end
10'd 399: begin rom_real <= 5'b10001; rom_imag <= 5'b00100; end
10'd 400: begin rom_real <= 5'b11000; rom_imag <= 5'b01101; end
10'd 401: begin rom_real <= 5'b00011; rom_imag <= 5'b01110; end
10'd 402: begin rom_real <= 5'b01100; rom_imag <= 5'b01000; end
10'd 403: begin rom_real <= 5'b01111; rom_imag <= 5'b00000; end
10'd 404: begin rom_real <= 5'b01100; rom_imag <= 5'b10111; end
10'd 405: begin rom_real <= 5'b00110; rom_imag <= 5'b10010; end
10'd 406: begin rom_real <= 5'b11110; rom_imag <= 5'b10001; end
10'd 407: begin rom_real <= 5'b11000; rom_imag <= 5'b10011; end
10'd 408: begin rom_real <= 5'b10011; rom_imag <= 5'b10111; end
10'd 409: begin rom_real <= 5'b10001; rom_imag <= 5'b11100; end
10'd 410: begin rom_real <= 5'b10001; rom_imag <= 5'b00001; end
10'd 411: begin rom_real <= 5'b10010; rom_imag <= 5'b00101; end
10'd 412: begin rom_real <= 5'b10011; rom_imag <= 5'b01000; end
10'd 413: begin rom_real <= 5'b10101; rom_imag <= 5'b01010; end
10'd 414: begin rom_real <= 5'b10111; rom_imag <= 5'b01100; end
10'd 415: begin rom_real <= 5'b11000; rom_imag <= 5'b01100; end
10'd 416: begin rom_real <= 5'b11001; rom_imag <= 5'b01011; end
10'd 417: begin rom_real <= 5'b11011; rom_imag <= 5'b01001; end
10'd 418: begin rom_real <= 5'b11100; rom_imag <= 5'b00111; end
10'd 419: begin rom_real <= 5'b11101; rom_imag <= 5'b00100; end
10'd 420: begin rom_real <= 5'b11110; rom_imag <= 5'b00001; end
10'd 421: begin rom_real <= 5'b11111; rom_imag <= 5'b00000; end
10'd 448: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 449: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 450: begin rom_real <= 5'b11111; rom_imag <= 5'b00010; end
10'd 451: begin rom_real <= 5'b11010; rom_imag <= 5'b11111; end
10'd 452: begin rom_real <= 5'b11110; rom_imag <= 5'b11000; end
10'd 453: begin rom_real <= 5'b01001; rom_imag <= 5'b11010; end
10'd 454: begin rom_real <= 5'b01001; rom_imag <= 5'b01000; end
10'd 455: begin rom_real <= 5'b11011; rom_imag <= 5'b01101; end
10'd 456: begin rom_real <= 5'b10001; rom_imag <= 5'b00001; end
10'd 457: begin rom_real <= 5'b10111; rom_imag <= 5'b10011; end
10'd 458: begin rom_real <= 5'b00110; rom_imag <= 5'b10010; end

```

```

10'd 459: begin rom_real <= 5'b01110; rom_imag <= 5'b11110; end
10'd 460: begin rom_real <= 5'b01001; rom_imag <= 5'b01011; end
10'd 461: begin rom_real <= 5'b11101; rom_imag <= 5'b01110; end
10'd 462: begin rom_real <= 5'b10010; rom_imag <= 5'b00111; end
10'd 463: begin rom_real <= 5'b10001; rom_imag <= 5'b11011; end
10'd 464: begin rom_real <= 5'b11000; rom_imag <= 5'b10010; end
10'd 465: begin rom_real <= 5'b00011; rom_imag <= 5'b10001; end
10'd 466: begin rom_real <= 5'b01100; rom_imag <= 5'b10111; end
10'd 467: begin rom_real <= 5'b01111; rom_imag <= 5'b00000; end
10'd 468: begin rom_real <= 5'b01100; rom_imag <= 5'b01000; end
10'd 469: begin rom_real <= 5'b00110; rom_imag <= 5'b01101; end
10'd 470: begin rom_real <= 5'b11110; rom_imag <= 5'b01110; end
10'd 471: begin rom_real <= 5'b11000; rom_imag <= 5'b01100; end
10'd 472: begin rom_real <= 5'b10011; rom_imag <= 5'b01000; end
10'd 473: begin rom_real <= 5'b10001; rom_imag <= 5'b00011; end
10'd 474: begin rom_real <= 5'b10001; rom_imag <= 5'b11110; end
10'd 475: begin rom_real <= 5'b10010; rom_imag <= 5'b11010; end
10'd 476: begin rom_real <= 5'b10011; rom_imag <= 5'b10111; end
10'd 477: begin rom_real <= 5'b10101; rom_imag <= 5'b10101; end
10'd 478: begin rom_real <= 5'b10111; rom_imag <= 5'b10011; end
10'd 479: begin rom_real <= 5'b11000; rom_imag <= 5'b10011; end
10'd 480: begin rom_real <= 5'b11001; rom_imag <= 5'b10100; end
10'd 481: begin rom_real <= 5'b11011; rom_imag <= 5'b10110; end
10'd 482: begin rom_real <= 5'b11100; rom_imag <= 5'b11000; end
10'd 483: begin rom_real <= 5'b11101; rom_imag <= 5'b11011; end
10'd 484: begin rom_real <= 5'b11110; rom_imag <= 5'b11110; end
10'd 485: begin rom_real <= 5'b11111; rom_imag <= 5'b11111; end
10'd 512: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 513: begin rom_real <= 5'b11111; rom_imag <= 5'b11111; end
10'd 514: begin rom_real <= 5'b11110; rom_imag <= 5'b11110; end
10'd 515: begin rom_real <= 5'b11101; rom_imag <= 5'b11011; end
10'd 516: begin rom_real <= 5'b11100; rom_imag <= 5'b11000; end
10'd 517: begin rom_real <= 5'b11011; rom_imag <= 5'b10110; end
10'd 518: begin rom_real <= 5'b11001; rom_imag <= 5'b10100; end
10'd 519: begin rom_real <= 5'b11000; rom_imag <= 5'b10011; end
10'd 520: begin rom_real <= 5'b10111; rom_imag <= 5'b10011; end
10'd 521: begin rom_real <= 5'b10101; rom_imag <= 5'b10101; end
10'd 522: begin rom_real <= 5'b10011; rom_imag <= 5'b10111; end
10'd 523: begin rom_real <= 5'b10010; rom_imag <= 5'b11010; end
10'd 524: begin rom_real <= 5'b10001; rom_imag <= 5'b11110; end
10'd 525: begin rom_real <= 5'b10001; rom_imag <= 5'b00011; end
10'd 526: begin rom_real <= 5'b10011; rom_imag <= 5'b01000; end
10'd 527: begin rom_real <= 5'b11000; rom_imag <= 5'b01100; end
10'd 528: begin rom_real <= 5'b11110; rom_imag <= 5'b01110; end
10'd 529: begin rom_real <= 5'b00110; rom_imag <= 5'b01101; end
10'd 530: begin rom_real <= 5'b01100; rom_imag <= 5'b01000; end
10'd 531: begin rom_real <= 5'b01111; rom_imag <= 5'b00000; end
10'd 532: begin rom_real <= 5'b01100; rom_imag <= 5'b10111; end
10'd 533: begin rom_real <= 5'b00011; rom_imag <= 5'b10001; end
10'd 534: begin rom_real <= 5'b11000; rom_imag <= 5'b10010; end
10'd 535: begin rom_real <= 5'b10001; rom_imag <= 5'b11011; end
10'd 536: begin rom_real <= 5'b10010; rom_imag <= 5'b00111; end
10'd 537: begin rom_real <= 5'b11101; rom_imag <= 5'b01110; end
10'd 538: begin rom_real <= 5'b01001; rom_imag <= 5'b01011; end
10'd 539: begin rom_real <= 5'b01110; rom_imag <= 5'b11110; end
10'd 540: begin rom_real <= 5'b00110; rom_imag <= 5'b10010; end
10'd 541: begin rom_real <= 5'b10111; rom_imag <= 5'b10011; end
10'd 542: begin rom_real <= 5'b10001; rom_imag <= 5'b00001; end
10'd 543: begin rom_real <= 5'b11011; rom_imag <= 5'b01101; end
10'd 544: begin rom_real <= 5'b01001; rom_imag <= 5'b01000; end
10'd 545: begin rom_real <= 5'b01001; rom_imag <= 5'b11010; end
10'd 546: begin rom_real <= 5'b11110; rom_imag <= 5'b11000; end
10'd 547: begin rom_real <= 5'b11010; rom_imag <= 5'b11111; end
10'd 548: begin rom_real <= 5'b11111; rom_imag <= 5'b00010; end
10'd 549: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 576: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 577: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 578: begin rom_real <= 5'b11111; rom_imag <= 5'b00010; end
10'd 579: begin rom_real <= 5'b11010; rom_imag <= 5'b11111; end

```



```

10'd 675: begin rom_real <= 5'b11010; rom_imag <= 5'b00000; end
10'd 676: begin rom_real <= 5'b11111; rom_imag <= 5'b11101; end
10'd 677: begin rom_real <= 5'b00000; rom_imag <= 5'b11111; end
10'd 704: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 705: begin rom_real <= 5'b00000; rom_imag <= 5'b11111; end
10'd 706: begin rom_real <= 5'b11111; rom_imag <= 5'b11101; end
10'd 707: begin rom_real <= 5'b11010; rom_imag <= 5'b00000; end
10'd 708: begin rom_real <= 5'b11110; rom_imag <= 5'b00111; end
10'd 709: begin rom_real <= 5'b01001; rom_imag <= 5'b00101; end
10'd 710: begin rom_real <= 5'b01001; rom_imag <= 5'b10111; end
10'd 711: begin rom_real <= 5'b11011; rom_imag <= 5'b10010; end
10'd 712: begin rom_real <= 5'b10001; rom_imag <= 5'b11110; end
10'd 713: begin rom_real <= 5'b10111; rom_imag <= 5'b01100; end
10'd 714: begin rom_real <= 5'b00110; rom_imag <= 5'b01101; end
10'd 715: begin rom_real <= 5'b01110; rom_imag <= 5'b00001; end
10'd 716: begin rom_real <= 5'b01001; rom_imag <= 5'b10100; end
10'd 717: begin rom_real <= 5'b11101; rom_imag <= 5'b10001; end
10'd 718: begin rom_real <= 5'b10010; rom_imag <= 5'b11000; end
10'd 719: begin rom_real <= 5'b10001; rom_imag <= 5'b00100; end
10'd 720: begin rom_real <= 5'b11000; rom_imag <= 5'b01101; end
10'd 721: begin rom_real <= 5'b00011; rom_imag <= 5'b01110; end
10'd 722: begin rom_real <= 5'b01100; rom_imag <= 5'b01000; end
10'd 723: begin rom_real <= 5'b01111; rom_imag <= 5'b00000; end
10'd 724: begin rom_real <= 5'b01100; rom_imag <= 5'b10111; end
10'd 725: begin rom_real <= 5'b00110; rom_imag <= 5'b10010; end
10'd 726: begin rom_real <= 5'b11110; rom_imag <= 5'b10001; end
10'd 727: begin rom_real <= 5'b11000; rom_imag <= 5'b10011; end
10'd 728: begin rom_real <= 5'b10011; rom_imag <= 5'b10111; end
10'd 729: begin rom_real <= 5'b10001; rom_imag <= 5'b11100; end
10'd 730: begin rom_real <= 5'b10001; rom_imag <= 5'b00001; end
10'd 731: begin rom_real <= 5'b10010; rom_imag <= 5'b00101; end
10'd 732: begin rom_real <= 5'b10011; rom_imag <= 5'b01000; end
10'd 733: begin rom_real <= 5'b10101; rom_imag <= 5'b01010; end
10'd 734: begin rom_real <= 5'b10111; rom_imag <= 5'b01100; end
10'd 735: begin rom_real <= 5'b11000; rom_imag <= 5'b01100; end
10'd 736: begin rom_real <= 5'b11001; rom_imag <= 5'b01011; end
10'd 737: begin rom_real <= 5'b11011; rom_imag <= 5'b01001; end
10'd 738: begin rom_real <= 5'b11100; rom_imag <= 5'b00111; end
10'd 739: begin rom_real <= 5'b11101; rom_imag <= 5'b00100; end
10'd 740: begin rom_real <= 5'b11110; rom_imag <= 5'b00001; end
10'd 741: begin rom_real <= 5'b11111; rom_imag <= 5'b00000; end
10'd 768: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 769: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 770: begin rom_real <= 5'b11111; rom_imag <= 5'b00010; end
10'd 771: begin rom_real <= 5'b11010; rom_imag <= 5'b11111; end
10'd 772: begin rom_real <= 5'b11110; rom_imag <= 5'b11000; end
10'd 773: begin rom_real <= 5'b01001; rom_imag <= 5'b11010; end
10'd 774: begin rom_real <= 5'b01001; rom_imag <= 5'b01000; end
10'd 775: begin rom_real <= 5'b11011; rom_imag <= 5'b01101; end
10'd 776: begin rom_real <= 5'b10001; rom_imag <= 5'b00001; end
10'd 777: begin rom_real <= 5'b10111; rom_imag <= 5'b10011; end
10'd 778: begin rom_real <= 5'b00110; rom_imag <= 5'b10010; end
10'd 779: begin rom_real <= 5'b01110; rom_imag <= 5'b11110; end
10'd 780: begin rom_real <= 5'b01001; rom_imag <= 5'b01011; end
10'd 781: begin rom_real <= 5'b11101; rom_imag <= 5'b01110; end
10'd 782: begin rom_real <= 5'b10010; rom_imag <= 5'b00111; end
10'd 783: begin rom_real <= 5'b10001; rom_imag <= 5'b11011; end
10'd 784: begin rom_real <= 5'b11000; rom_imag <= 5'b10010; end
10'd 785: begin rom_real <= 5'b00011; rom_imag <= 5'b10001; end
10'd 786: begin rom_real <= 5'b01100; rom_imag <= 5'b10111; end
10'd 787: begin rom_real <= 5'b01111; rom_imag <= 5'b00000; end
10'd 788: begin rom_real <= 5'b01100; rom_imag <= 5'b01000; end
10'd 789: begin rom_real <= 5'b00110; rom_imag <= 5'b01101; end
10'd 790: begin rom_real <= 5'b11110; rom_imag <= 5'b01110; end
10'd 791: begin rom_real <= 5'b11000; rom_imag <= 5'b01100; end
10'd 792: begin rom_real <= 5'b10011; rom_imag <= 5'b01000; end
10'd 793: begin rom_real <= 5'b10001; rom_imag <= 5'b00011; end
10'd 794: begin rom_real <= 5'b10001; rom_imag <= 5'b11110; end
10'd 795: begin rom_real <= 5'b10010; rom_imag <= 5'b11010; end

```

```

10'd 796: begin rom_real <= 5'b10011; rom_imag <= 5'b10111; end
10'd 797: begin rom_real <= 5'b10101; rom_imag <= 5'b10101; end
10'd 798: begin rom_real <= 5'b10111; rom_imag <= 5'b10011; end
10'd 799: begin rom_real <= 5'b11000; rom_imag <= 5'b10011; end
10'd 800: begin rom_real <= 5'b11001; rom_imag <= 5'b10100; end
10'd 801: begin rom_real <= 5'b11011; rom_imag <= 5'b10110; end
10'd 802: begin rom_real <= 5'b11100; rom_imag <= 5'b11000; end
10'd 803: begin rom_real <= 5'b11101; rom_imag <= 5'b11011; end
10'd 804: begin rom_real <= 5'b11110; rom_imag <= 5'b11110; end
10'd 805: begin rom_real <= 5'b11111; rom_imag <= 5'b11111; end
10'd 832: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 833: begin rom_real <= 5'b00000; rom_imag <= 5'b11111; end
10'd 834: begin rom_real <= 5'b11111; rom_imag <= 5'b11101; end
10'd 835: begin rom_real <= 5'b11010; rom_imag <= 5'b00000; end
10'd 836: begin rom_real <= 5'b11110; rom_imag <= 5'b00111; end
10'd 837: begin rom_real <= 5'b01001; rom_imag <= 5'b00101; end
10'd 838: begin rom_real <= 5'b01001; rom_imag <= 5'b10111; end
10'd 839: begin rom_real <= 5'b11011; rom_imag <= 5'b10010; end
10'd 840: begin rom_real <= 5'b10001; rom_imag <= 5'b11110; end
10'd 841: begin rom_real <= 5'b10111; rom_imag <= 5'b01100; end
10'd 842: begin rom_real <= 5'b00110; rom_imag <= 5'b01101; end
10'd 843: begin rom_real <= 5'b01110; rom_imag <= 5'b00001; end
10'd 844: begin rom_real <= 5'b01001; rom_imag <= 5'b10100; end
10'd 845: begin rom_real <= 5'b11101; rom_imag <= 5'b10001; end
10'd 846: begin rom_real <= 5'b10010; rom_imag <= 5'b11000; end
10'd 847: begin rom_real <= 5'b10001; rom_imag <= 5'b00100; end
10'd 848: begin rom_real <= 5'b11000; rom_imag <= 5'b01101; end
10'd 849: begin rom_real <= 5'b00011; rom_imag <= 5'b01110; end
10'd 850: begin rom_real <= 5'b01100; rom_imag <= 5'b01000; end
10'd 851: begin rom_real <= 5'b01111; rom_imag <= 5'b00000; end
10'd 852: begin rom_real <= 5'b01100; rom_imag <= 5'b10111; end
10'd 853: begin rom_real <= 5'b00110; rom_imag <= 5'b10010; end
10'd 854: begin rom_real <= 5'b11110; rom_imag <= 5'b10001; end
10'd 855: begin rom_real <= 5'b11000; rom_imag <= 5'b10011; end
10'd 856: begin rom_real <= 5'b10011; rom_imag <= 5'b10111; end
10'd 857: begin rom_real <= 5'b10001; rom_imag <= 5'b11100; end
10'd 858: begin rom_real <= 5'b10001; rom_imag <= 5'b00001; end
10'd 859: begin rom_real <= 5'b10010; rom_imag <= 5'b00101; end
10'd 860: begin rom_real <= 5'b10011; rom_imag <= 5'b01000; end
10'd 861: begin rom_real <= 5'b10101; rom_imag <= 5'b01010; end
10'd 862: begin rom_real <= 5'b10111; rom_imag <= 5'b01100; end
10'd 863: begin rom_real <= 5'b11000; rom_imag <= 5'b01100; end
10'd 864: begin rom_real <= 5'b11001; rom_imag <= 5'b01011; end
10'd 865: begin rom_real <= 5'b11011; rom_imag <= 5'b01001; end
10'd 866: begin rom_real <= 5'b11100; rom_imag <= 5'b00111; end
10'd 867: begin rom_real <= 5'b11101; rom_imag <= 5'b00100; end
10'd 868: begin rom_real <= 5'b11110; rom_imag <= 5'b00001; end
10'd 869: begin rom_real <= 5'b11111; rom_imag <= 5'b00000; end
10'd 896: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 897: begin rom_real <= 5'b11111; rom_imag <= 5'b11111; end
10'd 898: begin rom_real <= 5'b11110; rom_imag <= 5'b11110; end
10'd 899: begin rom_real <= 5'b11101; rom_imag <= 5'b11011; end
10'd 900: begin rom_real <= 5'b11100; rom_imag <= 5'b11000; end
10'd 901: begin rom_real <= 5'b11011; rom_imag <= 5'b10110; end
10'd 902: begin rom_real <= 5'b11001; rom_imag <= 5'b10100; end
10'd 903: begin rom_real <= 5'b11000; rom_imag <= 5'b10011; end
10'd 904: begin rom_real <= 5'b10111; rom_imag <= 5'b10011; end
10'd 905: begin rom_real <= 5'b10101; rom_imag <= 5'b10101; end
10'd 906: begin rom_real <= 5'b10011; rom_imag <= 5'b10111; end
10'd 907: begin rom_real <= 5'b10010; rom_imag <= 5'b11010; end
10'd 908: begin rom_real <= 5'b10001; rom_imag <= 5'b11110; end
10'd 909: begin rom_real <= 5'b10001; rom_imag <= 5'b00011; end
10'd 910: begin rom_real <= 5'b10011; rom_imag <= 5'b01000; end
10'd 911: begin rom_real <= 5'b11000; rom_imag <= 5'b01100; end
10'd 912: begin rom_real <= 5'b11110; rom_imag <= 5'b01110; end
10'd 913: begin rom_real <= 5'b00110; rom_imag <= 5'b01101; end
10'd 914: begin rom_real <= 5'b01100; rom_imag <= 5'b01000; end
10'd 915: begin rom_real <= 5'b01111; rom_imag <= 5'b00000; end
10'd 916: begin rom_real <= 5'b01100; rom_imag <= 5'b10111; end

```

```

10'd 917: begin rom_real <= 5'b00011; rom_imag <= 5'b10001; end
10'd 918: begin rom_real <= 5'b11000; rom_imag <= 5'b10010; end
10'd 919: begin rom_real <= 5'b10001; rom_imag <= 5'b11011; end
10'd 920: begin rom_real <= 5'b10010; rom_imag <= 5'b00111; end
10'd 921: begin rom_real <= 5'b11101; rom_imag <= 5'b01110; end
10'd 922: begin rom_real <= 5'b01001; rom_imag <= 5'b01011; end
10'd 923: begin rom_real <= 5'b01110; rom_imag <= 5'b11110; end
10'd 924: begin rom_real <= 5'b00110; rom_imag <= 5'b10010; end
10'd 925: begin rom_real <= 5'b10111; rom_imag <= 5'b10011; end
10'd 926: begin rom_real <= 5'b10001; rom_imag <= 5'b00001; end
10'd 927: begin rom_real <= 5'b11011; rom_imag <= 5'b01101; end
10'd 928: begin rom_real <= 5'b01001; rom_imag <= 5'b01000; end
10'd 929: begin rom_real <= 5'b01001; rom_imag <= 5'b11010; end
10'd 930: begin rom_real <= 5'b11110; rom_imag <= 5'b11000; end
10'd 931: begin rom_real <= 5'b11010; rom_imag <= 5'b11111; end
10'd 932: begin rom_real <= 5'b11111; rom_imag <= 5'b00010; end
10'd 933: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 960: begin rom_real <= 5'b00000; rom_imag <= 5'b00000; end
10'd 961: begin rom_real <= 5'b11111; rom_imag <= 5'b00000; end
10'd 962: begin rom_real <= 5'b11110; rom_imag <= 5'b00001; end
10'd 963: begin rom_real <= 5'b11101; rom_imag <= 5'b00100; end
10'd 964: begin rom_real <= 5'b11100; rom_imag <= 5'b00111; end
10'd 965: begin rom_real <= 5'b11011; rom_imag <= 5'b01001; end
10'd 966: begin rom_real <= 5'b11001; rom_imag <= 5'b01011; end
10'd 967: begin rom_real <= 5'b11000; rom_imag <= 5'b01100; end
10'd 968: begin rom_real <= 5'b10111; rom_imag <= 5'b01100; end
10'd 969: begin rom_real <= 5'b10101; rom_imag <= 5'b01010; end
10'd 970: begin rom_real <= 5'b10011; rom_imag <= 5'b01000; end
10'd 971: begin rom_real <= 5'b10010; rom_imag <= 5'b00101; end
10'd 972: begin rom_real <= 5'b10001; rom_imag <= 5'b00001; end
10'd 973: begin rom_real <= 5'b10001; rom_imag <= 5'b11100; end
10'd 974: begin rom_real <= 5'b10011; rom_imag <= 5'b10111; end
10'd 975: begin rom_real <= 5'b11000; rom_imag <= 5'b10011; end
10'd 976: begin rom_real <= 5'b11110; rom_imag <= 5'b10001; end
10'd 977: begin rom_real <= 5'b00110; rom_imag <= 5'b10010; end
10'd 978: begin rom_real <= 5'b01100; rom_imag <= 5'b10111; end
10'd 979: begin rom_real <= 5'b01111; rom_imag <= 5'b00000; end
10'd 980: begin rom_real <= 5'b01100; rom_imag <= 5'b01000; end
10'd 981: begin rom_real <= 5'b00011; rom_imag <= 5'b01110; end
10'd 982: begin rom_real <= 5'b11000; rom_imag <= 5'b01101; end
10'd 983: begin rom_real <= 5'b10001; rom_imag <= 5'b00100; end
10'd 984: begin rom_real <= 5'b10010; rom_imag <= 5'b11000; end
10'd 985: begin rom_real <= 5'b11101; rom_imag <= 5'b10001; end
10'd 986: begin rom_real <= 5'b01001; rom_imag <= 5'b10100; end
10'd 987: begin rom_real <= 5'b01110; rom_imag <= 5'b00001; end
10'd 988: begin rom_real <= 5'b00110; rom_imag <= 5'b01101; end
10'd 989: begin rom_real <= 5'b10111; rom_imag <= 5'b01100; end
10'd 990: begin rom_real <= 5'b10001; rom_imag <= 5'b11110; end
10'd 991: begin rom_real <= 5'b11011; rom_imag <= 5'b10010; end
10'd 992: begin rom_real <= 5'b01001; rom_imag <= 5'b10111; end
10'd 993: begin rom_real <= 5'b01001; rom_imag <= 5'b00101; end
10'd 994: begin rom_real <= 5'b11110; rom_imag <= 5'b00111; end
10'd 995: begin rom_real <= 5'b11010; rom_imag <= 5'b00000; end
10'd 996: begin rom_real <= 5'b11111; rom_imag <= 5'b11101; end
10'd 997: begin rom_real <= 5'b00000; rom_imag <= 5'b11111; end
    default: begin
        rom_real <= 5'sd0;
        rom_imag <= 5'sd0;
    end
endcase
end
endmodule

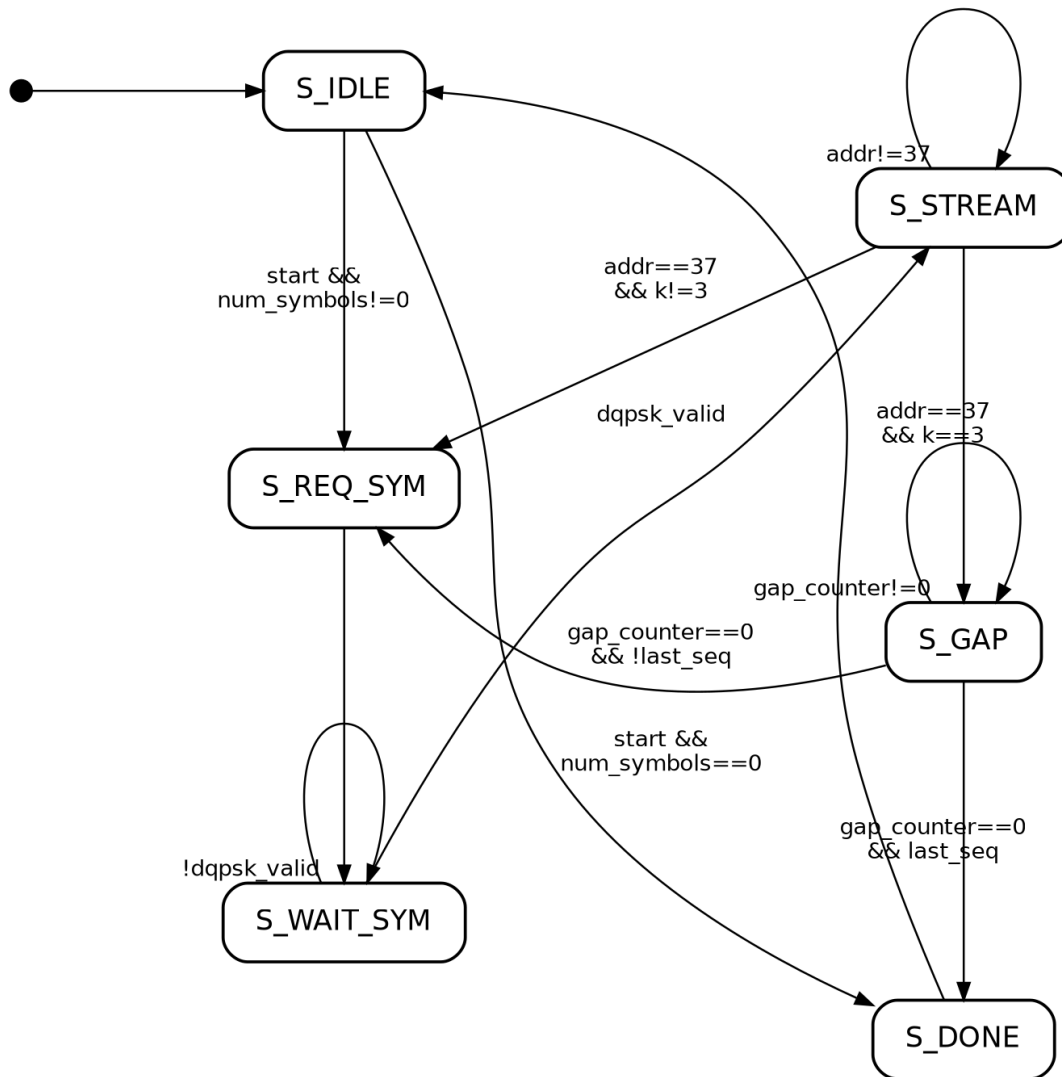
```

2.2.25 csk_generator:

2.2.25.1 Purpose

Walks the chirp_rom address bus (subchirp_sel, sample_addr) in step with the incoming DQPSK symbol stream, and inserts the correct inter-chirp-sequence time gap. Does not touch the ROM or the multiplier itself — this module only produces the address sequence, the latched sign bits for the multiplier, and the valid/gap timing.

2.2.25.2 Finite state machine



States:

- S_IDLE — waiting for a start pulse.
- S_REQ_SYM — asserts next_symbol_req for one cycle.
- S_WAIT_SYM — waits for dqpks_valid, then latches the sign bits.
- S_STREAM — walks sample_addr 0..37 for the current subchirp (k = subchirp_sel).

- S_GAP — counts down the inter-sequence gap. This state is entered after EVERY completed chirp sequence, including the last one in a packet.
- S_DONE — one-cycle done pulse, then returns to S_IDLE.

2.2.25.3 Defects identified during verification

Three defects were identified by the self-checking testbench rather than by inspection. Each is documented below together with the evidence that exposed it, since that evidence forms part of the justification for the corresponding correction.

2.2.25.4 Defect 1 — missing trailing gap

The original version skipped the gap after the LAST chirp sequence of a packet and went straight to done. Length arithmetic against the already-validated MATLAB reference proved this wrong:

152 samples/sequence x 48 sequences = 7296

confirmed TxchirpSequences length (25B, 1 Mb/s) = 9216

difference = 1920

1920 = 24*10 + 24*70 (24 even + 24 odd gaps, chirp_index=1)

This only balances if EVERY sequence, including the last, is followed by a gap. Fixed: the gap now always runs after a completed sequence; whether it is followed by DONE or by the next S_REQ_SYM depends on a latched last_seq flag (see Section 4.4).

2.2.25.5 Defect 2 — sample_valid misaligned with sample_addr by one cycle

sample_addr correctly reads 0 on the very first S_STREAM cycle (set up by the preceding S_WAIT_SYM cycle), but a REGISTERED sample_valid only caught up one cycle later — by which point sample_addr had already advanced to 1, so the true addr=0 sample was never flagged valid, and (in combination with a second timing gap at subchirp boundaries) each subchirp streamed 40 valid cycles instead of 38. Confirmed via the unit testbench's address-walk check, which failed on every sample, always off by exactly one address.

Fixed: sample_valid is now purely combinational (assign sample_valid = (state == S_STREAM));, so it is always aligned with whatever sample_addr/subchirp_sel currently hold, by construction — no lag is possible.

2.2.25.6 Defect 3 — off-by-one in the gap counter

Loading gap_counter with the raw gap length (e.g. 10) and counting down to 0 inclusive spends 11 cycles (10, 9, ..., 1, 0), not 10 — a classic fencepost error. Confirmed via the testbench: measured exactly one extra cycle per gap (82 vs. 80 expected over 2 gaps; 1968 vs. 1920 over 48).

Fixed: the counter now loads (gap_length - 1), so counting down to 0 takes exactly gap_length cycles.

2.2.25.7 Other design decisions

- Gap parity: the gap following the first chirp sequence in a packet uses gap_even, the second uses gap_odd, alternating thereafter. This assignment is what makes the length arithmetic of Defect 1 balance exactly for an even sequence count of 48, which is independent supporting evidence that the assignment is correct. Direct confirmation against the MATLAB TxchirpSequences reference is performed at system level.

- num_symbols is assumed to always be an exact multiple of 4 (the upstream padding/coding-rate math guarantees this for any well-formed packet). This is not re-checked in hardware.
- num_symbols == 0 (Task 4's payload-length-0 edge case): the FSM skips straight from S_IDLE to S_DONE, requesting nothing, streaming nothing, and inserting no gap (there is no sequence to trail).
- During a gap, subchirp_sel/sample_addr hold their last streamed values (don't-care downstream, since sample_valid is low) — the multiplier must gate its own output on valid_in, not trust the ROM address during gap cycles.
- last_seq is a latched flag, set the moment a sequence's trailing gap is entered, recording whether that sequence was the final one in the packet — this is what lets S_GAP decide between S_DONE and S_REQ_SYM without re-deriving the comparison a second time.
- sign_real_out / sign_imag_out remain registered (set inside the S_STREAM case body), which means they update to reflect a newly-issued address ONE cycle after sample_valid/sample_addr do. This is an intentional interface characteristic — not a bug — that the top-level wrapper relies on to naturally line up with chirp_rom's own one-cycle read latency (Section 6.2).
- start, dqpsk_valid are expected to be single-cycle pulses; chirp_index and num_symbols are latched only at the start pulse and are not affected by the inputs changing mid-packet.
- chirp_index mapping: hardware value 0-3 corresponds to the standard's $m = 1-4$ ($\text{chirp_index_reg} + 1 = m$). Kept consistent with chirp_rom.

2.2.25.8 Interface

Signal	Direction	Width	Description
clk, reset	IN	1 each	System clock; synchronous, active-high reset
start	IN	1	Begin generating for this packet (single-cycle pulse)
chirp_index	IN	2	Selected chirp sequence (m), latched at start
num_symbols	IN	NUM_SYMBOLS_WIDTH	Total DQPSK symbols in this packet, latched at start
dqpsk_valid	IN	1	A new DQPSK symbol is available
dqpsk_sign_real/imag	IN	1 each	DQPSK symbol sign bits (0 = +1, 1 = -1)
subchirp_sel	OUT	2	Drives chirp_rom address (k)
sample_addr	OUT	6	Drives chirp_rom address (sample index, 0-37 valid)
sign_real_out/imag_out	OUT	1 each	Latched sign, registered — see note in Section 4.4
sample_valid	OUT	1	Combinational — high exactly while state == S_STREAM
next_symbol_req	OUT	1	Pulses to request the next DQPSK symbol
done	OUT	1	Packet modulation complete (one-cycle pulse)

2.2.25.9 RTL — csk_generator.v

```
// =====
module csk_generator #(
    parameter NUM_SYMBOLS_WIDTH = 12 // covers up to 4095 DQPSK symbols;
                                     // largest real packet (127-byte
                                     // payload, 250 kb/s) needs ~2880
) (
    input wire clk,
    input wire reset, // synchronous, active-high

    input wire start, // begin generating for this packet
    input wire [1:0] chirp_index,
    input wire [NUM_SYMBOLS_WIDTH-1:0] num_symbols, // total DQPSK symbols in this packet, latched
    at start

    input wire dqpsk_valid, // new DQPSK symbol available
    input wire dqpsk_sign_real, // 0 = +1, 1 = -1
    input wire dqpsk_sign_imag, // 0 = +1, 1 = -1

    output reg [1:0] subchirp_sel,
    output reg [5:0] sample_addr,
    output reg sign_real_out, // latched sign, passthrough to multiplier
    output reg sign_imag_out,
    output wire sample_valid, // high while a real chirp sample is being
    addressed
    output reg next_symbol_req, // pulses to request the next DQPSK symbol
    output reg done, // packet modulation complete (1-cycle pulse)
    output wire gap_active
);

reg [6:0] gap_even; // max value 70 -> needs 7 bits
reg [6:0] gap_odd;

reg [1:0] chirp_index_reg;

always @(*) begin
    case (chirp_index_reg)
        2'd0: begin gap_even = 7'd10; gap_odd = 7'd70; end // m=1
        2'd1: begin gap_even = 7'd20; gap_odd = 7'd60; end // m=2
        2'd2: begin gap_even = 7'd30; gap_odd = 7'd50; end // m=3
        2'd3: begin gap_even = 7'd40; gap_odd = 7'd40; end // m=4
    endcase
end

// FSM state encoding
localparam S_IDLE = 3'd0;
localparam S_REQ_SYM = 3'd1; // assert next_symbol_req for one cycle
localparam S_WAIT_SYM = 3'd2; // wait for dqpsk_valid, then latch signs
localparam S_STREAM = 3'd3; // walk sample_addr 0..37 for current subchirp
localparam S_GAP = 3'd4; // count down the inter-sequence gap (always runs, even after the last
sequence)
localparam S_DONE = 3'd5; // one-cycle done pulse

reg [2:0] state;

assign sample_valid = (state == S_STREAM);

assign gap_active = (state == S_GAP); // Raghad fix 3

// Internal counters / registers

reg [NUM_SYMBOLS_WIDTH-1:0] symbol_count; // DQPSK symbols consumed so far
reg [NUM_SYMBOLS_WIDTH-1:0] num_symbols_reg;
reg seq_parity; // toggles per completed chirp sequence (0=this gap even,
1=this gap odd)
reg [6:0] gap_counter;
```

```

    reg                                last_seq;        // latched: the gap currently running follows the FINAL
sequence

    reg sign_real_latched;
    reg sign_imag_latched;

    always @(posedge clk) begin
        if (reset) begin
            state                <= S_IDLE;
            subchirp_sel         <= 2'd0;
            sample_addr          <= 6'd0;
            sign_real_out        <= 1'b0;
            sign_imag_out        <= 1'b0;
            next_symbol_req      <= 1'b0;
            done                 <= 1'b0;
            symbol_count         <= {NUM_SYMBOLS_WIDTH{1'b0}};
            num_symbols_reg      <= {NUM_SYMBOLS_WIDTH{1'b0}};
            chirp_index_reg      <= 2'd0;
            seq_parity           <= 1'b0;
            gap_counter          <= 7'd0;
            last_seq             <= 1'b0;
            sign_real_latched    <= 1'b0;
            sign_imag_latched    <= 1'b0;
        end else begin
            next_symbol_req <= 1'b0;
            done            <= 1'b0;

            case (state)

                S_IDLE: begin
                    if (start) begin
                        chirp_index_reg <= chirp_index;
                        num_symbols_reg <= num_symbols;
                        symbol_count    <= {NUM_SYMBOLS_WIDTH{1'b0}};
                        subchirp_sel    <= 2'd0;
                        seq_parity      <= 1'b0;

                        if (num_symbols == {NUM_SYMBOLS_WIDTH{1'b0}}) begin
                            // edge case: empty packet, nothing to stream, no gap
                            state <= S_DONE;
                        end else begin
                            state <= S_REQ_SYM;
                        end
                    end
                end

                S_REQ_SYM: begin
                    next_symbol_req <= 1'b1;    // one-cycle pulse
                    state           <= S_WAIT_SYM;
                end

                S_WAIT_SYM: begin
                    if (dqpsk_valid) begin
                        sign_real_latched <= dqpsk_sign_real;
                        sign_imag_latched <= dqpsk_sign_imag;
                        sample_addr       <= 6'd0;
                        state             <= S_STREAM;
                    end
                    // else: keep waiting, next_symbol_req already fell
                    // back to 0 by default above -- encoder is
                    // expected to hold the request internally if it needs
                    // more than one cycle to respond
                end

                S_STREAM: begin
                    sign_real_out <= sign_real_latched;
                    sign_imag_out <= sign_imag_latched;
                    // subchirp_sel already holds the current k

```

```

        if (sample_addr == 6'd37) begin
            // last sample of this subchirp being output now
            symbol_count <= symbol_count + 1'b1;

            if (subchirp_sel == 2'd3) begin
                // last subchirp of this chirp sequence done --
                // a gap ALWAYS follows, whether or not more
                // sequences remain
                subchirp_sel <= 2'd0;
                // Load (gap_length - 1): counting inclusively
                // from N down to 0 spends N+1 cycles, not N --
                // loading N-1 makes the countdown take exactly
                // N cycles
                gap_counter <= (seq_parity ? gap_odd : gap_even) - 1'b1;
                seq_parity <= ~seq_parity;
                last_seq <= ((symbol_count + 1'b1) == num_symbols_reg);
                state <= S_GAP;
            end else begin
                // move to next subchirp within the same sequence
                subchirp_sel <= subchirp_sel + 1'b1;
                state <= S_REQ_SYM;
            end

            end else begin
                sample_addr <= sample_addr + 1'b1;
            end
        end

    S_GAP: begin
        if (gap_counter == 7'd0) begin
            // this gap has just finished counting down
            if (last_seq)
                state <= S_DONE; // that was the trailing gap -- packet complete
            else
                state <= S_REQ_SYM; // start the next chirp sequence
        end else begin
            gap_counter <= gap_counter - 1'b1;
        end
    end

    S_DONE: begin
        done <= 1'b1; // one-cycle pulse
        state <= S_IDLE;
    end

    default: state <= S_IDLE;

endcase
end
end
endmodule

```

2.2.26 complex_multiplier:

2.2.26.1 Derivation

Combines one chirp_rom sample ($\text{rom_real} + j*\text{rom_imag}$) with one DQPSK symbol, represented purely as two sign bits, to produce the final modulated transmit sample:

$$(a + bj)(c + dj) = (ac - bd) + (ad + bc)j$$

where $a = \text{dqpsk real sign } (+1/-1)$, $b = \text{dqpsk imag sign } (+1/-1)$, $c = \text{rom_real}$, $d = \text{rom_imag}$. Because a and b are always exactly $+1$ or -1 , ac , bd , ad and bc are each just a sign flip of c or d — not a true

multiply. The module is therefore four conditional negations and two adders, with zero multiplier hardware.

Output word length is 6 bits, derived in Section 2.8 from the worst-case ROM magnitude (+/-15) — not assumed.

2.2.26.2 Timing

Registered, one cycle of latency from (rom_real, rom_imag, sign_real, sign_imag, valid_in) to (tx_real, tx_imag, valid_out). This is a deliberate pipeline stage — see Section 6 for why it lines up correctly with chirp_rom's own one-cycle read latency.

2.2.26.3 Interface

Signal	Direction	Width	Description
clk, reset	IN	1 each	System clock; synchronous, active-high reset
rom_real, rom_imag	IN	ROM_WIDTH (signed)	c, d — from chirp_rom
sign_real, sign_imag	IN	1 each	a, b — DQPSK sign bits (0 = +1, 1 = -1)
valid_in	IN	1	rom_real/rom_imag/signs are aligned and valid this cycle
tx_real, tx_imag	OUT	OUT_WIDTH (signed)	(ac - bd), (ad + bc) — registered
valid_out	OUT	1	valid_in, registered by one cycle

2.2.26.4 RTL — complex_multiplier.v

```
//      (a + bj)(c + dj) = (ac - bd) + (ad + bc)j
//
//  where a = dqpsk real sign (+1/-1), b = dqpsk imag sign (+1/-1),
//          c = rom_real,                d = rom_imag.
module complex_multiplier #(
    parameter ROM_WIDTH = 5,
    parameter OUT_WIDTH = 6    // ROM_WIDTH + 1
) (
    input wire          clk,
    input wire          reset,
    input wire signed [ROM_WIDTH-1:0] rom_real,    // c, from chirp_rom
    input wire signed [ROM_WIDTH-1:0] rom_imag,    // d, from chirp_rom
    input wire          sign_real,    // a: 0 = +1, 1 = -1 (DQPSK real sign)
    input wire          sign_imag,    // b: 0 = +1, 1 = -1 (DQPSK imag sign)
    input wire          valid_in,    // rom_real/rom_imag/signs are aligned and valid this
cycle
    output reg signed [OUT_WIDTH-1:0] tx_real,    // (ac - bd)
    output reg signed [OUT_WIDTH-1:0] tx_imag,    // (ad + bc)
    output reg          valid_out
);
    // Sign-extend the ROM samples to the output width BEFORE the
    // conditional negate, so the negation itself can never overflow
    // (negating the most-negative ROM_WIDTH-bit value would overflow at
    // ROM_WIDTH bits, but not once extended to OUT_WIDTH bits).
    wire signed [OUT_WIDTH-1:0] c_ext = {{(OUT_WIDTH-ROM_WIDTH){rom_real[ROM_WIDTH-1]}}, rom_real};
    wire signed [OUT_WIDTH-1:0] d_ext = {{(OUT_WIDTH-ROM_WIDTH){rom_imag[ROM_WIDTH-1]}}, rom_imag};
    // Four conditional negations, no actual multiplier hardware.
    wire signed [OUT_WIDTH-1:0] ac = sign_real ? -c_ext : c_ext;
    wire signed [OUT_WIDTH-1:0] bd = sign_imag ? -d_ext : d_ext;
    wire signed [OUT_WIDTH-1:0] ad = sign_real ? -d_ext : d_ext;
    wire signed [OUT_WIDTH-1:0] bc = sign_imag ? -c_ext : c_ext;
    always @(posedge clk) begin
        if (reset) begin
            tx_real  <= {OUT_WIDTH{1'b0}};
            tx_imag  <= {OUT_WIDTH{1'b0}};
            valid_out <= 1'b0;
        end
    end
endmodule
```

```

end else begin
    tx_real  <= ac - bd;
    tx_imag  <= ad + bc;
    valid_out <= valid_in;
end
end
endmodule

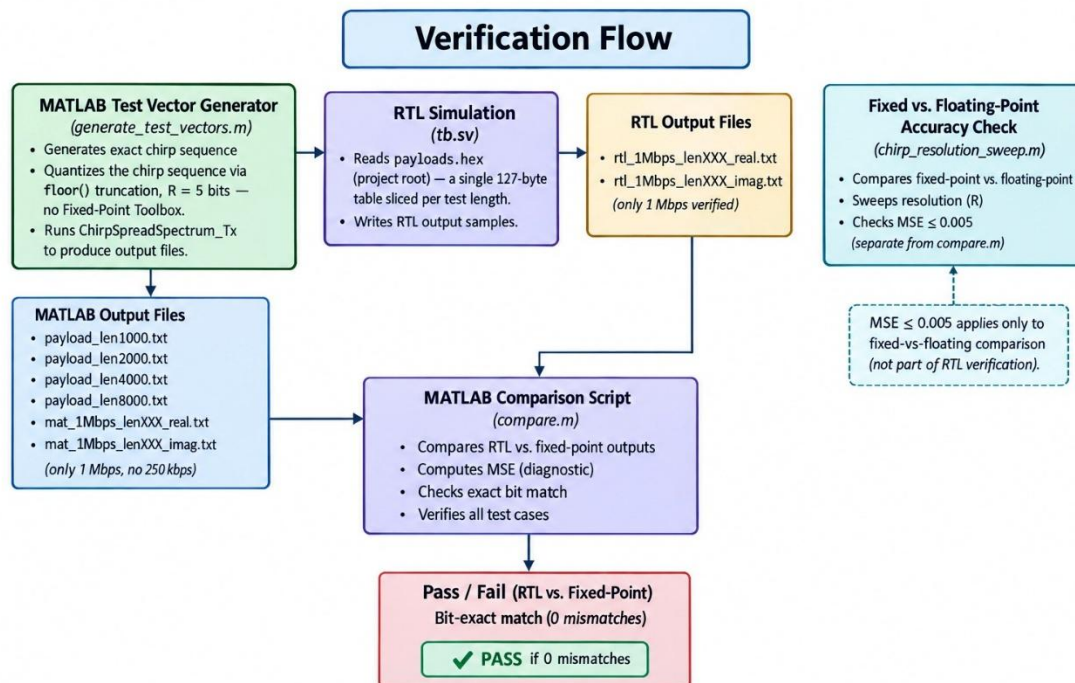
```

3 VERIFICATION METHODOLOGY

Verification in this project happens at two independent levels:

- Model-level accuracy → does quantizing the algorithm to fixed-point (in particular, the chirp sequence generator, the only floating-point block in the design) introduce acceptable error relative to the exact algorithm? This is judged by Mean Square Error (MSE) against the 0.005 threshold from the standard.
- Implementation correctness → Does the Verilog RTL correctly implement the fixed-point model it was specified against? Because the fixed-point MATLAB model is the literal numeric specification for the hardware, this comparison is judged by bit-exact equality, not by an MSE tolerance.

Stage	Compared against	Criterion	Tooling
Fixed-point model	Floating-point model	$MSE \leq 0.005$	MATLAB
RTL (Verilog)	Fixed-point model	Bit-exact (0 mismatches)	Verilog simulator + MATLAB compare script



3.1 Integration Testbench

The full-chain testbench (tb.sv) drives a payload into the complete css_tx_top transmitter, runs it to completion, and compares the captured Tx_real/Tx_imag stream against the fixed-point MATLAB model's output for that same payload.

3.1.1 Test Vectors

Test vectors are generated by generate_test_vectors.m, which runs the fixed-point MATLAB transmitter chain (ChirpSpreadSpectrum_Tx.m) across four payload lengths chosen to exercise edge cases and typical operation

- L = 0 bytes (empty payload)
- L = 1 byte (the smallest non-empty payload)
- L = 50 bytes (a mid-sized payload with varied bit patterns)
- L = 127 bytes (the largest PSDU length the standard allows)

Payload bytes follow a simple rule (byte $i = i \bmod 256$), so a single 127-byte payloads.hex file, loaded once by the testbench, can supply all four test lengths just by using the first L bytes of it.

```
=====
Generating Reference Vectors: 1Mbps
=====
[OK] L = 0 | Total Samples: 2880
[OK] L = 1 | Total Samples: 3072
[OK] L = 50 | Total Samples: 15552
[OK] L = 127 | Total Samples: 35328
=====
Generating Reference Vectors: 250kbps
=====
[OK] L = 0 | Total Samples: 7680
[OK] L = 1 | Total Samples: 7680
[OK] L = 50 | Total Samples: 59904
[OK] L = 127 | Total Samples: 136704

All test vectors generated successfully in "test_vectors/".
>>
```

3.2 Chirp Sequence Fixed-Point Resolution

As noted above, the chirp sequence generator is the only place in the design where floating-point behavior shows up. In generate_test_vectors.m, its samples are quantized at 5 bits ($R = 5$) (the same resolution used by the RTL's chirp ROM [chirp_rom.v, ROM_WIDTH = 5]) using floor() truncation, matching the rounding mode used everywhere else in the design:

```
chirpRomBitNumber = 5;
chirpSeq_Tx = floor(chirpSeq * (2^(chirpRomBitNumber - 1) - 1));
```

3.3 Simulation and Capture

For each rate/length combination, the testbench:

- Loads the payload bytes into the DUT's payload RAM through payload_waddr / payload_wdata.
- Sets payload_length and chirp_index, then pulses start_tx for one clock cycle.

- Captures Tx_real and Tx_imag on every cycle where tx_valid is high, until tx_done pulses.
- Compares each captured sample against the matching MATLAB golden sample as it comes in, and if something doesn't match, reports the first divergence with enough context to locate it — which chirp sequence, which subchirp, which DQPSK symbol, and whether it falls inside the preamble/SFD or the payload.
- Fails the run if fifo_overflow ever goes high, or if the simulation times out before tx_done.

```
# =====
# CSS PHY BASIC RTL TEST SUMMARY - 1Mbps
# Completed tests : 4
# Failed tests    : 0
#
# =====
# ** Note: $stop    : C:/Users/Lenovo/Downloads/Verifications/tb.sv(485)
#    Time: 617270 ns Iteration: 1 Instance: /tb
# Break in Module tb at C:/Users/Lenovo/Downloads/Verifications/tb.sv line 485
VSIM 39>
```

3.4 Comparison Script

compare.m does the same comparison a second time, independently, on the MATLAB side, it reads the RTL's captured output files and reports per-sample mismatch counts for the real and imaginary channels, either for one rate/length at a time or all of them together.

```
>> compare('1Mbps', 0);

=====
Sample Length Check (1Mbps, L = 0)
=====

RTL Sample Count    : 2850
MATLAB Sample Count : 2880
[INFO] 30 trailing MATLAB samples are zero (preallocation pad) - OK

--- Comparison Results (1Mbps, L = 0) ---
Real Channel  - Mismatches: 0 / 2880
Imag Channel  - Mismatches: 0 / 2880
[VERDICT: PASS] RTL output is BIT-EXACT with MATLAB stage 13!
```

```
>> compare('1Mbps', 1);

=====
Sample Length Check (1Mbps, L = 1)
=====

RTL Sample Count    : 3072
MATLAB Sample Count : 3072

--- Comparison Results (1Mbps, L = 1) ---
Real Channel  - Mismatches: 0 / 3072
Imag Channel  - Mismatches: 0 / 3072
[VERDICT: PASS] RTL output is BIT-EXACT with MATLAB stage 13!
>>
```

```
>> compare('1Mbps', 50);

=====
Sample Length Check (1Mbps, L = 50)
=====

RTL Sample Count    : 15522
MATLAB Sample Count : 15552
[INFO] 30 trailing MATLAB samples are zero (preallocation pad) - OK

--- Comparison Results (1Mbps, L = 50) ---
Real Channel  - Mismatches: 0 / 15552
Imag Channel  - Mismatches: 0 / 15552
[VERDICT: PASS] RTL output is BIT-EXACT with MATLAB stage 13!
>>
```

```
>> compare('1Mbps', 127);

=====
Sample Length Check (1Mbps, L = 127)
=====

RTL Sample Count    : 35328
MATLAB Sample Count : 35328

--- Comparison Results (1Mbps, L = 127) ---
Real Channel  - Mismatches: 0 / 35328
Imag Channel  - Mismatches: 0 / 35328
[VERDICT: PASS] RTL output is BIT-EXACT with MATLAB stage 13!
>>
```

For payloads where the number of chirp sequences works out odd, MATLAB's chirpModulation.m pre-allocates its output array assuming a fixed 192-sample slot per sequence, but actually advances through the array using the real, alternating gap length (10 or 70 samples for chirp index $m = 1$). With an odd number of sequences, that leaves a handful of array slots at the very end (30, in these test cases) that never get written and stay zero from the initial allocation — they don't correspond to anything actually transmitted. This was checked directly: every one of those trailing samples is confirmed zero in the MATLAB output, so the RTL's shorter (and physically meaningful) sample count is what's used for the comparison.

4 ACCURACY REPORT: FIXED-POINT VS. FLOATING-POINT

This section shows that the fixed-point model stays inside the standard's 0.005 MSE bound relative to the ideal floating-point algorithm — which, since the RTL matches the fixed-point model exactly, is also the RTL's accuracy relative to floating point (more on that in Section 6.3).

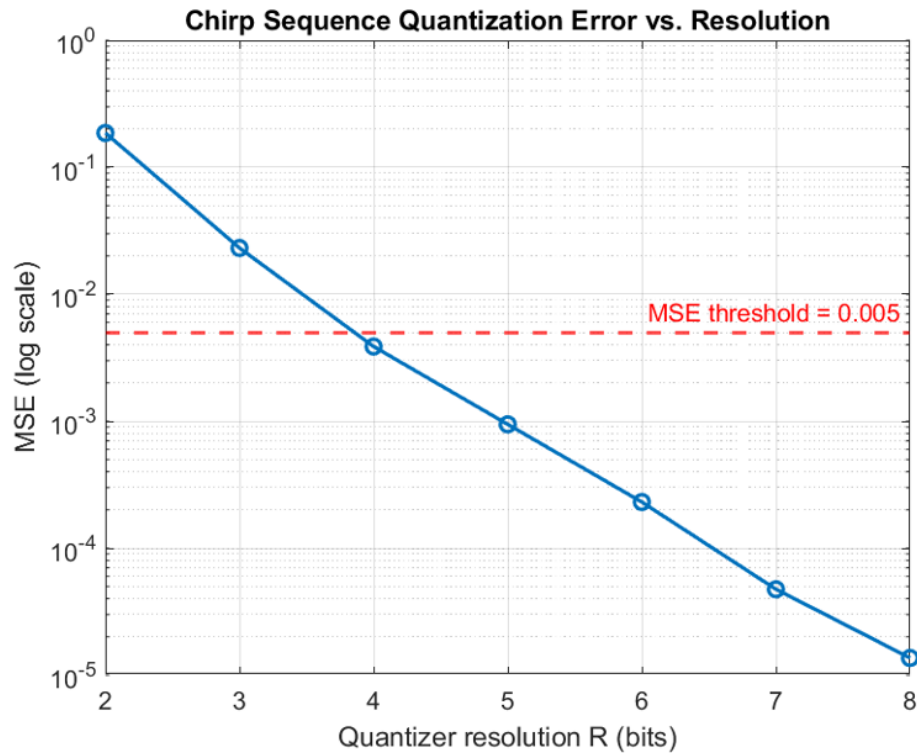
4.1 Chirp Quantizer Resolution Sweep

The quantizer resolution R was swept from 2 to 8 bits. For each value of R , the chirp sequence was quantized and compared against the exact floating-point chirp using the standard's own MSE formula:

$$MSE(R) = \sum |exact - fixed(R)|^2 / \sum |exact|^2 \quad (N = 152 \text{ samples, one full chirp sequence})$$

R (bits)	MSE	Pass (≤ 0.005)?
2	0.186578	No
3	0.023155	No
4	0.003876	Yes
5	0.000945	Yes
6	0.000231	Yes
7	0.000047	Yes
8	0.000014	Yes

$R = 4$ bits is the minimum that clears the 0.005 bound. This project uses $R = 5$ for the chirp ROM (chirp_rom.v, ROM_WIDTH = 5), which gives some extra margin (MSE ≈ 0.000945 , about 5 $\times$ below the threshold) for a small increase in ROM size and word length compared to $R = 4$.



4.2 Full-Chain Fixed-Point vs. Floating-Point MSE

As a second check, this time on the actual transmitted waveform rather than a single chirp sequence, the full fixed-point chain was run against the full floating-point chain for every test case, with MSE computed over the entire output:

Rate	Payload length (bytes)	Samples	Normalized MSE	Result
1 Mb/s	0	2880	0.000891	PASS
1 Mb/s	1	3072	0.000891	PASS
1 Mb/s	50	15552	0.000891	PASS
1 Mb/s	127	35328	0.000891	PASS

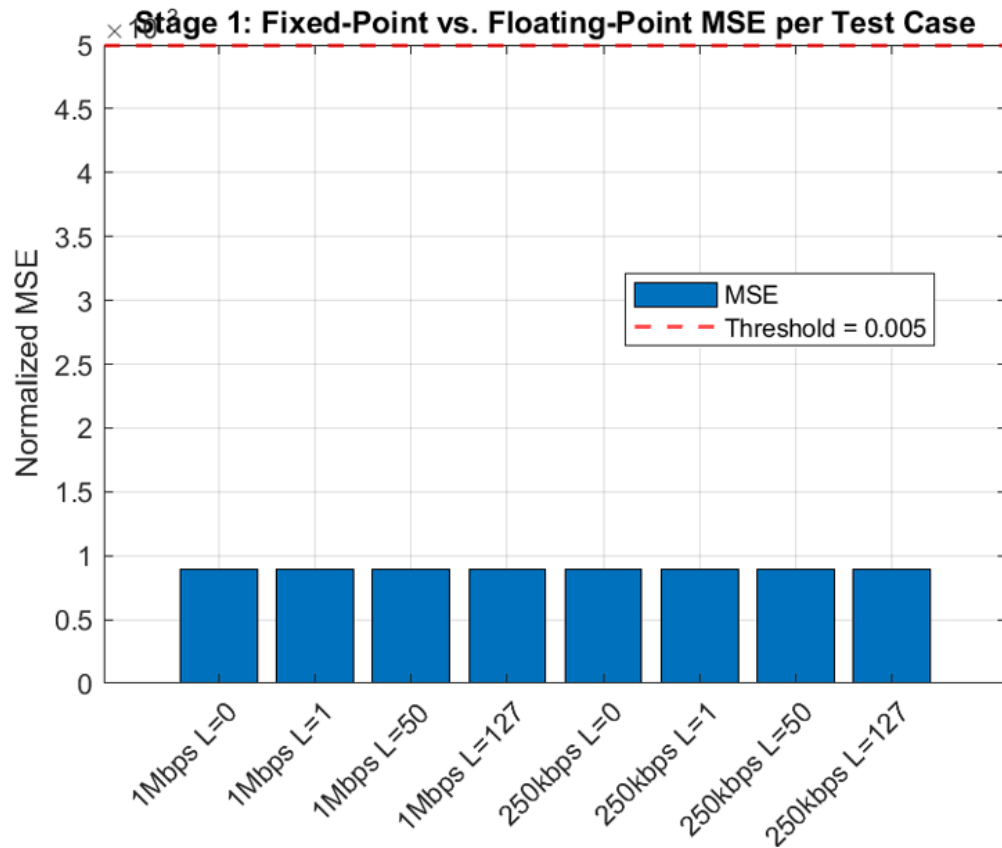
The MSE comes out identical across all four lengths, which makes sense rather than looking suspicious: the chirp generator is the only source of error, and the same four chirp waveforms repeat throughout the packet no matter how long the payload is, so the error and the signal energy both grow at the same rate as the packet gets longer. That the number holds steady across four very different payload lengths is itself a small sanity check that the measurement is being done consistently.

=====

STAGE 1 VERIFICATION: Floating-Point vs. Fixed-Point Model (MSE <= 0.005)

=====

Rate: 1Mbps	L = 0	Samples: 2880	Normalized MSE: 0.000891	[PASS]
Rate: 1Mbps	L = 1	Samples: 3072	Normalized MSE: 0.000891	[PASS]
Rate: 1Mbps	L = 50	Samples: 15552	Normalized MSE: 0.000891	[PASS]
Rate: 1Mbps	L = 127	Samples: 35328	Normalized MSE: 0.000891	[PASS]
Rate: 250kbps	L = 0	Samples: 7680	Normalized MSE: 0.000891	[PASS]
Rate: 250kbps	L = 1	Samples: 7680	Normalized MSE: 0.000891	[PASS]
Rate: 250kbps	L = 50	Samples: 59904	Normalized MSE: 0.000891	[PASS]
Rate: 250kbps	L = 127	Samples: 136704	Normalized MSE: 0.000891	[PASS]

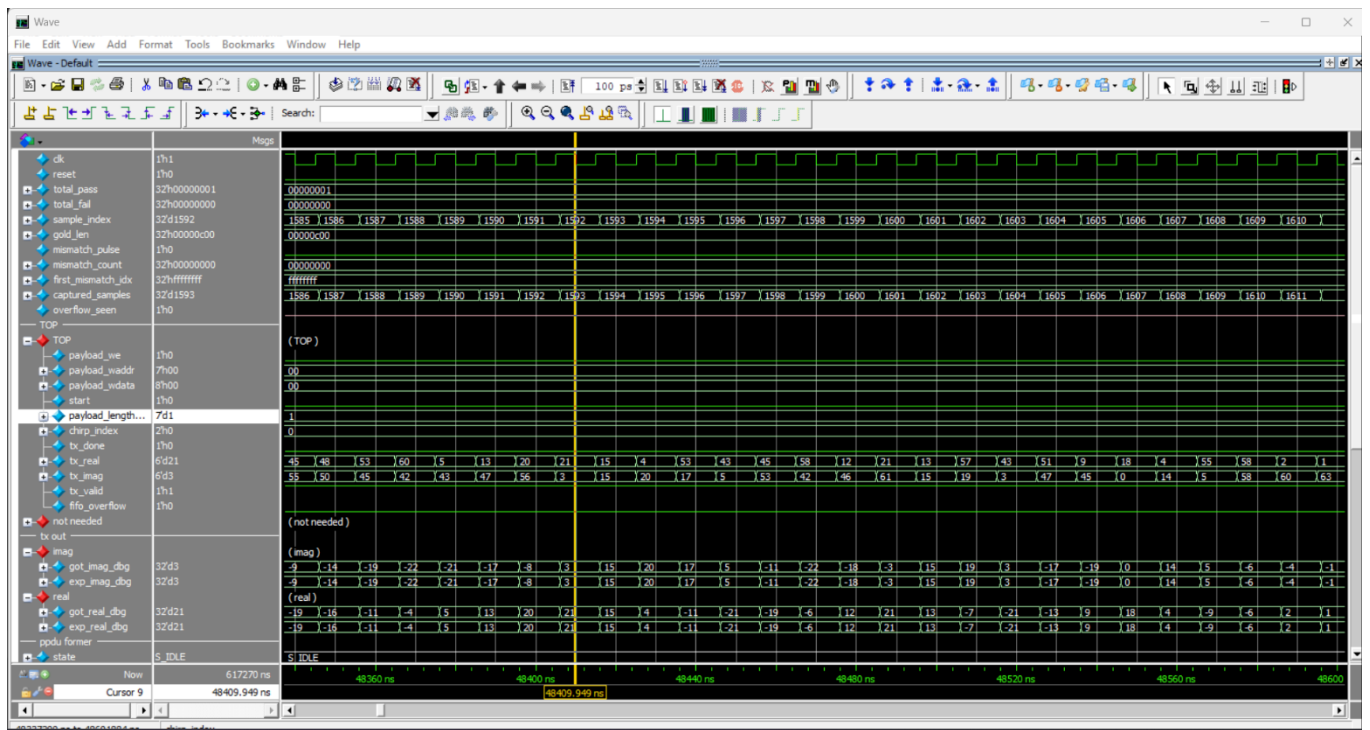
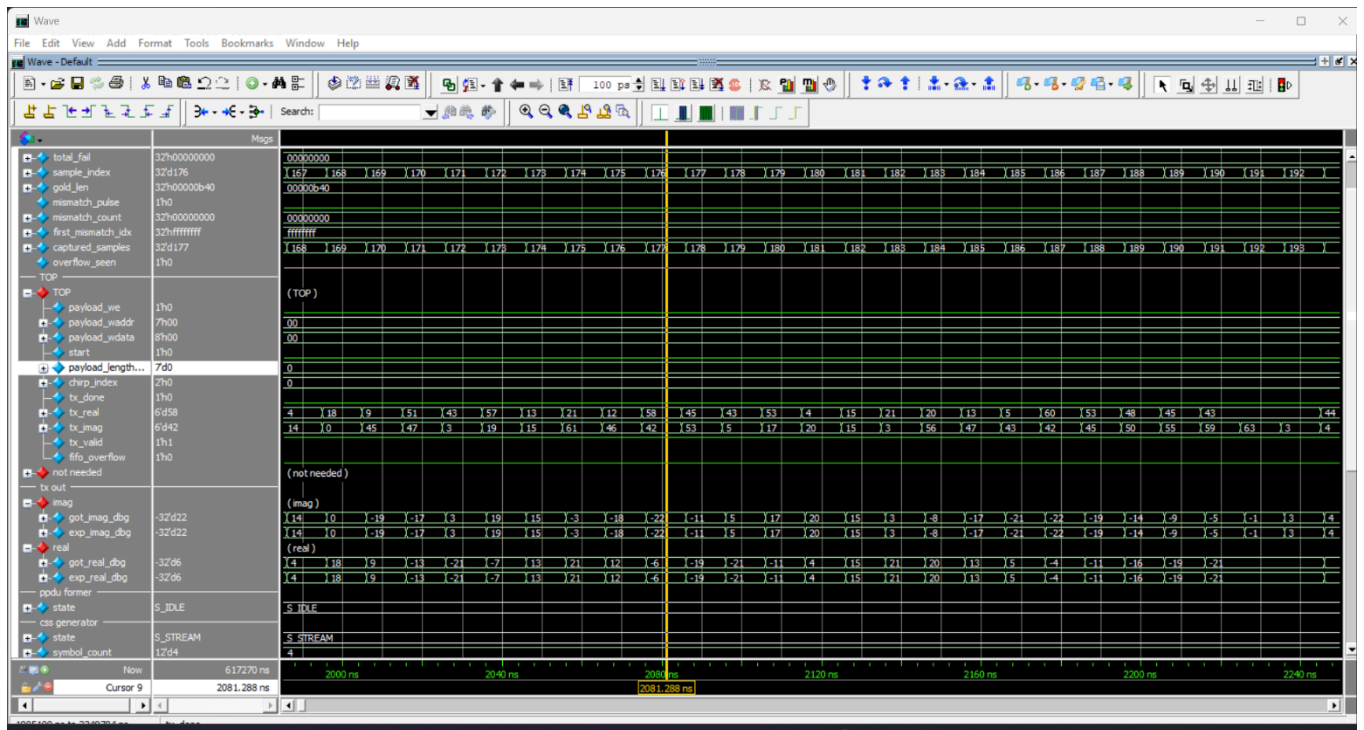


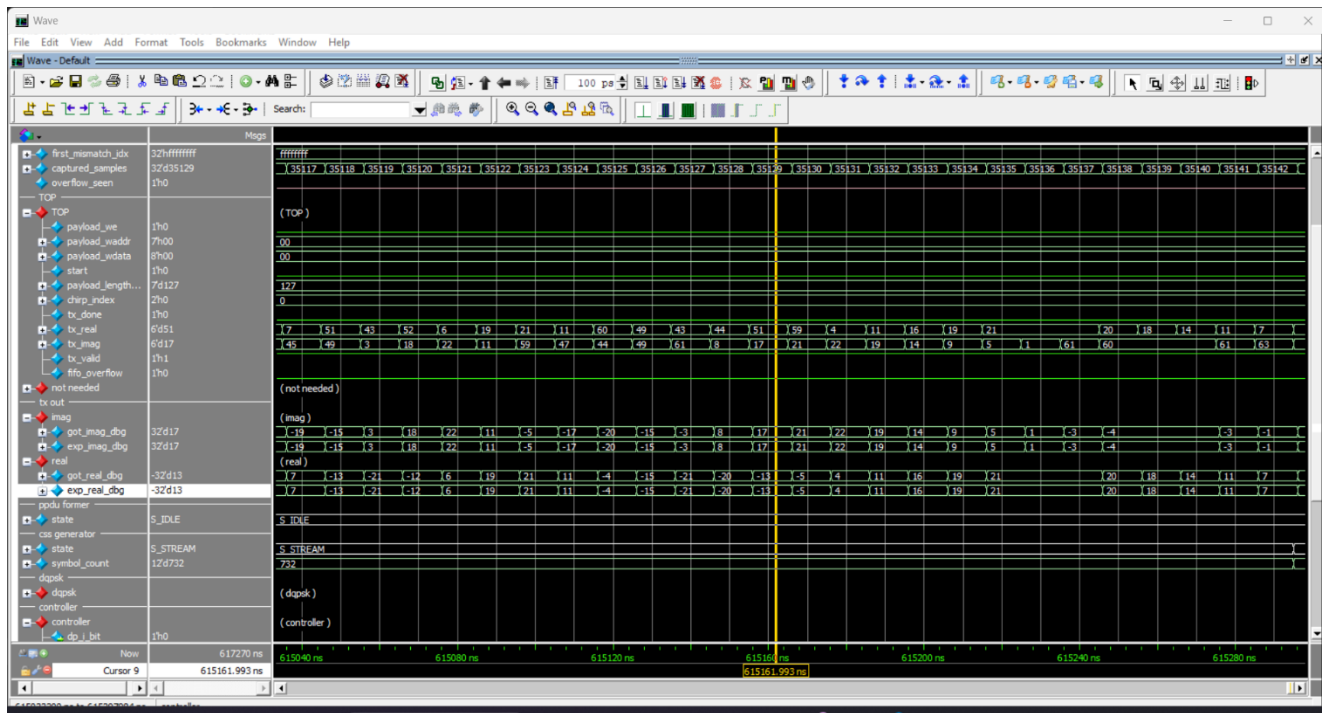
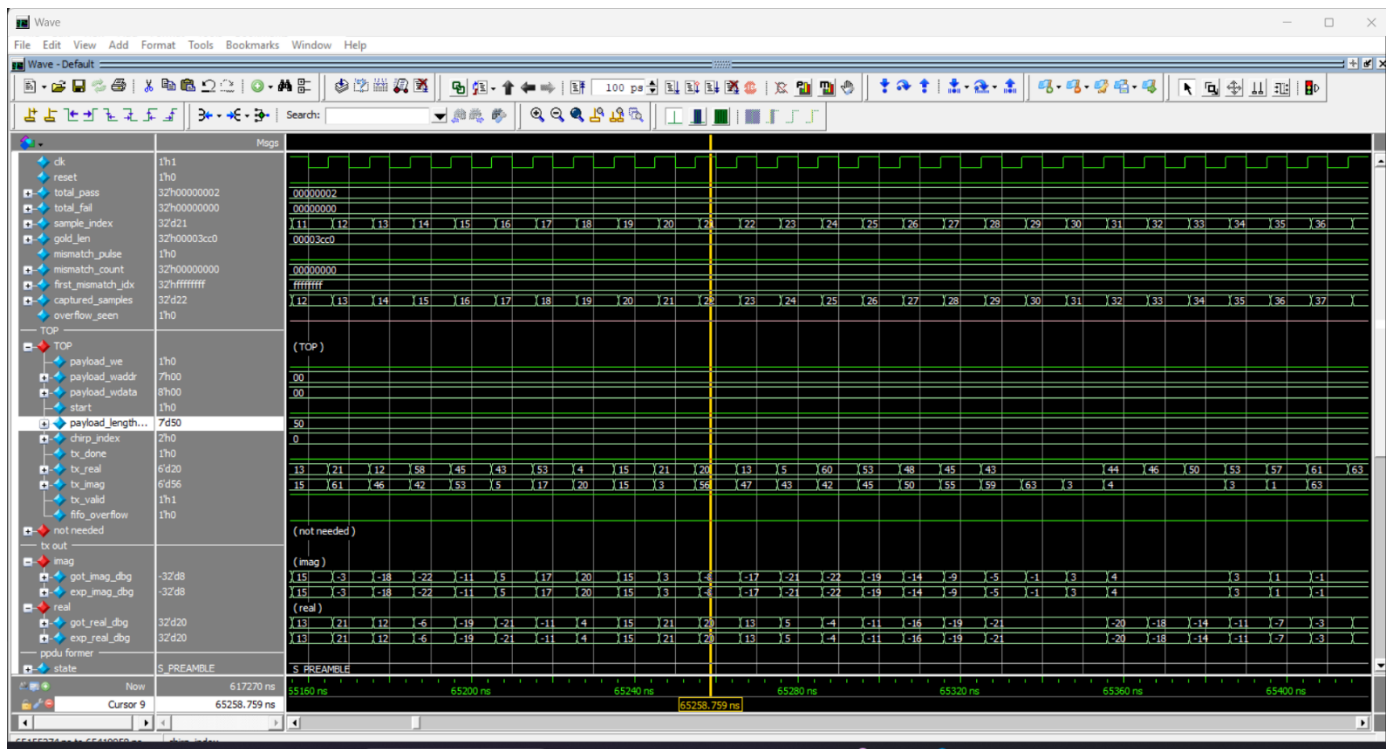
5 DEBUGGING LOG: ISSUES FOUND AND FIXED

A testbench only means something if it's actually shown to catch real bugs. The table below lists what came up during this process, what caused it, and how it was fixed — each one confirmed resolved by re-running the full integration test.

#	Symptom	Root cause	Fix
1	num_symbols didn't scale with payload length, the RTL kept processing the same 56-symbol packet no matter what length was requested.	controller.v was computing num_symbols from the registered payload_length_reg, which hadn't updated yet on the same cycle that dp_start (driven straight from start_tx) sampled it. A one-cycle race.	Recomputed num_symbols directly from the raw payload_length input instead, so it's valid on the same cycle as start_tx / dp_start.
2	RTL sample count came up short of the MATLAB golden count, by a fixed amount, for certain payload lengths	zero_padding.v special-cased the zero-remainder case and added no padding bits there, while MATLAB's formula (paddingBy = GROUP_SIZE - mod(bits, GROUP_SIZE)) always adds a full group of padding, even when the remainder is already zero.	Removed the special case: pad_bits = GROUP_SIZE - remainder, unconditionally — matching MATLAB exactly.
3	RTL sample count was short by exactly (gap length × number of sequences); tx_valid never went high during the gaps between sequences.	The RTL only asserted tx_valid while actively streaming chirp ROM samples, but MATLAB's output array reserves and zero-fills sample slots for the gap too.	Added a gap_active signal out of csk_generator, pipelined to line up with the ROM/multiplier latency, and used it to also assert tx_valid (with tx_real = tx_imag = 0) for the length of each gap.
4	One DQPSK symbol's sign bits came out rotated by a fixed 90°/180° relative to MATLAB for certain payload byte values, and the error then carried through every symbol after it (a signature of the differential $S(n) = X(n) \cdot S(n-4)$ relation — one wrong symbol pollutes everything downstream).	Two real suspects were checked and ruled out first: the order chips get shifted out in (parallel_to_serial.v) and the order incoming bits get grouped into an address in (serial_to_parallel.v) were both re-derived from scratch against the MATLAB side (the symbol mapper's ROM bit convention, and binary2decimal.m's bit weighting) and confirmed correct. The Walsh-Hadamard codeword table itself (symbol_mapper.v) was also rebuilt from globalSettings.m's hadamard(4) construction and matched all 8 entries.	The actual cause turned out to be in zero_padding.v: the payload RAM's read address was being advanced on word_idx == 7 (all three bits of word_idx set) instead of word_idx == 6 — off by one in when the next payload byte gets fetched relative to bit serialization. Changing the condition to word_idx == 3'd6 fixed every remaining mismatch.

6 BEHAVIORAL (RTL) SIMULATION EVIDENCE





7 FUTURE IMPROVEMENTS

- **Complete 250 kb/s Support:** Fix the controller/interleaver & any other design bug, then repeat the same unit and integration verification used for 1 Mb/s.
- **Broaden Test Coverage:** Test all payload lengths from 0 to 127 bytes and try to reach highest coverage percentage.
- **Add Assertions and Functional Coverage:** Add SystemVerilog assertions for key design conditions, such as preventing fifo_overflow and ensuring valid symbol counts, together with functional coverage for FSM states and boundary cases.

8 SYNTHESIS AND IMPLEMENTATION

8.1 ASIC FLOW:

Design (Top)	css_tx_top
Process / Library	SAED 90nm (saed90nm_max, worst corner, 125 °C)
Clock	32 MHz, 31.25 ns period
Flow Tools	Design Compiler (synthesis) → IC Compiler II (dlib, floorplan, powerplan, CTS, routing)

This report tracks `css_tx_top` through the ASIC implementation flow: logic synthesis (Stage 1), design library / data-prep setup (Stage 2), floorplanning (Stage 3), powerplanning (Stage 4), CTS (Stage 5) and routing (Stage 6). Each section gives the key results and any issues carried forward to the next stage.

8.1.1 Stage 1 — Logic Synthesis

Synthesized in Design Compiler against `saed90nm_max` (worst corner) at 32 MHz. Result: clean synthesis with zero hold violations and a minor setup shortfall confined to two RAM-like structures.

Metric	Value
Total cell area	102,720.61
Total cells	8,367 (1,847 sequential)
Total dynamic power / leakage	39.32 μ W / 353.09 μ W
Setup WNS / TNS / endpoints	-0.38 ns / -26.72 ns / 201
Hold violations	0
Max-transition violations	13 of 8,572 nets

All setup violations trace to u_payload_ram and u_controller/chip_mem — both synthesized as flip-flop arrays (no SRAM macros bound yet) and both driven by the statistical, pre-layout wire-load model, which is known to over-estimate capacitance on wide memory-style muxes. Worst path slack (-0.38 ns) is <1.2% of the clock period. No RTL changes recommended; cleared to proceed.

8.1.2 Stage 2 — Design Library Setup

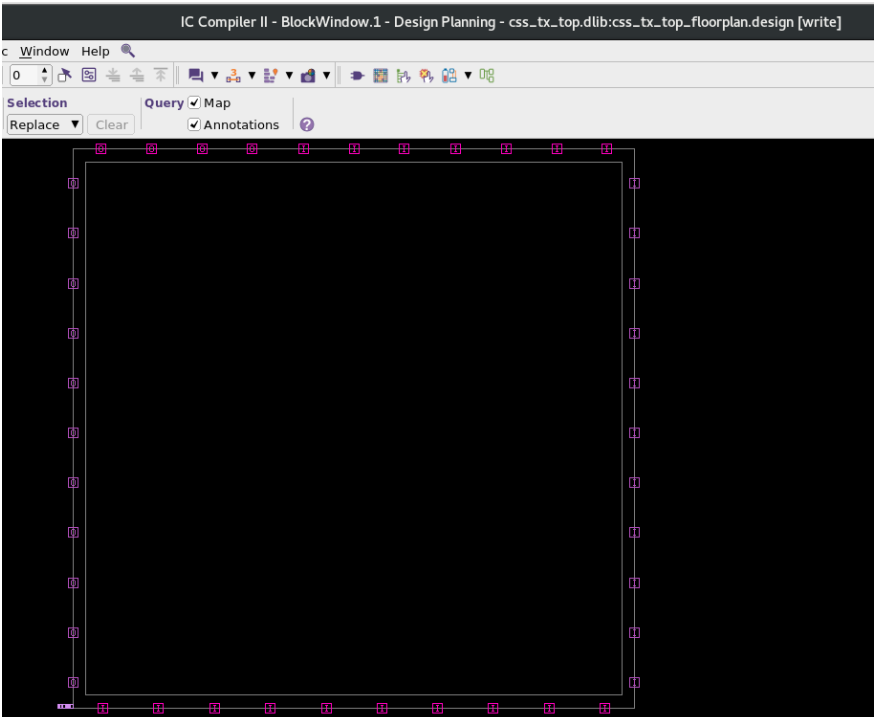
IC Compiler II design library (.dlib) created for css_tx_top: Stage 1 netlist and SDC imported, TLUPlus parasitic technology (saed90nm_1p9m_1t_Cmax, worst corner, 125 °C) attached for later RC extraction, and the block checkpointed as css_tx_top_dlib.

Check	Result
Clock read-back	clk, 31.25 ns, matches Stage 1 SDC
Parasitic tech attached	tluplus_max → saed90nm_1p9m_1t_Cmax (confirmed)
Netlist / SDC import	No errors
Preliminary timing (report_timing)	All paths MET, but with zero interconnect delay — not a real signoff

8.1.3 Stage 3 — Floorplanning

The die/core outline and I/O pin ring were placed in IC Compiler II (Design Planning), and timing was re-checked using floorplan-based RC estimation (VR mode) instead of the zero-interconnect placeholder from Stage 2.

8.1.3.1 Floorplan



Rectangular core with I/O pins evenly distributed on all four edges; no hard macros, keepouts, or placement blockages are defined (blockage.rpt returns no matching objects — expected, since the design uses no hard IP). No standard cells are placed yet at this stage.

8.1.3.2 Utilization

Metric	Value
Utilization ratio	60.3%
Total core area	170,402.92
Total cell area	102,720.61 (matches Stage 1)
Hard macros / blockages	0

60% utilization leaves comfortable headroom for placement and routing without congestion. Cell area is unchanged from synthesis, confirming no netlist drift through the floorplan step.

8.1.3.3 Timing (QoR, floorplan-based RC estimate)

Metric	Value
Critical path length / slack	11.97 ns / +18.39 ns (MET)
Setup: TNS / violating paths	0.00 ns / 0
Hold: WNS / TNS / violations	-0.03 ns / -0.03 ns / 1
Max transition / max cap violations	0 / 0 (8,591 nets checked)

Setup is now fully clean under the floorplan-based RC estimate, and the 13 max-transition violations from Stage 1 have cleared — consistent with the Stage 1 conclusion that those were wire-load-model pessimism rather than real logic issues. One new, very small hold violation appears (-0.03 ns, a single path); this is typical at the floorplan stage before clock tree synthesis and should be re-checked post-CTS/post-route rather than acted on now.

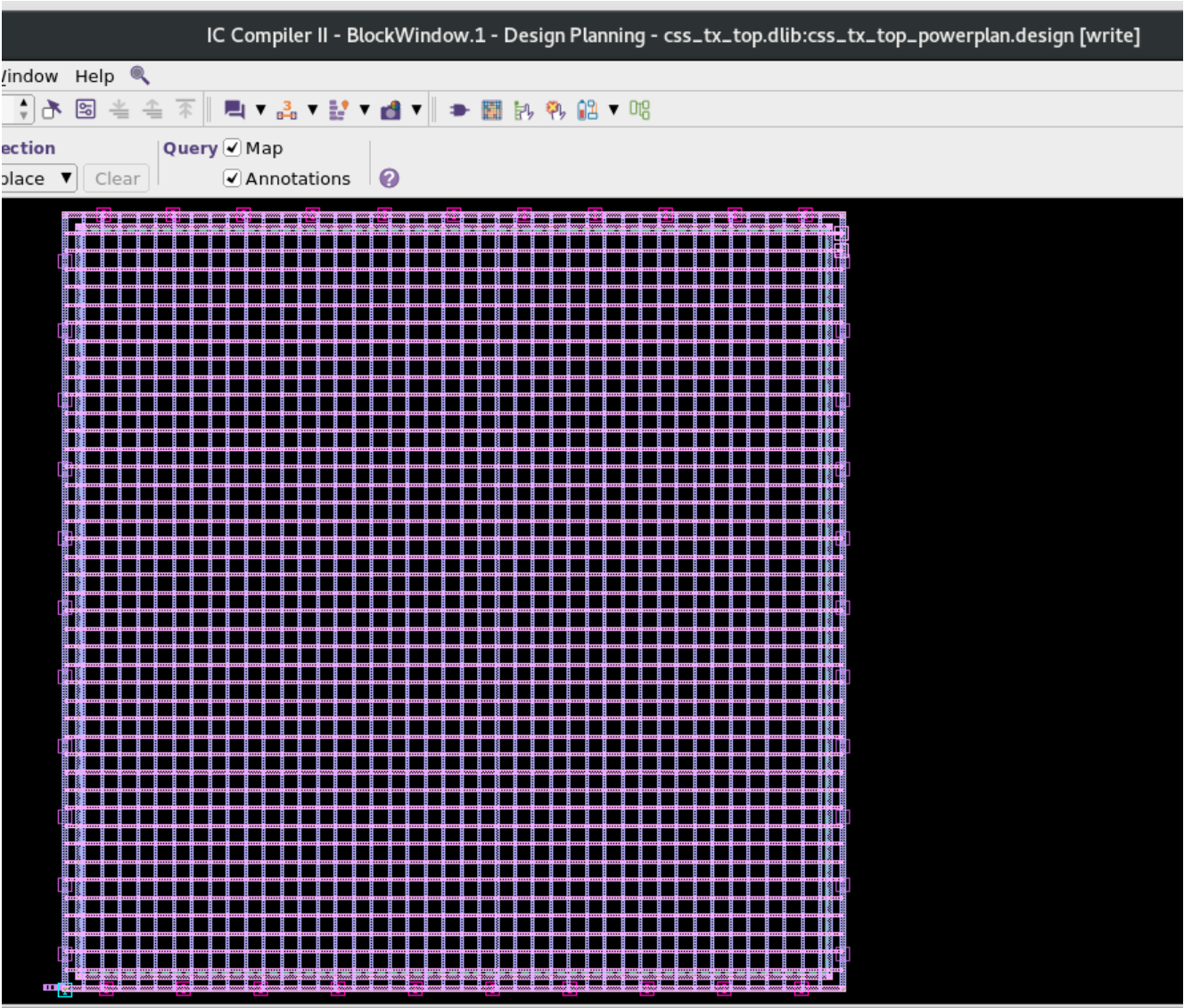
Note: this is still a pre-placement RC estimate (virtual routing, not extracted parasitics), so it is an improved but still not final signoff-quality result.

8.1.4 Stage 4 — Power Plan

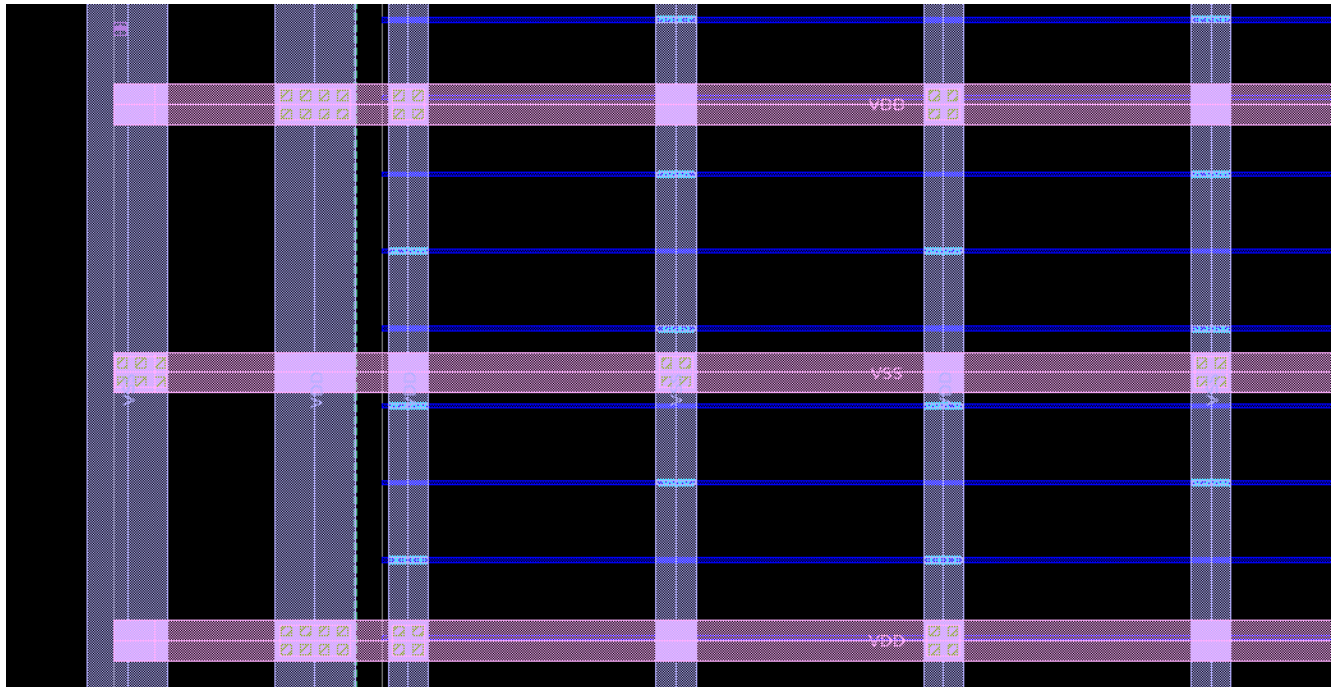
Built on the Stage 3 floorplan in IC Compiler II: a core-level VDD/VSS ring, an M8/M9 power mesh, and M1 standard-cell power rails, followed by full connectivity/DRC verification

8.1.4.1 Structure

Element	Spec
Core ring	M9 (horiz.) / M8 (vert.), width 3, spacing 4, offset 1 from core edge
Power mesh	M8 (vert.) / M9 (horiz.) straps, width 1.5, pitch 20, interleaved, extended to design boundary
Std-cell rails	M1, width 0.16, one pair per cell row
Nets	VDD (power), VSS (ground) — tied to all standard-cell pins via connect_pg_net



Ring + mesh + rails across the full core (left) and a zoomed-in view of a strap/rail intersection with via stacks (right, not to same scale).



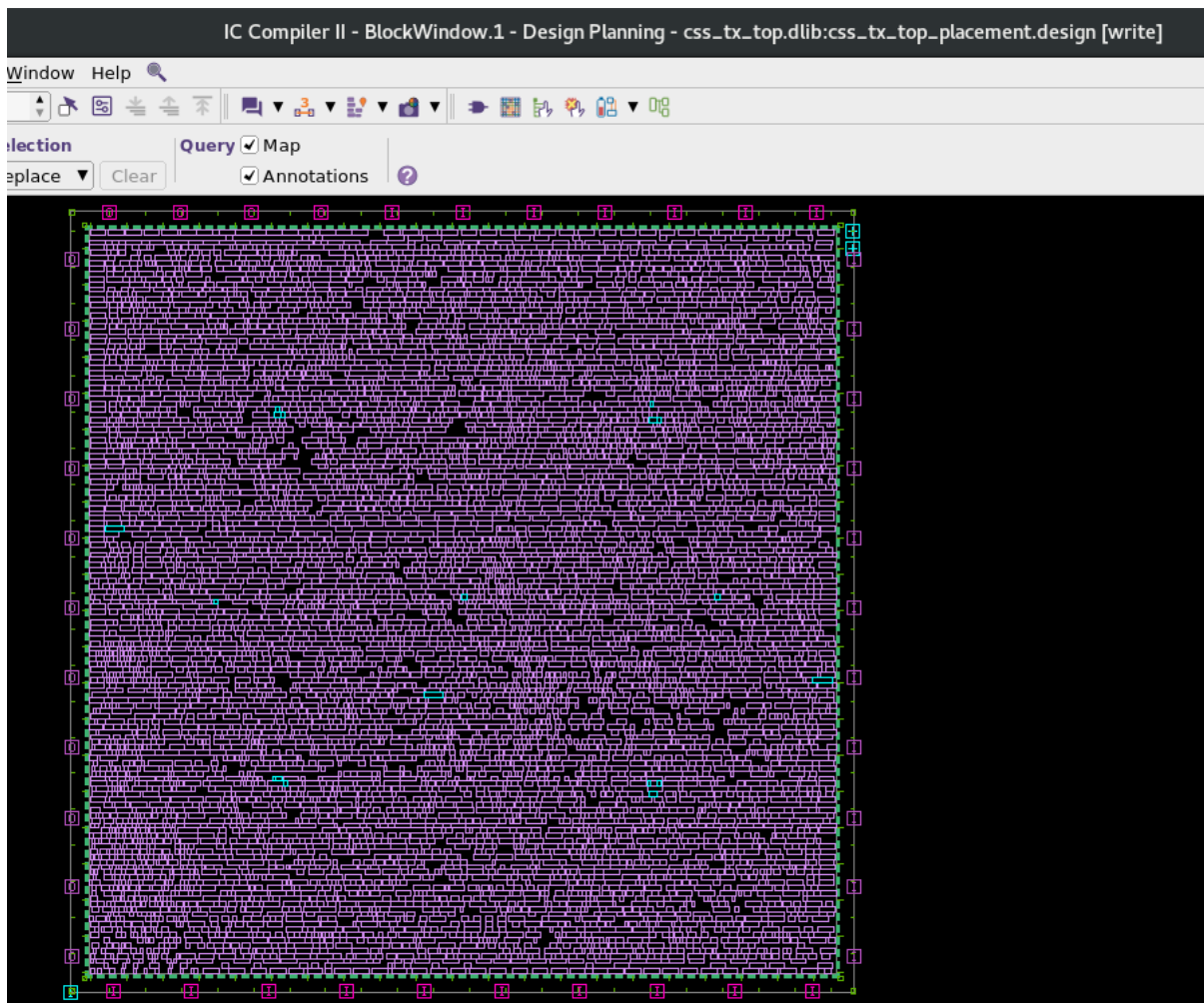
8.1.4.2 Verification

Check	Result
check_pg_drc	No errors found
check_pg_missing_vias	0 missing vias on VDD, 0 on VSS
check_pg_connectivity — VDD	118 wires / 11,113 vias / 84 terminals — 0 floating (wires, vias, 8,359 std cells)
check_pg_connectivity — VSS	118 wires / 11,113 vias / 84 terminals — 0 floating (wires, vias, 8,359 std cells)

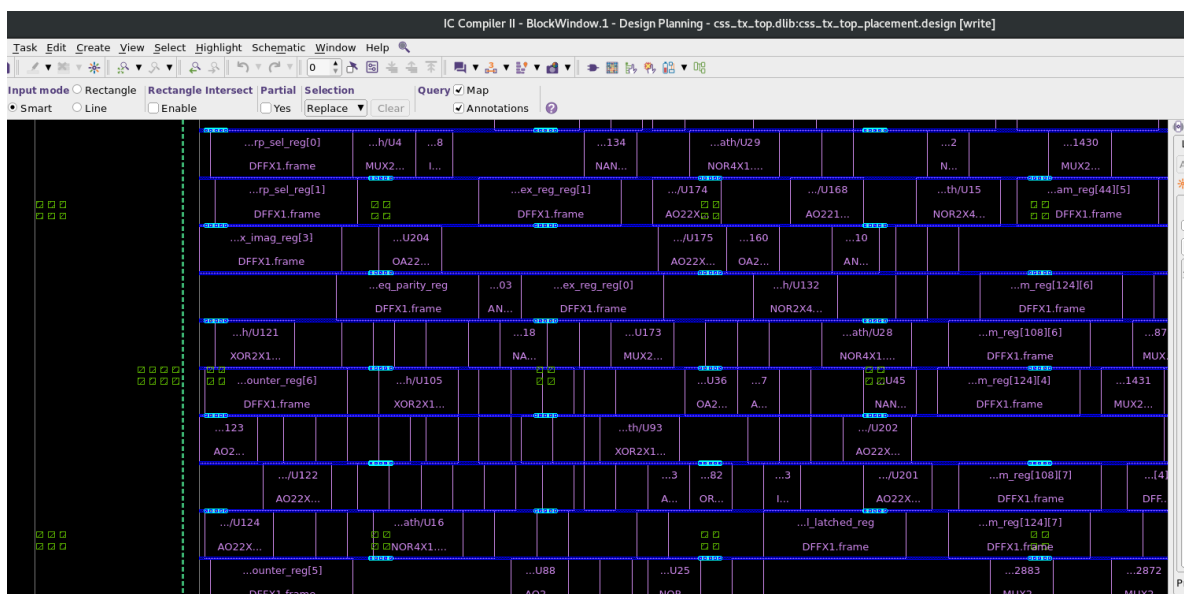
Power/ground network is fully clean: every standard cell resolves to both VDD and VSS with no floating nets, wires, or vias, and no DRC or via violations. DEF, netlist, and SDC were written out and the block saved. Cleared to proceed to placement.

8.1.4.3 Stage 5 — Placement

Detailed placement was run on top of the power plan (timing-driven, congestion-driven, high effort), followed by placement optimization (place_opt), spare-cell insertion for ECO margin, and re-verification of the power/ground network.



Full-core placement (left) and a zoomed schematic view of placed/mapped cells (right, not to same scale).



8.1.4.4 QOR after Placement:

Metric	Value
Leaf cells / Buf-Inv cells	7,461 / 1,197 (down from 8,367 / 2,109 pre-placement)
Cell area (netlist)	99,293.18 (down from 102,720.61)
Critical path length / slack	14.04 ns / +16.32 ns (MET)
Setup: TNS / violating paths	0.00 ns / 0
Hold: WNS / TNS / violations	0.00 ns / 0.00 ns / 0
Max transition / max cap violations	0 / 0
Utilization	58.2% (was 60.3% at floorplan)

Placement optimization resolved both open items from earlier stages: the single hold violation from Stage 3 is now clean, and the Stage 1 max-transition violations are fully cleared. Cell area and buffer count also dropped, reflecting re-sizing/re-buffering done during place_opt.

8.1.4.5 Power/Ground Re-verification:

Check	Result
check_pg_drc	No errors found
check_pg_missing_vias	0 missing vias on VDD, 0 on VSS
check_pg_connectivity	7,461 std cells — 0 floating on VDD or VSS

Notes

- 18 spare cells added (NAND2X1, INVX1, OR2X1, SDFFX1, MUX21X1) and tied, reserved for post-silicon ECO.
- High-fanout check flags only clk (1,843) and reset (21) — expected, since the clock is still an ideal net; a real clock tree is built in the next stage.
- No placement blockages are defined, consistent with the floorplan and power-plan stages.

DEF, netlist, and SDC were written out and the block saved. Cleared to proceed to clock tree synthesis.

8.1.5 Stage 6 — Clock Tree Synthesis (CTS)

The ideal clock network was removed and a real clock tree was built and routed for clk (CCD flow, target skew 0.1 ns, high hold-control effort), using non-default routing rules for extra shielding on the internal/root clock nets. Post-CTS optimization (final_opto) and PG re-verification followed.

8.1.5.1 Clock Tree:



12-level buffer/inverter tree built from clk, using IBUFFX/NBUFFX/INVS cells (x2–x16). Root and internal clock nets route on M6–M9 and M3–M6 respectively with 2x width/spacing non-default rules for robustness; sink nets route on M1–M3.

8.1.5.2 Timing After CTS:

Metric	Value
Setup: WNS / TNS / violations	16.32 ns / 0.00 ns / 0
Hold: WNS / TNS / violations	0.06 ns / 0.00 ns / 0
report_constraint -all_violators	0 late-timing, 0 early-timing violations
Nets with DRC violations	0
Target skew	0.1 ns

Both setup and hold are fully clean after CTS and post-CTS optimization — confirmed independently by report_constraint -all_violators, which finds zero violators in either direction.

8.1.5.3 Power/Ground and Routing Verification:

Check	Result
check_pg_drc	No errors found
check_pg_missing_vias	0 missing vias on VDD, 0 on VSS
check_pg_connectivity	7,937 std cells — 0 floating on VDD or VSS
check_routes -drc true	0 DRC violations (28,974 μ m routed, mostly M1–M4 clock wiring)

check_routes also flags most non-clock signal nets as still “open” (7,653 of 8,130) — expected at this stage, since only the clock network has been routed so far; full signal routing is the next stage. The DRC check itself, which is what matters at CTS, comes back completely clean.

8.1.5.4 Stage 7 — Routing (Final PnR Stage)

Detailed routing and post-route optimization (route_opt) were run on top of the CTS design, followed by decap and filler cell insertion to close out the layout, then the full signoff verification suite (LVS, routing DRC, legality, PG DRC).

8.1.5.5 Routing Results:

Metric	Value
Total wire length	292,018 μm (290,639 μm routed)
Total vias	63,518
Open nets	0 of 8,130
Routing DRC violations	0
Dominant layers	M2 (103,130 μm) and M3 (109,798 μm) — bulk of signal routing on lower metals

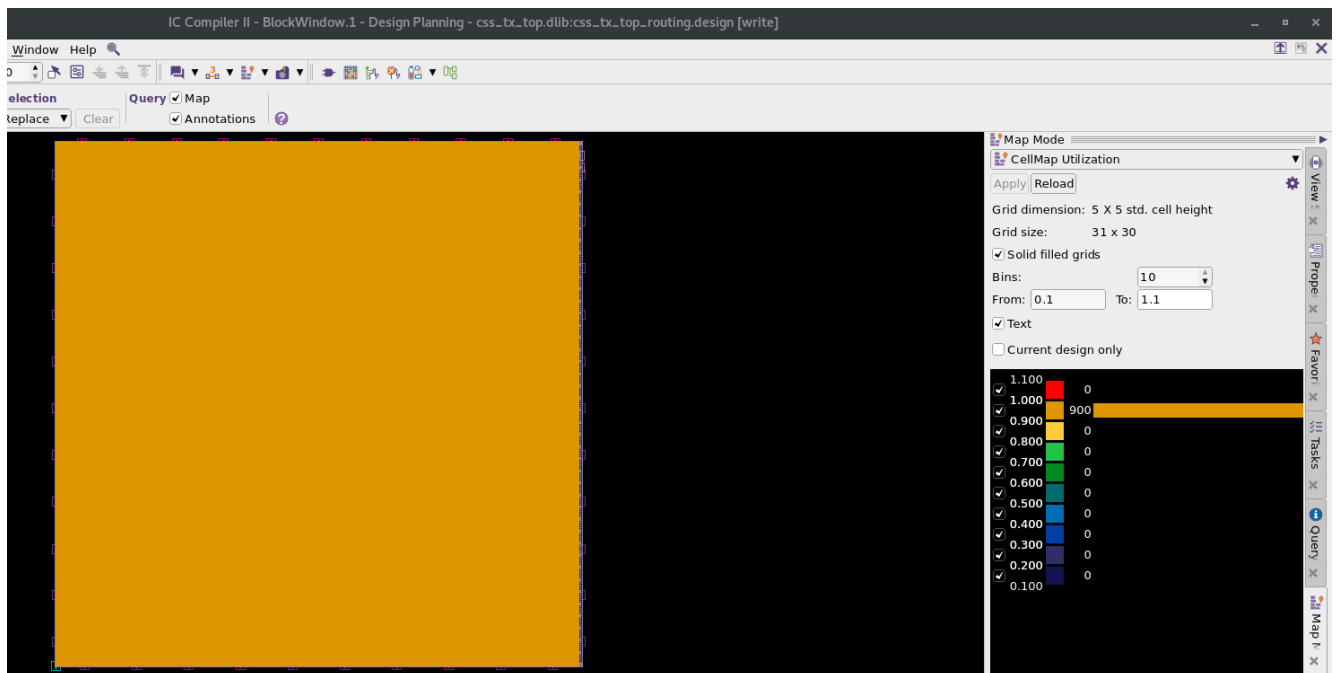
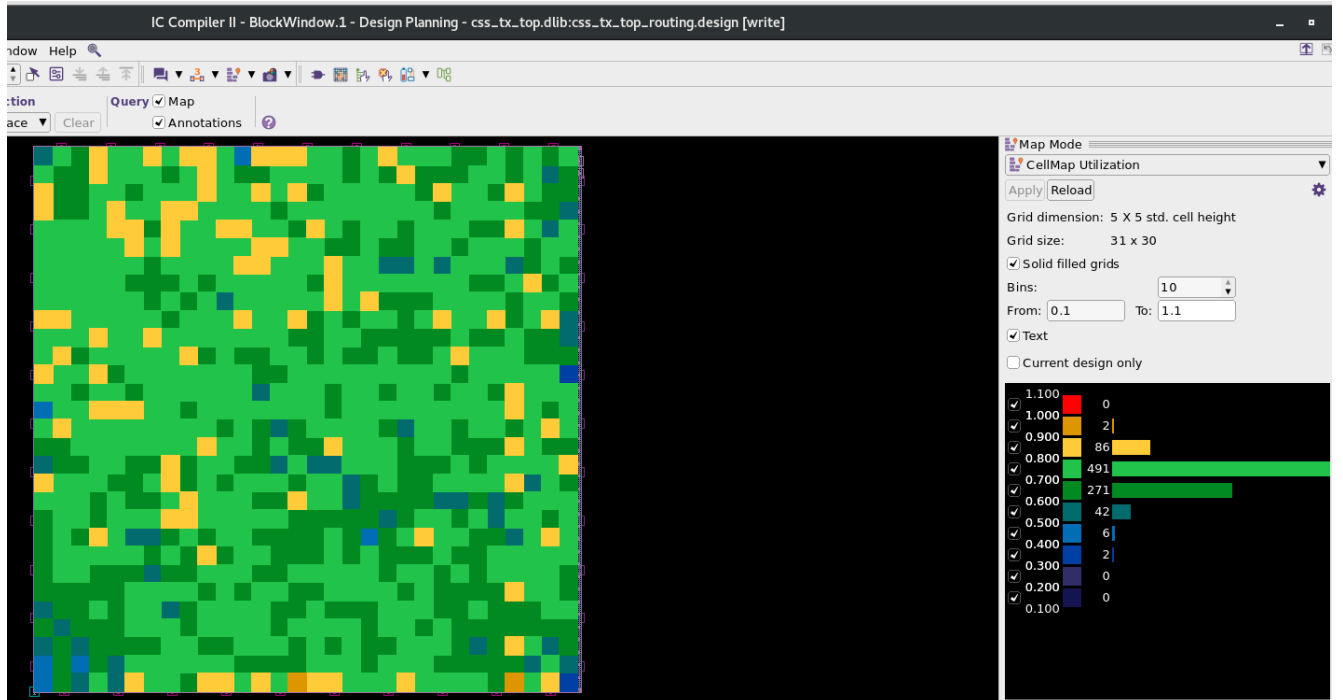
8.1.5.6 Final sign-off verification:

Check	Result
check_routes	0 DRC violations, 0 open nets (8,130 nets checked)
check_lvs	0 shorts, 0 opens, 0 floating routes
check_legality	0 violations across all categories (overlap, spacing, site, orientation, region, ...)
check_pg_drc	No errors found

Every signoff-level physical check comes back completely clean — the routed layout matches the netlist exactly (LVS), has zero DRC violations, and is fully legal

8.1.5.7 Decap / Filler Insertion

Decap cells (DCAP*) were inserted at a 30% utilization target for supply-noise mitigation, followed by tiered non-metal fillers (SHFILL128/64/3/2/1) to close every remaining gap in the rows — 19,793 filler cells in total. The cell-map utilization view confirms the effect: mixed 50–90% row density before filler insertion becomes a uniform 90% across all 900 grid bins afterward.





Cell-map utilization before (top) and after (bottom) decap/filler insertion.

8.1.6 Final Outputs

- DEF, full gate-level Verilog netlist, and SDC written out.
- A GLS-only netlist (logical, excluding PG pins/fillers/decaps) written for gate-level simulation.
- Parasitics exported to SPEF for PrimeTime signoff STA and SDF generation.
- Block saved; GDSII stream-out and CDL/LVS-for-tapeout are the remaining foundry-facing steps (pending the layer map).

8.1.7 Overall Status — Flow Complete

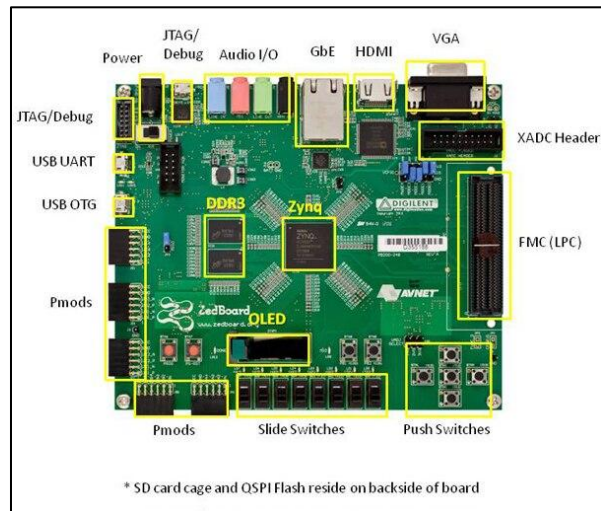
- Run signoff STA in PrimeTime using the exported SPEF, across all required PVT corners (this flow covered only the worst 125 °C corner).
- Generate GDSII/OASIS with the foundry layer map and run final tapeout-level LVS/DRC against the full transistor netlist.
- Retain the 18 ECO spare/tie cells' placement in case post-silicon metal-only fixes are needed.
- No RTL or constraint changes are recommended — physical implementation is complete and clean end to end.

css_tx_top has passed every stage of the ASIC implementation flow — synthesis, design library setup, floorplan, power plan, placement, CTS, and routing — with a clean bill of health at each step. Routing closes with 0 DRC violations, 0 open nets, 0 LVS errors, 0 legality violations, and 0 PG DRC errors, and the layout is fully filled. The design is physically complete and ready for signoff STA and GDSII tapeout.

8.2 FPGA FLOW:

8.2.1 FPGA type:

The **ZedBoard** is a popular evaluation and development platform built around the AMD Xilinx Zynq-7000 All Programmable System-on-Chip (SoC), specifically combining a dual-core ARM Cortex-A9 processing system with 28nm Artix-7 programmable logic (XC7Z020-CLG484-1). It features 512 MB of DDR3 memory, 256 Mb of QSPI Flash, and an onboard USB-JTAG programming interface alongside standard Gigabit Ethernet, USB OTG, HDMI output, VGA, and OLED display options. For hardware prototyping and expansion, the board offers eight 12-pin PMOD expansion connectors and an FMC (FPGA Mezzanine Card) connector. As part of the Information Technology Institute Digital IC Design 2026 training program, the ZedBoard served as a primary target hardware platform for synthesizing, implementing, and physically verifying custom register-transfer level (RTL) architectures—such as digital transmitters and signal processing IP blocks—within Xilinx Vivado.



8.2.2 Constraint file:

```
# -----  
#  
#  
# /_____\br/># / \====\ \==/  
# /___\====\___\ AVNET Design Resource Center  
# \=====/  
# \=====/  
# -----  
#  
# Created With Avnet UCF Generator V0.4.0  
# Date: Saturday, June 30, 2012  
# Time: 12:18:55 AM  
#  
# This design is the property of Avnet. Publication of this  
# design is not authorized without written consent from Avnet.  
#  
# Please direct any questions to:  
# ZedBoard.org Community Forums  
# http://www.zedboard.org
```

```

#
# Disclaimer:
#   Avnet, Inc. makes no warranty for the use of this code or design.
#   This code is provided "As Is". Avnet, Inc assumes no responsibility for
#   any errors, which may appear in this code, nor does it make a commitment
#   to update the information contained herein. Avnet, Inc specifically
#   disclaims any implied warranties of fitness for a particular purpose.
#   Copyright(c) 2012 Avnet, Inc.
#   All rights reserved.
#
# -----
#
# Notes:
#
# 10 August 2012
#   IO standards based upon Bank 34 and Bank 35 Vcco supply options of 1.8V,
#   2.5V, or 3.3V are possible based upon the Vadj jumper (J18) settings.
#   By default, Vadj is expected to be set to 1.8V but if a different
#   voltage is used for a particular design, then the corresponding IO
#   standard within this UCF should also be updated to reflect the actual
#   Vadj jumper selection.
#
# 09 September 2012
#   Net names are not allowed to contain hyphen characters '-' since this
#   is not a legal VHDL87 or Verilog character within an identifier.
#   HDL net names are adjusted to contain no hyphen characters '-' but
#   rather use underscore '_' characters. Comment net name with the hyphen
#   characters will remain in place since these are intended to match the
#   schematic net names in order to better enable schematic search.
#
# 17 April 2014
#   Pin constraint for toggle switch SW7 was corrected to M15 location.
#
# 16 April 2015
#   Corrected the way that entire banks are assigned to a particular IO
#   standard so that it works with more recent versions of Vivado Design
#   Suite and moved the IO standard constraints to the end of the file
#   along with some better organization and notes like we do with our SOMs.
#
# 6 June 2016
#   Corrected error in signal name for package pin N19 (FMC Expansion
#   Connector)
#
# -----

```

```

#-----timing constraints-----
#-----

# create_clock -name clk -period 8 [get_ports clk]

# =====
# ZedBoard Constraints File for IEEE 802.15.4 CSS PHY Transmitter (css_tx_top)
# Target Board: ZedBoard (xc7z020clg484-1)
# =====

# -----
# 1. Primary Clock Definition
# -----

set_property PACKAGE_PIN Y9 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]

# Primary 100 MHz System Clock (10.000 ns Period, 50% Duty Cycle)
create_clock -period 31.250 -name sys_clk -waveform {0.000 5.000} [get_ports clk]

# Clock Jitter and Timing Uncertainty Constraints
set_clock_uncertainty -setup 0.200 [get_clocks sys_clk]
set_clock_uncertainty -hold 0.050 [get_clocks sys_clk]

# -----
# 2. Asynchronous Control Inputs (Push Buttons & DIP Switches)
# Bank 34 (2.5V Voltage Domain)
# -----

# Active-High Reset -> Push Button BTNC (Center Pin P16)
set_property PACKAGE_PIN P16 [get_ports reset]
set_property IOSTANDARD LVCMOS25 [get_ports reset]

# start_tx Pulse -> Push Button BTNU (Up Pin N15)
set_property PACKAGE_PIN N15 [get_ports start_tx]
set_property IOSTANDARD LVCMOS25 [get_ports start_tx]

# CSK Sequence Select: chirp_index[1:0] -> button1, button0
set_property PACKAGE_PIN T18 [get_ports {chirp_index[0]}]
set_property PACKAGE_PIN R18 [get_ports {chirp_index[1]}]

set_property IOSTANDARD LVCMOS25 [get_ports {chirp_index[*]}]

# Payload RAM Write Enable: payload_we -> SW7

```

```

set_property PACKAGE_PIN M15 [get_ports payload_we]
set_property IOSTANDARD LVCMOS25 [get_ports payload_we]

# Payload Length Register: payload_length[6:0]
set_property PACKAGE_PIN F22 [get_ports {payload_length[0]}] ;# SW0
set_property PACKAGE_PIN G22 [get_ports {payload_length[1]}] ;# SW1
set_property PACKAGE_PIN H22 [get_ports {payload_length[2]}] ;# SW2
set_property PACKAGE_PIN F21 [get_ports {payload_length[3]}] ;# SW3
set_property PACKAGE_PIN H19 [get_ports {payload_length[4]}] ;# SW4
set_property PACKAGE_PIN H18 [get_ports {payload_length[5]}] ;# SW5
set_property PACKAGE_PIN H17 [get_ports {payload_length[6]}] ;# SW6
set_property IOSTANDARD LVCMOS25 [get_ports {payload_length[*]}]

# -----
# 3. Memory & Address Buses (Mapped to PMOD Headers JC & JD)
# Bank 13 (3.3V Voltage Domain)
# -----
# Payload Write Data Bus: payload_wdata[7:0] -> Mapped to PMOD JC
set_property PACKAGE_PIN AB6 [get_ports {payload_wdata[0]}] ;# JC1_N
set_property PACKAGE_PIN AB7 [get_ports {payload_wdata[1]}] ;# JC1_P
set_property PACKAGE_PIN AA4 [get_ports {payload_wdata[2]}] ;# JC2_N
set_property PACKAGE_PIN Y4 [get_ports {payload_wdata[3]}] ;# JC2_P
set_property PACKAGE_PIN T6 [get_ports {payload_wdata[4]}] ;# JC3_N
set_property PACKAGE_PIN R6 [get_ports {payload_wdata[5]}] ;# JC3_P
set_property PACKAGE_PIN U4 [get_ports {payload_wdata[6]}] ;# JC4_N
set_property PACKAGE_PIN T4 [get_ports {payload_wdata[7]}] ;# JC4_P
set_property IOSTANDARD LVCMOS33 [get_ports {payload_wdata[*]}]

# Payload Write Address Bus: payload_waddr[6:0] -> Mapped to PMOD JD
set_property PACKAGE_PIN W7 [get_ports {payload_waddr[0]}] ;# JD1_N
set_property PACKAGE_PIN V7 [get_ports {payload_waddr[1]}] ;# JD1_P
set_property PACKAGE_PIN V4 [get_ports {payload_waddr[2]}] ;# JD2_N
set_property PACKAGE_PIN V5 [get_ports {payload_waddr[3]}] ;# JD2_P
set_property PACKAGE_PIN W5 [get_ports {payload_waddr[4]}] ;# JD3_N
set_property PACKAGE_PIN W6 [get_ports {payload_waddr[5]}] ;# JD3_P
set_property PACKAGE_PIN U5 [get_ports {payload_waddr[6]}] ;# JD4_N
set_property IOSTANDARD LVCMOS33 [get_ports {payload_waddr[*]}]

# -----
# 4. Status Outputs (On-Board LEDs)
# Bank 33 (3.3V Voltage Domain)
# -----
# Transmission Done Indicator -> LED LD0
set_property PACKAGE_PIN T22 [get_ports tx_done]
set_property IOSTANDARD LVCMOS33 [get_ports tx_done]

```

```

# Baseband Valid Flag Indicator -> LED LD1
set_property PACKAGE_PIN T21 [get_ports tx_valid]
set_property IOSTANDARD LVCMOS33 [get_ports tx_valid]

# Controller FIFO Overflow Alarm -> LED LD7
set_property PACKAGE_PIN U14 [get_ports fifo_overflow]
set_property IOSTANDARD LVCMOS33 [get_ports fifo_overflow]

# -----
# 5. Baseband I/Q Output Signals (Mapped to PMOD Headers JA & JB)
# Bank 13 (3.3V Voltage Domain)
# -----
# Real Baseband Component: tx_real[5:0] -> PMOD JA Pins (1, 2, 3, 4, 7, 8)
set_property PACKAGE_PIN Y11 [get_ports {tx_real[0]}] ;# JA1
set_property PACKAGE_PIN AA11 [get_ports {tx_real[1]}] ;# JA2
set_property PACKAGE_PIN Y10 [get_ports {tx_real[2]}] ;# JA3
set_property PACKAGE_PIN AA9 [get_ports {tx_real[3]}] ;# JA4
set_property PACKAGE_PIN AB11 [get_ports {tx_real[4]}] ;# JA7
set_property PACKAGE_PIN AB10 [get_ports {tx_real[5]}] ;# JA8
set_property IOSTANDARD LVCMOS33 [get_ports {tx_real[*]}]

# Imaginary Baseband Component: tx_imag[5:0] -> PMOD JB Pins (1, 2, 3, 4, 7, 8)
set_property PACKAGE_PIN W12 [get_ports {tx_imag[0]}] ;# JB1
set_property PACKAGE_PIN W11 [get_ports {tx_imag[1]}] ;# JB2
set_property PACKAGE_PIN V10 [get_ports {tx_imag[2]}] ;# JB3
set_property PACKAGE_PIN W8 [get_ports {tx_imag[3]}] ;# JB4
set_property PACKAGE_PIN V12 [get_ports {tx_imag[4]}] ;# JB7
set_property PACKAGE_PIN W10 [get_ports {tx_imag[5]}] ;# JB8
set_property IOSTANDARD LVCMOS33 [get_ports {tx_imag[*]}]

# -----
# 6. Timing Exceptions & Delay Constraints
# -----
# # Treat manual, non-clocked switch and push-button inputs as asynchronous
# set_false_path -from [get_ports reset]
# set_false_path -from [get_ports start_tx]
# set_false_path -from [get_ports {chirp_index[*]}]
# set_false_path -from [get_ports {payload_length[*]}]

set_false_path -from
u_tx_datapath/u_css_symbol_generator/u_complex_multiplier/tx_imag_reg[*]/C -
to tx_imag[*]

```

```

set_false_path -from
u_tx_datapath/u_css_symbol_generator/u_complex_multiplier/tx_real_reg[3]/C -
to tx_real[*]
set_false_path -from
u_tx_datapath/u_css_symbol_generator/u_csk_generator/done_reg/C -to tx_done
set_false_path -from
u_tx_datapath/u_css_symbol_generator/u_complex_multiplier/valid_out_reg/C -
to tx_valid
set_false_path -from u_controller/fifo_overflow_reg/C -to fifo_overflow
set_false_path -from
u_tx_datapath/u_css_symbol_generator/u_complex_multiplier/tx_real_reg[*]/C -to
tx_real

# set_input_delay -from
u_tx_datapath/u_css_symbol_generator/u_complex_multiplier/tx_imag_reg[0]/C -
to tx_imag[0]
# set_input_delay -from
u_tx_datapath/u_css_symbol_generator/u_complex_multiplier/tx_imag_reg[1]/C -
to tx_imag[1]

# # Relax timing paths for status LED display pins
# set_false_path -to [get_ports tx_done]
# set_false_path -to [get_ports fifo_overflow]

# Set Input/Output Delay Budget for High-Speed Bus Verification (PMODs)

set_input_delay -max 0.35 -clock [get_clocks sys_clk] [get_ports reset]
set_input_delay -min 0.35 -clock [get_clocks sys_clk] [get_ports reset]
# set_input_delay -max 0.5 -clock [get_clocks clk] [get_ports payload_we]
# set_input_delay -min 0.5 -clock [get_clocks clk] [get_ports payload_we]
# set_input_delay -max 0.5 -clock [get_clocks clk] [get_ports payload_waddr]
# set_input_delay -min 0.5 -clock [get_clocks clk] [get_ports payload_waddr]
# set_input_delay -max 0.5 -clock [get_clocks clk] [get_ports payload_wdata]
# set_input_delay -min 0.5 -clock [get_clocks clk] [get_ports payload_wdata]
set_input_delay -max 0.3 -clock [get_clocks sys_clk] [get_ports payload_length]
set_input_delay -min 0.3 -clock [get_clocks sys_clk] [get_ports payload_length]
set_input_delay -max 0.3 -clock [get_clocks sys_clk] [get_ports chirp_index]
set_input_delay -min 0.3 -clock [get_clocks sys_clk] [get_ports chirp_index]
set_input_delay -max 0.3 -clock [get_clocks sys_clk] [get_ports start_tx]
set_input_delay -min 0.3 -clock [get_clocks sys_clk] [get_ports start_tx]

```

```

set_output_delay -max 0.5 -clock [get_clocks sys_clk] [get_ports tx_done]
set_output_delay -min 0.5 -clock [get_clocks sys_clk] [get_ports tx_done]
# set_output_delay -max 0.5 -clock [get_clocks clk] [get_ports tx_real]
# set_output_delay -min 0.5 -clock [get_clocks clk] [get_ports tx_real]
# set_output_delay -max 0.5 -clock [get_clocks clk] [get_ports tx_imag]
# set_output_delay -min 0.5 -clock [get_clocks clk] [get_ports tx_imag]
# set_output_delay -max 0.5 -clock [get_clocks clk] [get_ports tx_valid]
# set_output_delay -min 0.5 -clock [get_clocks clk] [get_ports tx_valid]
set_output_delay -max 0.5 -clock [get_clocks sys_clk] [get_ports fifo_overflow]
set_output_delay -min 0.5 -clock [get_clocks sys_clk] [get_ports fifo_overflow]

set_input_delay -clock [get_clocks sys_clk] -max 0.5 [get_ports
{payload_wdata[*] payload_waddr[*] payload_we}]
set_input_delay -clock [get_clocks sys_clk] -min 0.500 [get_ports
{payload_wdata[*] payload_waddr[*] payload_we}]

set_output_delay -clock [get_clocks sys_clk] -max 0.5 [get_ports {tx_real[*]
tx_imag[*] tx_valid}]
set_output_delay -clock [get_clocks sys_clk] -min 0.500 [get_ports {tx_real[*]
tx_imag[*] tx_valid}]

# -----
# 7. Device Configuration Settings
# -----
#set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]

# Un-comment one or more of the following IOSTANDARD constraints according to
# the bank pin assignments that are required within a design.
# -----

# Note that the bank voltage for IO Bank 33 is fixed to 3.3V on ZedBoard.
##set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 33]];

# Set the bank voltage for IO Bank 34 to 1.8V by default.
# set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 34]];
# set_property IOSTANDARD LVCMOS25 [get_ports -of_objects [get_iobanks 34]];
##set_property IOSTANDARD LVCMOS18 [get_ports -of_objects [get_iobanks 34]];

# Set the bank voltage for IO Bank 35 to 1.8V by default.
# set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 35]];
# set_property IOSTANDARD LVCMOS25 [get_ports -of_objects [get_iobanks 35]];
##set_property IOSTANDARD LVCMOS18 [get_ports -of_objects [get_iobanks 35]];

# Note that the bank voltage for IO Bank 13 is fixed to 3.3V on ZedBoard.

```

```
##set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 13]];
```

8.2.3 Resource Utilization & timings:

We had done couple of runs with different synthesis and implementation strategies to select the best approach in terms of timing and utilization of resources to have the best WNS and the least resources

Name	Constraints	Status	WNS	TNS	WHS	THS	TPWS	Fail...	LUT	FF	BRAMS	URAM	DSP	Elapsed	Run Strategy	Report Str
synth_1	constrs_1	synth_design Complete!							301	294	1.5	0	0	9 00:01:09	Vivado Synthesis Defaults (Vivado Synthesis 2019)	Vivado Syn
impl_1	constrs_1	route_design Complete!	0.500	0.000	0.080	0.000	0.000	0	0	304	300	1.5	0	9 00:03:17	Flow_AreaOptimized_high (Vivado Synthesis 2019)	Vivado Imp
synth_2	constrs_1	synth_design Complete!							286	294	1.5	0	0	9 00:01:13	Flow_AreaOptimized_high (Vivado Synthesis 2019)	Vivado Syn
impl_2	constrs_1	route_design Complete!	0.700	0.000	0.056	0.000	0.000	0	0	290	300	1.5	0	9 00:03:22	Vivado Implementation Defaults (Vivado Implementation 2019)	Vivado Imp
synth_3 (active)	constrs_1	synth_design Complete!							401	294	1.5	0	0	9 00:01:11	Flow_AlternateRoutability (Vivado Synthesis 2019)	Vivado Syn
impl_3 (active)	constrs_1	route_design Complete!	0.618	0.000	0.056	0.000	0.000	0	0	401	300	1.5	0	9 00:03:29	Vivado Implementation Defaults (Vivado Implementation 2019)	Vivado Imp
synth_4	constrs_1	synth_design Complete!							288	294	1.5	0	0	9 00:01:12	Flow_AreaOptimized_medium (Vivado Synthesis 2019)	Vivado Syn
impl_4	constrs_1	route_design ERROR						0	0	295	300	1.5	0	9 00:03:36	Vivado Implementation Defaults (Vivado Implementation 2019)	Vivado Imp
synth_5	constrs_1	synth_design Complete!							434	308	1.5	0	0	9 00:01:10	Flow_PerfOptimized_high (Vivado Synthesis 2019)	Vivado Syn
impl_5	constrs_1	place_design ERROR												9 00:02:55	Vivado Implementation Defaults (Vivado Implementation 2019)	Vivado Imp
synth_6	constrs_1	synth_design Complete!							306	291	1.5	0	0	9 00:01:08	Flow_RuntimeOptimized (Vivado Synthesis 2019)	Vivado Syn
impl_6	constrs_1	route_design ERROR						0	0	308	297	1.5	0	9 00:03:18	Vivado Implementation Defaults (Vivado Implementation 2019)	Vivado Imp
synth_7	constrs_1	synth_design Complete!							301	294	1.5	0	0	9 00:01:09	Flow_AreaMultThresholdDSP (Vivado Synthesis 2019)	Vivado Syn
impl_7	constrs_1	route_design Complete!	0.500	0.000	0.080	0.000	0.000	0	0	304	300	1.5	0	9 00:03:19	Vivado Implementation Defaults (Vivado Implementation 2019)	Vivado Imp
synth_8	constrs_1	synth_design Complete!							286	294	1.5	0	0	9 00:01:13	Flow_AreaOptimized_high (Vivado Synthesis 2019)	Vivado Syn
impl_8	constrs_1	route_design Complete!	0.700	0.000	0.056	0.000	0.000	0	0	290	300	1.5	0	9 00:03:25	Vivado Implementation Defaults (Vivado Implementation 2019)	Vivado Imp

After observation run no. 8 (Flow AreaOptimized_High) strategy has the a WNS od 0.7 and LUT used 290 and FF used 300 which is considered the optimum option from all runs made

synth_8	constrs_1	synth_design Complete!								286	294	1.5	0	0	9 00:01:13	Flow_AreaOptimized_high (Vivado Synthesis 2019)	Vivado Syn
impl_8	constrs_1	route_design Complete!	0.700	0.000	0.056	0.000	0.000	0	0	290	300	1.5	0	0	9 00:03:25	Vivado Implementation Defaults (Vivado Implementation 2019)	Vivado Imp

8.2.3.1 Timing report:

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.700 ns	Worst Hold Slack (WHS): 0.056 ns	Worst Pulse Width Slack (WPWS): 3.750 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 1000	Total Number of Endpoints: 1000	Total Number of Endpoints: 321

All user specified timing constraints are met.

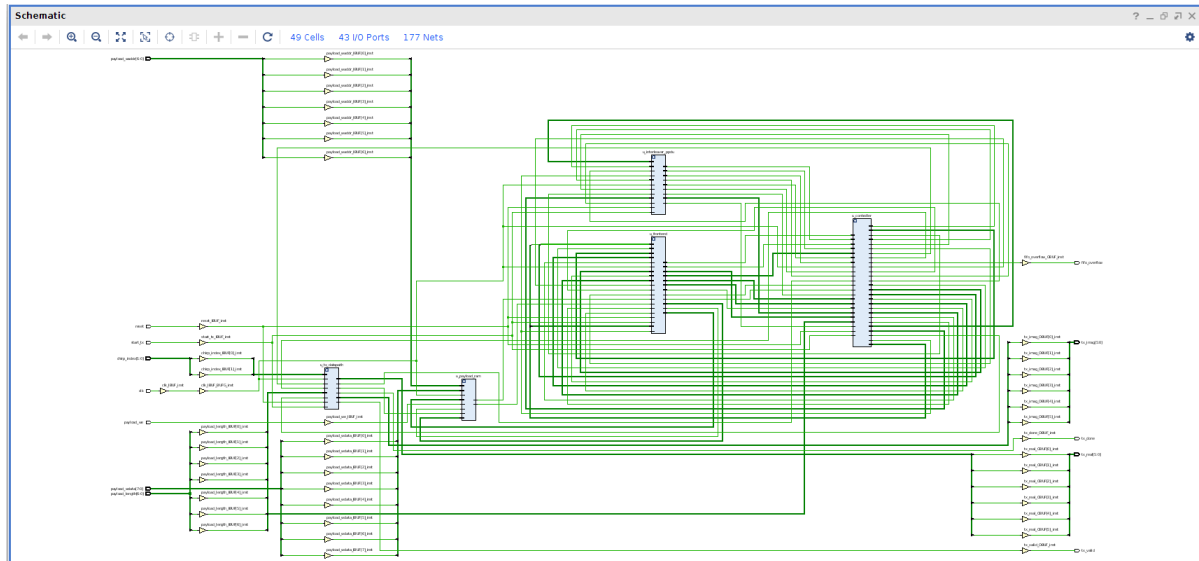
Tcl Console	Messages	Log	Reports	Design Runs	DRC	Methodology	Power	Timing	Utilization			
Intra-Clock Paths - sys_clk - Setup												
General Information	Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source	Destination
Timer Settings	Path 1	0.700	13	17	u_controller/..._reg_reg[1]/C	u_frontend/u...x_reg[10]/CE	8.720	4.152	4.568	10.0	sys_clk	
Design Timing Summary	Path 2	0.700	13	17	u_controller/..._reg_reg[1]/C	u_frontend/u...x_reg[11]/CE	8.720	4.152	4.568	10.0	sys_clk	
Clock Summary (1)	Path 3	0.700	13	17	u_controller/..._reg_reg[1]/C	u_frontend/u...x_reg[15]/CE	8.720	4.152	4.568	10.0	sys_clk	
Check Timing (0)	Path 4	0.700	13	17	u_controller/..._reg_reg[1]/C	u_frontend/u...ex_reg[8]/CE	8.720	4.152	4.568	10.0	sys_clk	
Intra-Clock Paths	Path 5	0.711	13	17	u_controller/..._reg_reg[1]/C	u_frontend/u...x_reg[14]/CE	8.708	4.152	4.556	10.0	sys_clk	
sys_clk	Path 6	0.711	13	17	u_controller/..._reg_reg[1]/C	u_frontend/u...ex_reg[2]/CE	8.708	4.152	4.556	10.0	sys_clk	
Setup 0.700 ns (10)	Path 7	0.711	13	17	u_controller/..._reg_reg[1]/C	u_frontend/u...ex_reg[3]/CE	8.708	4.152	4.556	10.0	sys_clk	
Hold 0.056 ns (10)	Path 8	0.711	13	17	u_controller/..._reg_reg[1]/C	u_frontend/u...ex_reg[7]/CE	8.708	4.152	4.556	10.0	sys_clk	
Pulse Width 3.750 ns (30)	Path 9	0.833	13	17	u_controller/..._reg_reg[1]/C	u_frontend/u...ex_reg[6]/CE	8.587	4.152	4.435	10.0	sys_clk	
Inter-Clock Paths	Path 10	0.833	13	17	u_controller/..._reg_reg[1]/C	u_frontend/u...ex_reg[9]/CE	8.587	4.152	4.435	10.0	sys_clk	
Other Path Groups												
Timing Summary - impl_8 (saved)												

8.2.3.2 Utilization report:

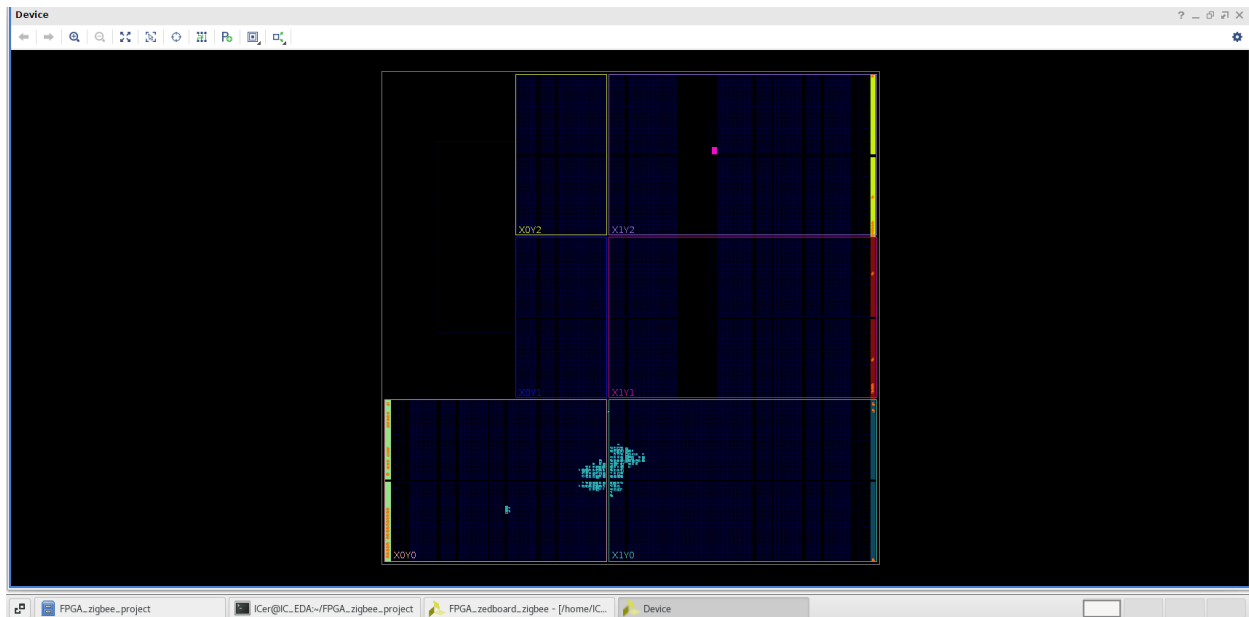
Utilization												
Hierarchy												
Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Block RAM Tile (140)	DSPs (220)	Bonded IPADs (2)	BUFIO (16)		
css_tx_top	290	300	11	126	274	16	300	1.5	43	1		
u_controller (controller)	97	102	8	49	81	16	0	0	0	0		
u_frontend (css_tx_frontend)	63	74	0	43	63	0	0	0	0	0		
u_mapper_j (symbol_mapper)	3	5	0	3	3	0	0	0	0	0		
u_mapper_q (symbol_mapper_2)	3	5	0	3	3	0	0	0	0	0		
u_s2p_j (serial_to_parallel)	4	9	0	3	4	0	0	0	0	0		
u_s2p_j (serial_to_parallel_3)	4	9	0	3	4	0	0	0	0	0		
u_zero_padding (zero_padding)	49	44	0	32	49	0	0	0	0	0		
u_interleaver_ppdu (interleaver_ppdu_top)	42	41	0	23	42	0	0	0	0	0		
u_interleaver_stage_j (interleaver_stage)	0	5	0	4	0	0	0	0	0	0		
u_interleaver_stage_q (interleaver_stage)	0	4	0	3	0	0	0	0	0	0		
u_ppdu_former (ppdu_former)	42	32	0	19	42	0	0	0	0	0		
u_p2s_j (parallel_to_serial)	15	10	0	9	15	0	0	0	0	0		
u_p2s_q (parallel_to_serial_1)	17	10	0	11	17	0	0	0	0	0		
u_payload_ram (payload_ram)	3	2	0	5	3	0	0.5	0	0	0		
u_tx_datapath (tx_datapath_top)	86	81	3	37	86	0	1	0	0	0		
u_css_symbol_generator (css_symbol_gen)	84	72	3	35	84	0	1	0	0	0		
u_dpdk_encoder (dpdk_encoder)	2	9	0	5	2	0	0	0	0	0		

8.2.4 Results:

8.2.4.1 Design after Synthesis:



8.2.4.2 Design after Implementation:



9 SUMMARY

This report details the design, implementation, and verification of an IEEE 802.15.4 Chirp Spread Spectrum (CSS) Physical Layer (PHY) transmitter. Developed for the ITI Summer Project by Team 2, the system processes raw payload bytes provided by the MAC layer into quantized real (Tx_real) and imaginary (Tx_imag) baseband complex I/Q output signals. The transmitter's key structural components and workflow include: System Controller & Frontend Architecture: Driven by a central Finite State Machine (FSM), payload data in a dedicated RAM is zero-padded, demultiplexed into In-phase (I) and Quadrature (Q) streams, and mapped using Walsh-Hadamard symbol mappers (handling both 1 Mbps and 250 kbps data rates). PPDU Assembly & Interleaving: Supports rate-dependent bit interleaving to protect against burst errors, integrates Synchronization Header (SHR) preamble and Start-of-Frame Delimiter (SFD) patterns, and serializes the streams into continuous chip flows. Modulation & Datapath: Serialized chips undergo QPSK phase mapping and multiplier-free DQPSK differential encoding before being modulated via raised-cosine windowed Chirp Sequential Keying (CSK) using ROM-based chirp generators and complex multipliers. Verification & Implementation: Includes fixed-point vs. floating-point Mean Squared Error (MSE) accuracy analysis, RTL behavioral simulation validation, and full physical implementation across ASIC (synthesis, floorplanning, power planning, CTS) and FPGA design flows.

10 REFERENCES

[1] IEEE 802.15.4 CSS PHY Transmitter Project Report (Team 2), Information Technology Institute (ITI) Summer Project, 2026.

[2] IEEE Standard for Low-Rate Wireless Networks, IEEE Std 802.15.4-2020 (Revision of IEEE Std 802.15.4-2015), pp. 1-800, May 2020.

[3] IEEE Standard for Information Technology - Telecommunications and Information Exchange Between Systems - Local and Metropolitan Area Networks - Specific Requirements - Part 15.4: Wireless Medium Access Control (MAC) and Physical Layer (PHY) Specifications for Low-Rate Wireless Personal Area Networks (WPANs) Amendment 1: Add Alternate PHYs, IEEE Std 802.15.4a-2007, pp. 1-225, Aug. 2007.

[4] Avnet Electronics Marketing, ZedBoard Board User's Manual, Version 2.2, Avnet, Inc., Phoenix, AZ, USA, Jan. 2014.

[5] P. P. Chu, FPGA Prototyping by VHDL Examples: Xilinx Spartan-3 Version, Hoboken, NJ, USA: Wiley-Interscience, 2008.

[6] P. P. Chu, RTL Hardware Design Using VHDL: Coding for Efficiency, Portability, and Scalability, Hoboken, NJ, USA: Wiley-Interscience, 2006.

[7] Xilinx Inc., Vivado Design Suite User Guide: Implementation, UG904 (v2022.2), Xilinx, Inc., San Jose, CA, USA, Nov. 2022.

[8] Xilinx Inc., 7 Series FPGAs Configurable Logic Block User Guide, UG474 (v1.8), Xilinx, Inc., San Jose, CA, USA, Sep. 2016.

11 APPENDIX

11.1 Testbenches:

```
`timescale 1ns / 1ps

module chirp_rom_tb;

    // -----
    // Clock and Control Signals
    // -----
    reg        clk;
    reg [1:0]  chirp_index;
    reg [1:0]  subchirp_sel;
    reg [5:0]  sample_addr;

    wire [4:0] rom_real;
    wire [4:0] rom_imag;

    // Error and Test Counters for Self-Checking
    integer error_count = 0;
    integer test_count  = 0;

    // -----
    // Instantiate Unit Under Test (UUT)
    // -----
    chirp_rom uut (
```

```

        .clk          (clk),
        .chirp_index  (chirp_index),
        .subchirp_sel (subchirp_sel),
        .sample_addr  (sample_addr),
        .rom_real      (rom_real),
        .rom_imag      (rom_imag)
    );

    // -----
    // Clock Generation (100 MHz Simulation Clock)
    // -----
    initial clk = 0;
    always #5 clk = ~clk; // 10 ns clock period

    // -----
    // Test Task for Self-Checking Range and Output
    // -----
    task check_rom_output;
        input [1:0] exp_m;
        input [1:0] exp_k;
        input [5:0] exp_n;
        begin
            @(posedge clk); #1; // Wait 1ns after clock edge for registered
            output stability
            test_count = test_count + 1;

            if (exp_n > 6'd37) begin
                // Check Unused Address Behavior (Should return 0)
                if (rom_real !== 5'sd0 || rom_imag !== 5'sd0) begin
                    $display("[FAIL] Unused Addr m=%0d k=%0d n=%0d | Expected:
(0,0) Got: (%0d,%0d)",
                                exp_m, exp_k, exp_n, $signed(rom_real),
                                $signed(rom_imag));
                    error_count = error_count + 1;
                end
            end else begin
                // Range Check for R=5 (-16 to +15)
                if ($signed(rom_real) < -16 || $signed(rom_real) > 15 ||
                    $signed(rom_imag) < -16 || $signed(rom_imag) > 15) begin
                    $display("[FAIL] Out of Range m=%0d k=%0d n=%0d | Got:
Real=%0d Imag=%0d",
                                exp_m, exp_k, exp_n, $signed(rom_real),
                                $signed(rom_imag));
                    error_count = error_count + 1;
                end
            end
        end
    endtask

```

```

        end
    end
endtask

// -----
// Main Stimulus Sequence
// -----
integer m, k, n;

initial begin
    // Initialize Inputs
    chirp_index = 0;
    subchirp_sel = 0;
    sample_addr = 0;

    $display("=====");
    $display("Starting Self-Checking Testbench for chirp_rom (R=5)");
    $display("=====");

    // Wait for clock stabilization
    #20;

    // -----
    // Test Case 1: Boundary & Full Valid Address Sweep (m=0..3, k=0..3,
n=0..37)
    // -----
    $display("[INFO] Running Valid Address Sweep and Range Checks...");
    for (m = 0; m < 4; m = m + 1) begin
        for (k = 0; k < 4; k = k + 1) begin
            for (n = 0; n <= 37; n = n + 1) begin
                chirp_index = m[1:0];
                subchirp_sel = k[1:0];
                sample_addr = n[5:0];
                check_rom_output(m[1:0], k[1:0], n[5:0]);
            end
        end
    end

    // -----
    // Test Case 2: Unused Address Sweep (n = 38..63)
    // -----
    $display("[INFO] Running Unused Address Checks (n = 38..63)...");
    for (m = 0; m < 4; m = m + 3) begin // Test m=0 and m=3 boundaries
        for (k = 0; k < 4; k = k + 3) begin // Test k=0 and k=3 boundaries
            for (n = 38; n < 64; n = n + 1) begin

```

```

        chirp_index = m[1:0];
        subchirp_sel = k[1:0];
        sample_addr = n[5:0];
        check_rom_output(m[1:0], k[1:0], n[5:0]);
    end
end
end

// -----
// Final Test Summary Report
// -----
#20;
$display("=====");
$display("TEST SUMMARY:");
$display("Total Assertions Executed: %0d", test_count);
if (error_count == 0) begin
    $display("RESULT: ALL TESTS PASSED SUCCESSFULLY! [PASS]");
end else begin
    $display("RESULT: %0d TESTS FAILED! [FAIL]", error_count);
end
$display("=====");

$finish;
end

endmodule

```

```

// =====
// csk_generator_tb.v (v2 -- rewritten to avoid an Icarus Verilog
// task-scheduling quirk found while building this testbench)
// =====
// Self-checking testbench for csk_generator (RTL v2, corrected
// trailing-gap behavior). Covers the "Basic tests" from the Task 3
// module spec, plus a dedicated regression test for the trailing-gap
// fix (Test 3 below).
//
// *** A note on testbench structure ***
// An earlier draft of this testbench used Verilog tasks containing
// `@(posedge clk)` (a `do_reset` task and a `run_packet` task). That
// version hung indefinitely under Icarus Verilog: confirmed, via
// multiple isolated experiments checking real DUT output ports (not
// just internal signals), that the RTL itself is correct -- a fully
// inline testbench with zero task calls drives the DUT perfectly. The
// hang was traced specifically to Icarus Verilog's handling of tasks

```

```

// that contain blocking timing controls: once such a task is called,
// a LATER top-level `@(posedge clk)` in the same process silently stops
// synchronizing with the DUT. This testbench avoids that pattern
// entirely: every `@(posedge clk)` lives either directly in the
// top-level initial block, or inside a `fork ... join` block (which
// was verified safe across repeated use). No task in this file
// contains a timing control.
//
// Simulate with:
// iverilog -o sim csk_generator.v csk_generator_tb.v && vvp sim
// =====

`timescale 1ns/1ps

module csk_generator_tb;

    localparam NUM_SYMBOLS_WIDTH = 12;

    reg                clk;
    reg                reset;
    reg                start;
    reg  [1:0]          chirp_index;
    reg  [NUM_SYMBOLS_WIDTH-1:0] num_symbols;
    reg                dqpsk_valid;
    reg                dqpsk_sign_real;
    reg                dqpsk_sign_imag;

    wire [1:0] subchirp_sel;
    wire [5:0] sample_addr;
    wire      sign_real_out;
    wire      sign_imag_out;
    wire      sample_valid;
    wire      next_symbol_req;
    wire      done;

    integer errors;
    integer test_num;

    // -----
    // DUT instantiation
    // -----
    csk_generator #(.NUM_SYMBOLS_WIDTH(NUM_SYMBOLS_WIDTH)) dut (
        .clk          (clk),
        .reset         (reset),
        .start         (start),

```

```

        .chirp_index      (chirp_index),
        .num_symbols      (num_symbols),
        .dqpsk_valid      (dqpsk_valid),
        .dqpsk_sign_real  (dqpsk_sign_real),
        .dqpsk_sign_imag  (dqpsk_sign_imag),
        .subchirp_sel     (subchirp_sel),
        .sample_addr      (sample_addr),
        .sign_real_out     (sign_real_out),
        .sign_imag_out     (sign_imag_out),
        .sample_valid      (sample_valid),
        .next_symbol_req   (next_symbol_req),
        .done              (done)
    );

    // -----
    // Clock: 10ns period
    // -----
    always #5 clk = ~clk;

    // -----
    // Symbol responder -- a plain always block (no tasks), confirmed
    // safe. Responds to next_symbol_req after `response_delay` extra
    // cycles, with alternating dummy sign values.
    // -----
    integer response_delay;
    reg    pending_req;
    integer delay_counter;
    reg    next_sign_real;
    reg    next_sign_imag;

    initial begin
        pending_req    = 0;
        dqpsk_valid    = 0;
        dqpsk_sign_real = 0;
        dqpsk_sign_imag = 0;
        next_sign_real  = 0;
        next_sign_imag  = 1;
        response_delay  = 0;
    end

    always @(posedge clk) begin
        dqpsk_valid <= 0;

        if (next_symbol_req && !pending_req) begin
            pending_req <= 1;

```

```

        delay_counter <= response_delay;
    end

    if (pending_req) begin
        if (delay_counter == 0) begin
            dqpsk_valid      <= 1;
            dqpsk_sign_real <= next_sign_real;
            dqpsk_sign_imag <= next_sign_imag;
            next_sign_real  <= ~next_sign_real;
            next_sign_imag  <= ~next_sign_imag;
            pending_req      <= 0;
        end else begin
            delay_counter <= delay_counter - 1;
        end
    end
end

// -----
// Shared accumulators, reset and populated inline per test
// -----
integer vc, gc, rq; // valid cycles, gap cycles, next_symbol_req pulses

// =====
// Test sequence -- everything below is either top-level inline
// code or a fork/join block. No task contains a timing control.
// =====
initial begin
    clk      = 0;
    reset    = 0;
    start    = 0;
    chirp_index = 0;
    num_symbols = 0;
    errors    = 0;
    test_num  = 0;

    // =====
    // TEST 1: single chirp sequence (4 symbols), fast responder.
    // Checks the subchirp_sel/sample_addr walk order directly
    // against an independent model computed inline.
    // =====
    test_num = 1;
    response_delay = 0;

    reset = 1; start = 0;
    @(posedge clk); @(posedge clk);

```

```

    reset = 0;
    @(posedge clk);

    start = 1; chirp_index = 2'd0; num_symbols = 12'd4;
    @(posedge clk);
    start = 0;

    fork
        begin : test1_addr_walk
            integer exp_k, exp_addr;
            exp_k = 0; exp_addr = 0;
            while (done !== 1'b1) begin
                @(posedge clk);
                if (sample_valid) begin
                    if (subchirp_sel !== exp_k || sample_addr !== exp_addr)
begin
                        $display("[TEST %0d] FAIL: expected (k=%0d,addr=%0d)
got (k=%0d,addr=%0d) at t=%0t",
                                test_num, exp_k, exp_addr, subchirp_sel,
                                sample_addr, $time);
                        errors = errors + 1;
                    end
                    if (exp_addr == 37) begin
                        exp_addr = 0;
                        exp_k = exp_k + 1;
                    end else begin
                        exp_addr = exp_addr + 1;
                    end
                end
            end
        end
    join
        $display("[TEST %0d] Address-walk check for one chirp sequence
complete.", test_num);

    // =====
    // TEST 2: gap length + parity, all 4 chirp indices, 2 sequences
    // each (num_symbols=8), multi-cycle responder (response_delay=1).
    // =====
    test_num = 2;
    response_delay = 1;

    begin : test2_gap_check
        integer ci;
        integer expected_even, expected_odd;

```

```

    for (ci = 0; ci < 4; ci = ci + 1) begin
        reset = 1; start = 0;
        @(posedge clk); @(posedge clk);
        reset = 0;
        @(posedge clk);

        case (ci)
            0: begin expected_even = 10; expected_odd = 70; end
            1: begin expected_even = 20; expected_odd = 60; end
            2: begin expected_even = 30; expected_odd = 50; end
            3: begin expected_even = 40; expected_odd = 40; end
        endcase

        vc = 0; gc = 0; rq = 0;
        start = 1; chirp_index = ci[1:0]; num_symbols = 12'd8;
        @(posedge clk);
        start = 0;

        fork
            begin : test2_wait
                while (done !== 1'b1) begin
                    @(posedge clk);
                    if (sample_valid)          vc = vc + 1;
                    if (dut.state == dut.S_GAP) gc = gc + 1;
                    if (next_symbol_req)       rq = rq + 1;
                end
            end
        join

        if (gc !== (expected_even + expected_odd)) begin
            $display("[TEST %0d] FAIL: chirp_index=%0d total gap cycles =
%0d, expected %0d (even=%0d + odd=%0d)",
                    test_num, ci, gc, expected_even+expected_odd,
expected_even, expected_odd);
            errors = errors + 1;
        end else begin
            $display("[TEST %0d] PASS: chirp_index=%0d total gap cycles =
%0d as expected", test_num, ci, gc);
        end
    end
end

// =====
// TEST 3 (REGRESSION): full 48-sequence packet, chirp_index=0

```

```

// (m=1), num_symbols=192. Must total exactly the validated
// MATLAB reference length of 9216 samples (streaming + gap).
// This is the test that would have caught the trailing-gap bug.
// =====
test_num = 3;
response_delay = 0;

reset = 1; start = 0;
@(posedge clk); @(posedge clk);
reset = 0;
@(posedge clk);

vc = 0; gc = 0; rq = 0;
start = 1; chirp_index = 2'd0; num_symbols = 12'd192;
@(posedge clk);
start = 0;

fork
    begin : test3_wait
        while (done != 1'b1) begin
            @(posedge clk);
            if (sample_valid)          vc = vc + 1;
            if (dut.state == dut.S_GAP) gc = gc + 1;
            if (next_symbol_req)       rq = rq + 1;
        end
    end
join

if (vc != 7296) begin
    $display("[TEST %0d] FAIL: valid (streaming) cycles = %0d, expected
7296 (152*48)", test_num, vc);
    errors = errors + 1;
end else begin
    $display("[TEST %0d] PASS: valid streaming cycles = 7296 as
expected", test_num);
end

if (gc != 1920) begin
    $display("[TEST %0d] FAIL: total gap cycles = %0d, expected 1920
(24*10 + 24*70)", test_num, gc);
    errors = errors + 1;
end else begin
    $display("[TEST %0d] PASS: total gap cycles = 1920 as expected
(trailing gap included)", test_num);
end
end

```

```

        if ((vc + gc) != 9216) begin
            $display("[TEST %0d] FAIL: valid+gap total = %0d, expected 9216
(matches validated TxchirpSequences length)",
                test_num, vc+gc);
            errors = errors + 1;
        end else begin
            $display("[TEST %0d] PASS: valid+gap total = 9216, matches the
validated MATLAB reference length", test_num);
        end

        if (rq != 192) begin
            $display("[TEST %0d] FAIL: next_symbol_req pulse count = %0d,
expected 192 (once per subchirp)", test_num, rq);
            errors = errors + 1;
        end else begin
            $display("[TEST %0d] PASS: next_symbol_req pulsed exactly once per
subchirp (192 times)", test_num);
        end

        // =====
        // TEST 4: empty packet edge case (num_symbols = 0). Should go
        // straight to done: zero streaming, zero gap, zero requests.
        // =====
        test_num = 4;

        reset = 1; start = 0;
        @(posedge clk); @(posedge clk);
        reset = 0;
        @(posedge clk);

        vc = 0; gc = 0; rq = 0;
        start = 1; chirp_index = 2'd0; num_symbols = 12'd0;
        @(posedge clk);
        start = 0;

        fork
            begin : test4_wait
                while (done != 1'b1) begin
                    @(posedge clk);
                    if (sample_valid)          vc = vc + 1;
                    if (dut.state == dut.S_GAP) gc = gc + 1;
                    if (next_symbol_req)       rq = rq + 1;
                end
            end
        end

```

```

join

    if (vc != 0 || gc != 0 || rq != 0) begin
        $display("[TEST %0d] FAIL: empty packet produced vc=%0d gc=%0d
rq=%0d, expected all zero",
            test_num, vc, gc, rq);
        errors = errors + 1;
    end else begin
        $display("[TEST %0d] PASS: empty packet (num_symbols=0) produced no
streaming, no gap, no requests", test_num);
    end

    // =====
    // TEST 5: done timing -- must deassert exactly one cycle after
    // pulsing (i.e. it really is a one-cycle pulse, not held high).
    // =====
    test_num = 5;

    reset = 1; start = 0;
    @(posedge clk); @(posedge clk);
    reset = 0;
    @(posedge clk);

    start = 1; chirp_index = 2'd0; num_symbols = 12'd4;
    @(posedge clk);
    start = 0;

    fork
        begin : test5_wait
            while (done != 1'b1) @(posedge clk);
        end
    join

    @(posedge clk);
    if (done != 1'b0) begin
        $display("[TEST %0d] FAIL: done did not deassert the cycle after
pulsing", test_num);
        errors = errors + 1;
    end else begin
        $display("[TEST %0d] PASS: done pulses for exactly one cycle",
test_num);
    end

    // =====
    // Summary

```

```

// =====
@(posedge clk);
if (errors == 0) begin
    $display("\n=====");
    $display(" ALL TESTS PASSED (0 errors across 5 test groups)");
    $display("=====\\n");
end else begin
    $display("\n=====");
    $display(" TESTS FAILED: %0d error(s) found", errors);
    $display("=====\\n");
end

$finish;
end

// -----
// Safety timeout, in case a real hang occurs
// -----
initial begin
    #2000000;
    $display("TIMEOUT: simulation did not finish in time -- likely DUT
hang");
    $finish;
end

endmodule

```

```

// =====
// tb_css_symbol_generator.sv
//
// Integration testbench for the Task 3 top-level wrapper: chirp_rom +
// csk_generator + complex_multiplier, wired together as
// css_symbol_generator.v.
//
// This is deliberately NOT just a repeat of the unit tests already run
// against csk_generator on its own. Those already proved the sequencing
// FSM is correct in isolation. What ONLY an integration test can catch:
// - is the 1-cycle ROM latency actually compensated correctly (Stage A)
// - is the complex multiply correctly fed the RIGHT rom sample paired
//   with the RIGHT sign, at the RIGHT cycle (Stage B)
// - does tx_valid land on the SAME cycle as the tx_real/tx_imag they
//   describe
// - does the wrapper's own chirp_index latch actually protect against
//   the caller changing chirp_index mid-packet (TC5 -- this is a

```

```

//      regression test for the exact bug class the wrapper was written
//      to prevent)
//
// How the data-correctness check works (Stage A / Stage B):
// This testbench does NOT hard-code any expected chirp sample values,
// and does NOT care what is actually inside chirp_rom_real.mem /
// chirp_rom_imag.mem -- real chirp data, placeholder data, anything.
// Instead it reads the DUT's OWN loaded ROM contents via a hierarchical
// reference (uut.u_chirp_rom.rom_real_mem / rom_imag_mem) and
// independently recomputes what complex_multiplier should have produced,
// using the exact same conditional-negate formula, then compares that
// against the DUT's real tx_real/tx_imag/tx_valid, cycle-exact. This
// means the same testbench is valid whether it's run against dummy
// memory files (as it was written and verified here) or against the
// real, MATLAB-exported chirp ROM contents -- swap the .mem files and
// rerun.
//
// Pipeline timing (see css_symbol_generator.v's own header comment for
// the full derivation):
//   cycle N    : address (chirp_index_latched, subchirp_sel, sample_addr)
//                 issued, sample_valid asserted combinationally
//   cycle N+1  : rom_real/rom_imag now respond to the cycle-N address;
//                 sign_real_out/sign_imag_out ALSO now reflect cycle N's
//                 sign (already registered inside csk_generator); the
//                 wrapper's sample_valid_d1 now equals cycle N's
//                 sample_valid
//   cycle N+2  : tx_real/tx_imag/tx_valid now reflect complex_multiplier's
//                 registered response to cycle N+1's inputs
// =====
`timescale 1ns/1ps

module tb_css_symbol_generator;

    localparam NUM_SYMBOLS_WIDTH = 12;
    localparam ROM_WIDTH         = 5;
    localparam OUT_WIDTH         = 6;

    // -----
    // Clock / reset
    // -----
    logic clk = 0;
    always #5 clk = ~clk;
    logic reset;

```

```

// -----
// DUT ports
// -----
logic      start;
logic [1:0] chirp_index;
logic [NUM_SYMBOLS_WIDTH-1:0] num_symbols;

logic      dqpsk_valid;
logic      dqpsk_sign_real;
logic      dqpsk_sign_imag;

logic      next_symbol_req;
logic      done;

logic signed [OUT_WIDTH-1:0] tx_real;
logic signed [OUT_WIDTH-1:0] tx_imag;
logic      tx_valid;

css_symbol_generator #(
    .NUM_SYMBOLS_WIDTH (NUM_SYMBOLS_WIDTH),
    .ROM_WIDTH          (ROM_WIDTH),
    .OUT_WIDTH          (OUT_WIDTH)
) uut (
    .clk                (clk),
    .reset              (reset),
    .start              (start),
    .chirp_index        (chirp_index),
    .num_symbols         (num_symbols),
    .dqpsk_valid        (dqpsk_valid),
    .dqpsk_sign_real    (dqpsk_sign_real),
    .dqpsk_sign_imag    (dqpsk_sign_imag),
    .next_symbol_req    (next_symbol_req),
    .done               (done),
    .tx_real            (tx_real),
    .tx_imag            (tx_imag),
    .tx_valid           (tx_valid)
);

// -----
// Bookkeeping
// -----
int unsigned error_count = 0;
int unsigned check_count = 0;

task automatic check(input bit cond, input string msg);

```

```

        check_count++;
        if (!cond) begin
            error_count++;
            $display("[%0t] ERROR: %s", $time, msg);
        end
    endtask

    // -----
    // DQPSK symbol source model (same style used for csk_generator's own
    // unit test): responds to next_symbol_req after a configurable
    // random delay, with signs pulled from a queue or random if empty.
    // -----
    bit sign_queue_sr[$];
    bit sign_queue_si[$];

    int resp_delay_min = 0;
    int resp_delay_max = 0;
    int delay;
    bit s_sr, s_si;

    initial begin
        dqpsk_valid = 1'b0; dqpsk_sign_real = 1'b0; dqpsk_sign_imag = 1'b0;
        forever begin
            @(posedge clk);
            if (next_symbol_req) begin
                delay = (resp_delay_max > resp_delay_min) ?
                    $urandom_range(resp_delay_max, resp_delay_min) :
resp_delay_min;
                repeat (delay) @(posedge clk);
                if (sign_queue_sr.size() != 0) begin
                    s_sr = sign_queue_sr.pop_front();
                    s_si = sign_queue_si.pop_front();
                end else begin
                    s_sr = $urandom_range(1,0);
                    s_si = $urandom_range(1,0);
                end
                dqpsk_sign_real <= s_sr;
                dqpsk_sign_imag <= s_si;
                dqpsk_valid <= 1'b1;
                @(posedge clk);
                dqpsk_valid <= 1'b0;
            end
        end
    end
end

```

```

// =====
// STAGE A -- one cycle after an address is issued, check the ROM
// response is bit-exact against the DUT's own loaded memory, and
// that sample_valid_d1 correctly reflects last cycle's sample_valid.
// =====
reg [1:0] addrA_chirp_index;
reg [1:0] addrA_subchirp_sel;
reg [5:0] addrA_sample_addr;
reg      addrA_valid;

always @(posedge clk) begin
    addrA_chirp_index  <= uut.chirp_index_latched;
    addrA_subchirp_sel <= uut.subchirp_sel;
    addrA_sample_addr  <= uut.sample_addr;
    addrA_valid        <= uut.sample_valid;
end

// =====
// STAGE B -- one more cycle later, snapshot the (now ROM-correct)
// rom_real/rom_imag + sign bits + delayed valid, exactly as
// complex_multiplier itself would have latched them.
// =====
reg signed [ROM_WIDTH-1:0] mulB_rom_real;
reg signed [ROM_WIDTH-1:0] mulB_rom_imag;
reg                      mulB_sign_real;
reg                      mulB_sign_imag;
reg                      mulB_valid;

always @(posedge clk) begin
    mulB_rom_real  <= uut.rom_real;
    mulB_rom_imag  <= uut.rom_imag;
    mulB_sign_real <= uut.sign_real_out;
    mulB_sign_imag <= uut.sign_imag_out;
    mulB_valid     <= uut.sample_valid_d1;
end

// =====
// Continuous checks -- sampled at negedge so all posedge updates
// (DUT's and this testbench's own shadow registers) have settled.
// =====
logic signed [ROM_WIDTH-1:0] exp_rom_real, exp_rom_imag;
logic signed [OUT_WIDTH-1:0] c_ext, d_ext;
logic signed [OUT_WIDTH-1:0] exp_tx_real, exp_tx_imag;
reg      chirp_index_at_start_valid;
int unsigned tx_valid_count = 0;

```

```

always @(negedge clk) begin
    if (!reset) begin

        // ---- Stage A checks: ROM correctness + valid-delay alignment ----
        if (addrA_valid) begin
            exp_rom_real = uut.u_chirp_rom.rom_real_mem[{addrA_chirp_index,
            addrA_subchirp_sel, addrA_sample_addr}];
            exp_rom_imag = uut.u_chirp_rom.rom_imag_mem[{addrA_chirp_index,
            addrA_subchirp_sel, addrA_sample_addr}];
            check(uut.rom_real === exp_rom_real,
                $sformatf("rom_real mismatch at addr(ci=%0d,k=%0d,a=%0d):
got %0d expected %0d",
                        addrA_chirp_index, addrA_subchirp_sel,
            addrA_sample_addr, uut.rom_real, exp_rom_real));
            check(uut.rom_imag === exp_rom_imag,
                $sformatf("rom_imag mismatch at addr(ci=%0d,k=%0d,a=%0d):
got %0d expected %0d",
                        addrA_chirp_index, addrA_subchirp_sel,
            addrA_sample_addr, uut.rom_imag, exp_rom_imag));
        end
        check(uut.sample_valid_d1 == addrA_valid,
            "sample_valid_d1 does not equal sample_valid delayed by exactly
1 cycle");

        // ---- Stage B checks: complex multiply correctness ----
        if (mulB_valid) begin
            c_ext = {{(OUT_WIDTH-ROM_WIDTH){mulB_rom_real[ROM_WIDTH-1]}},
            mulB_rom_real};
            d_ext = {{(OUT_WIDTH-ROM_WIDTH){mulB_rom_imag[ROM_WIDTH-1]}},
            mulB_rom_imag};
            exp_tx_real = (mulB_sign_real ? -c_ext : c_ext) - (mulB_sign_imag
            ? -d_ext : d_ext);
            exp_tx_imag = (mulB_sign_real ? -d_ext : d_ext) + (mulB_sign_imag
            ? -c_ext : c_ext);
            check(tx_real === exp_tx_real,
                $sformatf("tx_real mismatch: got %0d expected %0d
(rom_real=%0d rom_imag=%0d sr=%0b si=%0b)",
                        tx_real, exp_tx_real, mulB_rom_real,
            mulB_rom_imag, mulB_sign_real, mulB_sign_imag));
            check(tx_imag === exp_tx_imag,
                $sformatf("tx_imag mismatch: got %0d expected %0d
(rom_real=%0d rom_imag=%0d sr=%0b si=%0b)",
                        tx_imag, exp_tx_imag, mulB_rom_real,
            mulB_rom_imag, mulB_sign_real, mulB_sign_imag));
        end
    end
end

```

```

        end
        check(tx_valid == mulB_valid,
            "tx_valid does not equal sample_valid delayed by exactly 2
cycles total");

        if (tx_valid) tx_valid_count++;

        // ---- chirp_index latch regression check (TC5 relies on this) ----
        if (chirp_index_at_start_valid)
            check(uut.chirp_index_latched == chirp_index_at_start,
                $sformatf("chirp_index_latched drifted mid-packet: now %0d,
expected %0d (latched at start)",
                        uut.chirp_index_latched, chirp_index_at_start));
        end
    end

    // Tracks what chirp_index SHOULD stay latched at, from the testbench's
    // own point of view -- independent of uut.chirp_index_latched, so the
    // check above is a genuine external check, not a circular one.
    reg [1:0] chirp_index_at_start;

    // -----
    // Reset
    // -----
    task automatic do_reset();
        reset = 1'b1; start = 1'b0; chirp_index = 2'd0; num_symbols = '0;
        sign_queue_sr.delete(); sign_queue_si.delete();
        chirp_index_at_start_valid = 1'b0;
        repeat (3) @(posedge clk);
        reset <= 1'b0;
        @(posedge clk);
    endtask

    // -----
    // Drive one packet, wait for done, return the observed tx_valid count.
    // `drift_index`, if >= 0, is written to the LIVE chirp_index input a
    // few cycles into the packet -- TC5 uses this to prove the wrapper's
    // latch actually holds against exactly this kind of interference.
    // -----
    task automatic run_packet(
        input [1:0] idx,
        input [NUM_SYMBOLS_WIDTH-1:0] n_syms,
        input int drift_index, // -1 = no drift
        output int unsigned tx_valid_count_out
    )

```

```

);
    int unsigned timeout_cycles;

    tx_valid_count = 0;

    @(posedge clk);
    chirp_index <= idx;
    num_symbols <= n_syms;
    start <= 1'b1;
    chirp_index_at_start = idx;
    @(posedge clk); // <-- DUT samples start=1 HERE and updates
                    //      chirp_index_latched on THIS edge

    start <= 1'b0;
    chirp_index_at_start_valid = 1'b1; // only safe to start checking now,
                                      // one cycle later than before

    if (drift_index >= 0) begin
        repeat (5) @(posedge clk);
        chirp_index <= drift_index[1:0]; // interference: should have NO
effect on this packet
    end

    timeout_cycles = 0;
    while (!done && timeout_cycles < 200000) begin
        @(posedge clk);
        timeout_cycles++;
    end
    check(timeout_cycles < 200000,
        $sformatf("TIMEOUT waiting for done (chirp_index=%0d,
num_symbols=%0d)", idx, n_syms));

    repeat (4) @(posedge clk); // let the 2-cycle output pipeline fully drain
past done
    chirp_index_at_start_valid = 1'b0;

    tx_valid_count_out = tx_valid_count;
endtask

// =====
// TEST SEQUENCE
// =====
int unsigned tvc;

initial begin
    chirp_index_at_start_valid = 1'b0;

```

```

$display("=====");
$display(" css_symbol_generator integration testbench starting");
$display("=====");

// ----- TC1: reset -----
$display("\n-- TC1: reset behavior --");
reset = 1'b1; start = 1'b0; chirp_index = 2'd0; num_symbols = '0;
repeat (3) @(posedge clk);
check(tx_valid == 1'b0, "tx_valid not 0 immediately after reset");
check(done      == 1'b0, "done not 0 immediately after reset");
do_reset();

// ----- TC2: empty packet -----
$display("\n-- TC2: empty packet (num_symbols == 0) --");
run_packet(2'd0, '0, -1, tvc);
check(tvc == 0, $sformatf("empty packet produced %0d tx_valid samples,
expected 0", tvc));
do_reset();

// ----- TC3: single sequence, all chirp_index values -----
$display("\n-- TC3: single chirp sequence (num_symbols=4), all
chirp_index values --");
for (int idx = 0; idx < 4; idx++) begin
    run_packet(idx[1:0], 12'd4, -1, tvc);
    check(tvc == 4*38, $sformatf("idx=%0d: expected %0d tx_valid samples,
got %0d", idx, 4*38, tvc));
    do_reset();
end

// ----- TC4: multi-sequence packet, all chirp_index values -----
$display("\n-- TC4: multi-sequence packet (3 sequences = 12 symbols), all
chirp_index values --");
for (int idx = 0; idx < 4; idx++) begin
    run_packet(idx[1:0], 12'd12, -1, tvc);
    check(tvc == 12*38, $sformatf("idx=%0d: expected %0d tx_valid
samples, got %0d", idx, 12*38, tvc));
    do_reset();
end

// ----- TC5: chirp_index drift mid-packet (wrapper-latch
regression) -----

```

```

        $display("\n-- TC5: chirp_index changes mid-packet -- must NOT affect
this packet's ROM data --");
        run_packet(2'd1, 12'd8, 3 /* drift to index 3 mid-packet */, tvc);
        check(tvc == 8*38, $sformatf("TC5: expected %0d tx_valid samples, got
%0d", 8*38, tvc));
        do_reset();

        // ----- TC6: variable DQPSK response latency -----
        $display("\n-- TC6: variable DQPSK response latency (0-4 cycle stalls) --
");
        resp_delay_min = 0; resp_delay_max = 4;
        run_packet(2'd2, 12'd8, -1, tvc);
        check(tvc == 8*38, $sformatf("TC6: expected %0d tx_valid samples, got
%0d", 8*38, tvc));
        resp_delay_min = 0; resp_delay_max = 0;
        do_reset();

        // ----- TC7: back-to-back packets -----
        $display("\n-- TC7: back-to-back packets, different
chirp_index/num_symbols each time --");
        run_packet(2'd0, 12'd4, -1, tvc);
        check(tvc == 4*38, "TC7 (packet 1): unexpected tx_valid count");
        run_packet(2'd3, 12'd8, -1, tvc);
        check(tvc == 8*38, "TC7 (packet 2): unexpected tx_valid count -- possible
stale state from packet 1");
        do_reset();

        // ----- Summary -----
        $display("\n=====");
        if (error_count == 0)
            $display(" ALL TESTS PASSED  (%0d checks, 0 errors)", check_count);
        else
            $display(" TESTS FAILED   (%0d checks, %0d errors)", check_count,
error_count);
        $display("=====");

        $finish;
    end

    // -----
    // Safety watchdog
    // -----
    initial begin
        #5_000_000;
        $display("GLOBAL WATCHDOG TIMEOUT -- simulation did not finish in time");
    end

```

```

        $finish;
    end

endmodule

```

```

`timescale 1ns/1ps

module tb_chirp_rom;

    reg [7:0] addr;
    wire signed [5:0] chirp_real, chirp_imag;
    integer errors;

    chirp_rom #( .SAMPLES(152), .MEMFILE("vectors/chirp_m1.mem") )
        dut ( .addr(addr), .chirp_real(chirp_real), .chirp_imag(chirp_imag) );

    task check;
        input [7:0] a;
        input signed [5:0] exp_r, exp_i;
        begin
            addr = a; #1;
            if (chirp_real != exp_r || chirp_imag != exp_i) begin
                $display("FAIL addr=%0d real=%0d(exp %0d) imag=%0d(exp %0d)",
                    a, chirp_real, exp_r, chirp_imag, exp_i);
                errors = errors + 1;
            end
        end
    endtask

    initial begin
        errors = 0;
        // spot checks against chirp_m1.mem (== project's chirpSequence.txt,
        chirpIndex=1),
        // 6-bit two's complement: real then imag, 152 samples each
        check(8'd0, 6'b000000, 6'b000000); // 0 , 0
        check(8'd151, 6'b100001, 6'b000000); // -31 , 0 (last sample of the
        sequence)
        check(8'd75, 6'b000000, 6'b100001); // 0 , -31 (subchirp-1/subchirp-
        2 boundary area)

        if (errors == 0)
            $display("PASS: tb_chirp_rom -- ROM contents match chirpSequence.txt
            (chirpIndex=1)");
        else

```

```

        $display("FAIL: tb_chirp_rom -- %0d mismatches", errors);
    $finish;
end

endmodule

```

```

`timescale 1ns/1ps

module tb_css_modulator;

    reg signed [1:0] s_real, s_imag;
    reg signed [5:0] chirp_real, chirp_imag;
    wire signed [7:0] tx_real, tx_imag;
    integer errors;

    css_modulator dut (
        .s_real(s_real), .s_imag(s_imag),
        .chirp_real(chirp_real), .chirp_imag(chirp_imag),
        .tx_real(tx_real), .tx_imag(tx_imag)
    );

    task check;
        input signed [1:0] sr, si;
        input signed [5:0] cr, ci;
        input signed [7:0] exp_r, exp_i;
        begin
            s_real = sr; s_imag = si; chirp_real = cr; chirp_imag = ci; #1;
            if (tx_real != exp_r || tx_imag != exp_i) begin
                $display("FAIL S=(%0d,%0d) C=(%0d,%0d) -> Tx=(%0d,%0d) expected
(%0d,%0d)",
                    sr, si, cr, ci, tx_real, tx_imag, exp_r, exp_i);
                errors = errors + 1;
            end
        end
    endtask

    initial begin
        errors = 0;
        // (Sr+jSi)*(Cr+jCi) = (Sr*Cr - Si*Ci) + j(Sr*Ci + Si*Cr)
        check(2'sd1, 2'sd1, 6'sd5, -6'sd3, 8'sd8, 8'sd2); // 5+3 , -3+5
        check(-2'sd1, 2'sd1, 6'sd10, 6'sd4, -8'sd14, 8'sd6); // -10-4 , -
4+10
    end
endmodule

```

```

        check(2'sd1, -2'sd1, -6'sd20, 6'sd15, -8'sd5, 8'sd35); // -20+15 ,
15+20
        check(-2'sd1,-2'sd1, 6'sd31, -6'sd32, -8'sd63, 8'sd1); // -31-32 ,
32-31
        check(2'sd1, 2'sd1, 6'sd0, 6'sd0, 8'sd0, 8'sd0); // silence
sample

        if (errors == 0)
            $display("PASS: tb_css_modulator -- complex multiply matches hand-
computed products");
        else
            $display("FAIL: tb_css_modulator -- %0d mismatches", errors);
        $finish;
    end

endmodule

```

```

`timescale 1ns/1ps

module tb_css_tx_frontend;

    parameter DATA_WIDTH = 8;
    parameter ADDR_WIDTH = 7;

    // Memory File Paths - Adjust if necessary
    parameter MEMFILE_1M = "../vectors/symbol_mapper_1Mbps.mem";
    parameter MEMFILE_250K = "../vectors/symbol_mapper_250kbps.mem";

    reg clk;
    reg reset;
    reg load_1m, load_250k;
    reg enable_1m, enable_250k;
    reg [6:0] payload_length_reg;
    reg [DATA_WIDTH-1:0] ram_memory [0:(1<<ADDR_WIDTH)-1];

    // DUT 1: 1 Mbps Mode (N=3, M=4, GROUP_SIZE=6)
    wire [ADDR_WIDTH-1:0] payload_rd_addr_1m;
    wire frame_done_1m;
    wire [3:0] i_codeword_1m, q_codeword_1m;
    wire i_valid_1m, q_valid_1m;
    wire [15:0] padded_total_bits_1m ;

    // DUT 2: 250 kbps Mode (N=6, M=32, GROUP_SIZE=24)

```

```

wire [ADDR_WIDTH-1:0] payload_rd_addr_250k;
wire frame_done_250k;
wire [31:0] i_codeword_250k, q_codeword_250k;
wire i_valid_250k, q_valid_250k;
wire [15:0] padded_total_bits_250k ;

// Output check trackers
integer err_count_1m = 0;
integer err_count_250k = 0;
integer i_cnt_1m = 0, q_cnt_1m = 0;
integer i_cnt_250k = 0, q_cnt_250k = 0;

// Golden RAM Tables for Verification
reg [3:0] rom_1m [0:7];
reg [31:0] rom_250k [0:63];

// Expected Golden Queues
reg [3:0] exp_i_cw_1m [0:255], exp_q_cw_1m [0:255];
reg [31:0] exp_i_cw_250k [0:255], exp_q_cw_250k [0:255];
integer total_words_1m = 0, total_words_250k = 0;

// Load memory files and check for path errors
task create_mem_files;
begin
    $readmemb(MEMFILE_1M, rom_1m);
    $readmemb(MEMFILE_250K, rom_250k);

    // Diagnostic check for memory load failure
    if (rom_1m[0] === 4'bxxxx) begin
        $display("[ERROR] Failed to load 1Mbps MEM file from: %s",
MEMFILE_1M);
    end
    if (rom_250k[0] === 32'bxxxx) begin
        $display("[ERROR] Failed to load 250kbps MEM file from: %s",
MEMFILE_250K);
    end
end
endtask

// Helper task to compute golden streams
task generate_golden_data;
input [6:0] p_len;
input integer n_in;
input integer group_size;

```

```

input integer is_250k_mode;

integer i, b, idx, bits_tot, padded_tot;
reg [11:0] phr;
reg stream [0:4095];
reg [5:0] grp_i, grp_q;
integer g_i_cnt, g_q_cnt;
begin
    phr = {3'b000, 2'b00, p_len};
    bits_tot = 12 + (p_len * 8);
    padded_tot = bits_tot + ((group_size - (bits_tot % group_size)) %
group_size);

    idx = 0;
    for (i = 0; i < 12; i = i + 1) begin
        stream[idx] = phr[i];
        idx = idx + 1;
    end
    for (i = 0; i < p_len; i = i + 1) begin
        for (b = 0; b < 8; b = b + 1) begin
            stream[idx] = ram_memory[i][b];
            idx = idx + 1;
        end
    end
    while (idx < padded_tot) begin
        stream[idx] = 1'b0;
        idx = idx + 1;
    end

    g_i_cnt = 0; g_q_cnt = 0;
    grp_i = 0; grp_q = 0;

    if (!is_250k_mode) begin
        total_words_1m = padded_tot / (2 * n_in);
        for (i = 0; i < padded_tot; i = i + 2) begin
            grp_i[g_i_cnt % n_in] = stream[i];
            grp_q[g_q_cnt % n_in] = stream[i+1];
            g_i_cnt = g_i_cnt + 1;
            g_q_cnt = g_q_cnt + 1;

            if (g_i_cnt % n_in == 0) begin
                exp_i_cw_1m[(g_i_cnt/n_in)-1] = rom_1m[grp_i[2:0]];
                exp_q_cw_1m[(g_q_cnt/n_in)-1] = rom_1m[grp_q[2:0]];
                grp_i = 0; grp_q = 0;
            end
        end
    end
end

```

```

        end
    end else begin
        total_words_250k = padded_tot / (2 * n_in);
        for (i = 0; i < padded_tot; i = i + 2) begin
            grp_i[g_i_cnt % n_in] = stream[i];
            grp_q[g_q_cnt % n_in] = stream[i+1];
            g_i_cnt = g_i_cnt + 1;
            g_q_cnt = g_q_cnt + 1;

            if (g_i_cnt % n_in == 0) begin
                exp_i_cw_250k[(g_i_cnt/n_in)-1] = rom_250k[grp_i[5:0]];
                exp_q_cw_250k[(g_q_cnt/n_in)-1] = rom_250k[grp_q[5:0]];
                grp_i = 0; grp_q = 0;
            end
        end
    end
end
endtask

// DUT 1: 1 Mbps Instance
css_tx_frontend #(
    .GROUP_SIZE(6), .N_IN(3), .M_OUT(4), .DATA_RATE(0) ) uut_1m (
        .clk(clk), .reset(reset), .load(load_1m), .enable(enable_1m),
        .payload_length_reg(payload_length_reg),
        .payload_rd_data(ram_memory[payload_rd_addr_1m]),
        .payload_rd_addr(payload_rd_addr_1m),
        .padded_total_bits(padded_total_bits_1m),
        .frame_done(frame_done_1m),
        .i_codeword(i_codeword_1m), .i_codeword_valid(i_valid_1m),
        .q_codeword(q_codeword_1m), .q_codeword_valid(q_valid_1m)
    );

// DUT 2: 250 kbps Instance
css_tx_frontend #(
    .GROUP_SIZE(24), .N_IN(6), .M_OUT(32), .DATA_RATE(1) ) uut_250k (
        .clk(clk), .reset(reset), .load(load_250k), .enable(enable_250k),
        .payload_length_reg(payload_length_reg),
        .payload_rd_data(ram_memory[payload_rd_addr_250k]),
        .payload_rd_addr(payload_rd_addr_250k),
        .padded_total_bits(padded_total_bits_250k),
        .frame_done(frame_done_250k),
        .i_codeword(i_codeword_250k), .i_codeword_valid(i_valid_250k),
        .q_codeword(q_codeword_250k), .q_codeword_valid(q_valid_250k)
    );

```

```

// Clock Generation
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

// Self-checking monitors
always @(posedge clk) begin
    if (i_valid_1m) begin
        if (i_codeword_1m !== exp_i_cw_1m[i_cnt_1m]) begin
            $display("[FAIL 1M I] Index %0d | Exp: %b | Got: %b", i_cnt_1m,
exp_i_cw_1m[i_cnt_1m], i_codeword_1m);
            err_count_1m = err_count_1m + 1;
        end
        i_cnt_1m = i_cnt_1m + 1;
    end
    if (q_valid_1m) begin
        if (q_codeword_1m !== exp_q_cw_1m[q_cnt_1m]) begin
            $display("[FAIL 1M Q] Index %0d | Exp: %b | Got: %b", q_cnt_1m,
exp_q_cw_1m[q_cnt_1m], q_codeword_1m);
            err_count_1m = err_count_1m + 1;
        end
        q_cnt_1m = q_cnt_1m + 1;
    end

    if (i_valid_250k) begin
        if (i_codeword_250k !== exp_i_cw_250k[i_cnt_250k]) begin
            $display("[FAIL 250K I] Index %0d | Exp: %b | Got: %b",
i_cnt_250k, exp_i_cw_250k[i_cnt_250k], i_codeword_250k);
            err_count_250k = err_count_250k + 1;
        end
        i_cnt_250k = i_cnt_250k + 1;
    end
    if (q_valid_250k) begin
        if (q_codeword_250k !== exp_q_cw_250k[q_cnt_250k]) begin
            $display("[FAIL 250K Q] Index %0d | Exp: %b | Got: %b",
q_cnt_250k, exp_q_cw_250k[q_cnt_250k], q_codeword_250k);
            err_count_250k = err_count_250k + 1;
        end
        q_cnt_250k = q_cnt_250k + 1;
    end
end

// Stimulus Execution
initial begin

```

```

create_mem_files();

ram_memory[0] = 8'hA5;
ram_memory[1] = 8'h3C;
ram_memory[2] = 8'hF0;
ram_memory[3] = 8'h54;

reset = 1'b1;
load_1m = 1'b0; enable_1m = 1'b0;
load_250k = 1'b0; enable_250k = 1'b0;

// -----
// Test Case 1: 1 Mbps Mode (N=3, M=4)
// -----
payload_length_reg = 7'd4;
generate_golden_data(7'd4, 3, 6, 0);

#20; @(posedge clk); reset = 1'b0;
@(posedge clk); load_1m = 1'b1;
@(posedge clk); load_1m = 1'b0; enable_1m = 1'b1;

wait(frame_done_1m);
repeat(5) @(posedge clk);
enable_1m = 1'b0;

// -----
// Test Case 2: 250 kbps Mode (N=6, M=32)
// -----
payload_length_reg = 7'd3; // Correct payload length for Test Case 2
i_cnt_250k = 0; q_cnt_250k = 0; // Reset index trackers
generate_golden_data(7'd3, 6, 24, 1);

#20; @(posedge clk); reset = 1'b1;
#20; @(posedge clk); reset = 1'b0;
@(posedge clk); load_250k = 1'b1;
@(posedge clk); load_250k = 1'b0; enable_250k = 1'b1;

wait(frame_done_250k);
repeat(5) @(posedge clk);
enable_250k = 1'b0;

$display("\n=====");
$display("                FRONTEND TEST REPORT                ");
$display("=====");
$display(" Mode 1 (1 Mbps, N=3, M=4):");

```

```

        $display("    I/Q Words Checked: %0d / %0d", i_cnt_1m, total_words_1m);
        $display("    Errors Found      : %0d", err_count_1m);
        $display(" Mode 2 (250 kbps, N=6, M=32):");
        $display("    I/Q Words Checked: %0d / %0d", i_cnt_250k,
total_words_250k);
        $display("    Errors Found      : %0d", err_count_250k);
        $display("=====");

        if (err_count_1m == 0 && err_count_250k == 0 && i_cnt_1m ==
total_words_1m && i_cnt_250k == total_words_250k)
            $display(" OVERALL STATUS      : PASSED SUCCESS\n");
        else
            $display(" OVERALL STATUS      : FAILED ERROR\n");

        $finish;
    end

endmodule

```

```

`timescale 1ns/1ps
//=====
// tb_css_tx_top.v -- full-chain integration testbench (Section 6 of the
// project spec: "a full-chain testbench that applies a payload, runs the
// simulation, and compares... against the... MATLAB model output").
//
// No bit-exact full-chain MATLAB golden file was provided among the project
// inputs (only per-block references: preambleSFD.txt, chirpSequence.txt,
// payload.txt), so this testbench is self-checking against two independent
// sources of truth instead:
// 1) The exact arithmetic worked example in Section 13 of the deep-dive
// doc, for the project's own default configuration (payloadLength=25,
// chirpIndex=1, 1 Mbps): padded_total_bits=216, n_codewords/path=36,
// n_groups=48, total_symbols=192, total Tx samples=9216.
// 2) A bit-exact hand-derived expected sample: since the first 32
// preamble chips are constant '1' on both I and Q, the first 4 QPSK/
// DQPSK symbols of the packet are all (Xr,Xi)=(1,0), so
// S(0)=S(1)=S(2)=S(3) = (1,0)*(1,1) = (1,1) (the initial condition
// passes straight through). At chirp_rom address 151 (last sample of
// chirp_m1.mem, chirpIndex=1) the stored sample is (real,imag) =
// (-31, 0), so the very first group's last modulated sample must be
// Tx = (1+j1)*(-31+j0) = (-31, -31).
//=====
module tb_css_tx_top;

```

```

reg clk, reset;
reg start_Tx;
reg [7:0] PayloadLength;
reg payload_wr_en;
reg [6:0] payload_wr_addr;
reg [7:0] payload_wr_data;
wire done_Tx;
wire signed [7:0] Tx_real, Tx_imag;
wire tx_valid;

css_tx_top dut (
    .clk(clk), .reset(reset),
    .start_Tx(start_Tx), .PayloadLength(PayloadLength),
    .payload_wr_en(payload_wr_en), .payload_wr_addr(payload_wr_addr),
    .payload_wr_data(payload_wr_data),
    .done_Tx(done_Tx), .Tx_real(Tx_real), .Tx_imag(Tx_imag),
    .tx_valid(tx_valid)
);

always #5 clk = ~clk;

reg [7:0] payload_bytes [0:27];
integer i, errors;
integer sample_count;
reg [2:0] prev_state;
reg      first151_checked;

localparam S_SUBCHIRP = 3'd2, S_GAP = 3'd3;

initial begin
    errors = 0;
    sample_count = 0;
    first151_checked = 1'b0;
    clk = 0; reset = 1; start_Tx = 0; PayloadLength = 8'd0;
    payload_wr_en = 0; payload_wr_addr = 0; payload_wr_data = 0;

    $readmemb("vectors/payload.mem", payload_bytes);

    repeat (3) @(negedge clk);
    reset = 0;
    @(negedge clk);

    // load the first 25 bytes (payloadLength=25, the project's own default)
    for (i = 0; i < 25; i = i + 1) begin

```

```

        payload_wr_en    = 1'b1;
        payload_wr_addr = i[6:0];
        payload_wr_data = payload_bytes[i];
        @(negedge clk);
    end
    payload_wr_en = 1'b0;

    PayloadLength = 8'd25;
    start_Tx = 1'b1;
    @(negedge clk);
    start_Tx = 1'b0;

    // ---- structural checks against the Section-13 worked example ----
    #1;
    if (dut.padded_total_bits != 16'd216) begin
        $display("FAIL padded_total_bits=%0d expected 216",
dut.padded_total_bits);
        errors = errors + 1;
    end
    if (dut.n_codewords_per_path != 16'd36) begin
        $display("FAIL n_codewords_per_path=%0d expected 36",
dut.n_codewords_per_path);
        errors = errors + 1;
    end
    if (dut.n_groups != 16'd48) begin
        $display("FAIL n_groups=%0d expected 48", dut.n_groups);
        errors = errors + 1;
    end
    end

    // ---- run until done_Tx, counting samples and catching the addr-151
event ----
    prev_state = dut.state;
    while (done_Tx != 1'b1) begin
        @(posedge clk);
        if (tx_valid) sample_count = sample_count + 1;

        // detect the edge SUBCHIRP(addr=151,group=0) -> GAP :
Tx_real/Tx_imag
        // now hold the modulated sample for chirp_rom address 151 of group 0
        if (!first151_checked && prev_state == S_SUBCHIRP && dut.state ==
S_GAP && dut.group_idx == 16'd0) begin
            first151_checked = 1'b1;
            #1;
            if (Tx_real != -8'sd31 || Tx_imag != -8'sd31) begin

```

```

        $display("FAIL first-group addr=151 sample = (%0d,%0d),
expected (-31,-31)", Tx_real, Tx_imag);
        errors = errors + 1;
    end else
        $display("INFO first-group addr=151 sample = (%0d,%0d)
matches hand-derived expectation", Tx_real, Tx_imag);
    end
    prev_state = dut.state;
end

// total modulated (non-gap) samples must be n_groups * 152 = 48*152 =
7296
if (sample_count != 7296) begin
    $display("FAIL total tx_valid sample count = %0d, expected 7296 (48
groups x 152)", sample_count);
    errors = errors + 1;
end else
    $display("INFO total modulated samples = %0d (matches 48 groups x
152)", sample_count);

    if (errors == 0)
        $display("PASS: tb_css_tx_top -- structural counts and first bit-
exact sample match expectations");
    else
        $display("FAIL: tb_css_tx_top -- %0d mismatches", errors);
    $finish;
end

// safety timeout
initial begin
    #2_000_000;
    $display("FAIL: tb_css_tx_top -- timeout waiting for done_Tx");
    $finish;
end

endmodule

```

```

`timescale 1ns/1ps

module tb_demux_iq;

    reg        bit_in;
    reg [15:0] bit_index;

```

```

wire      i_bit, q_bit, i_valid, q_valid;

integer i, errors;
reg [0:9] pattern; // 10 bits: bit1..bit10 in Section 5's own numbering
example

demux_iq dut (
    .bit_in(bit_in), .bit_index(bit_index),
    .i_bit(i_bit), .q_bit(q_bit),
    .i_valid(i_valid), .q_valid(q_valid)
);

initial begin
    errors = 0;
    pattern = 10'b1011001101; // arbitrary padded-stream fragment

    for (i = 0; i < 10; i = i + 1) begin
        bit_in = pattern[i];
        bit_index = i[15:0];
        #1;
        // 0-based: even index -> I, odd index -> Q (Section 5: "bit1->I,
        bit2->Q,..." 1-indexed == even/odd 0-indexed)
        if (i % 2 == 0) begin
            if (!(i_valid === 1'b1 && q_valid === 1'b0 && i_bit === bit_in))
begin
                $display("FAIL idx=%0d expected I-valid, got i_valid=%b
q_valid=%b i_bit=%b",
                    i, i_valid, q_valid, i_bit);
                errors = errors + 1;
            end
        end else begin
            if (!(q_valid === 1'b1 && i_valid === 1'b0 && q_bit === bit_in))
begin
                $display("FAIL idx=%0d expected Q-valid, got i_valid=%b
q_valid=%b q_bit=%b",
                    i, i_valid, q_valid, q_bit);
                errors = errors + 1;
            end
        end
    end

    if (errors == 0)
        $display("PASS: tb_demux_iq");
    else
        $display("FAIL: tb_demux_iq -- %0d mismatches", errors);

```

```

        $finish;
    end

endmodule

```

```

`timescale 1ns/1ps

module tb_dqpsk_encoder;

    reg clk, reset, symbol_valid;
    reg signed [1:0] x_real, x_imag;
    wire signed [1:0] s_real, s_imag;

    dqpsk_encoder dut (
        .clk(clk), .reset(reset), .symbol_valid(symbol_valid),
        .x_real(x_real), .x_imag(x_imag),
        .s_real(s_real), .s_imag(s_imag)
    );

    always #5 clk = ~clk;

    // ---- independent reference model: same recursion, computed in the TB ----
    integer ref_mem_r [0:3];
    integer ref_mem_i [0:3];
    integer ref_wptr;
    integer exp_r, exp_i;

    // The 4 possible QPSK symbols, cycled through in a known order so the
    // TB and the DUT are always driven identically.
    reg signed [1:0] seq_r [0:15];
    reg signed [1:0] seq_i [0:15];
    integer n, errors;

    initial begin
        // exercise all 4 QPSK constellation points, repeated, 16 symbols total
        seq_r[0]=1; seq_i[0]=0; seq_r[1]=0; seq_i[1]=-1;
        seq_r[2]=0; seq_i[2]=1; seq_r[3]=-1; seq_i[3]=0;
        seq_r[4]=-1; seq_i[4]=0; seq_r[5]=1; seq_i[5]=0;
        seq_r[6]=0; seq_i[6]=-1; seq_r[7]=0; seq_i[7]=1;
        seq_r[8]=0; seq_i[8]=1; seq_r[9]=-1; seq_i[9]=0;
        seq_r[10]=1; seq_i[10]=0; seq_r[11]=0; seq_i[11]=-1;
        seq_r[12]=0; seq_i[12]=-1; seq_r[13]=0; seq_i[13]=1;
        seq_r[14]=-1; seq_i[14]=0; seq_r[15]=1; seq_i[15]=0;
    end

```

```

end

initial begin
    errors = 0;
    clk = 0; reset = 1; symbol_valid = 0; x_real = 0; x_imag = 0;
    ref_mem_r[0]=1; ref_mem_r[1]=1; ref_mem_r[2]=1; ref_mem_r[3]=1;
    ref_mem_i[0]=1; ref_mem_i[1]=1; ref_mem_i[2]=1; ref_mem_i[3]=1;
    ref_wptr = 0;

    @(negedge clk); @(negedge clk);
    reset = 0;

    for (n = 0; n < 16; n = n + 1) begin
        @(negedge clk);
        x_real = seq_r[n]; x_imag = seq_i[n]; symbol_valid = 1;
        // reference model:  $S(n) = X(n) * S(n-4)$ 
        exp_r = seq_r[n]*ref_mem_r[ref_wptr] - seq_i[n]*ref_mem_i[ref_wptr];
        exp_i = seq_r[n]*ref_mem_i[ref_wptr] + seq_i[n]*ref_mem_r[ref_wptr];
        ref_mem_r[ref_wptr] = exp_r;
        ref_mem_i[ref_wptr] = exp_i;
        ref_wptr = (ref_wptr + 1) % 4;

        @(negedge clk); // one cycle later, DUT's s_real/s_imag reflect S(n)
        symbol_valid = 0;
        if (s_real != exp_r[1:0] || s_imag != exp_i[1:0]) begin
            $display("FAIL n=%0d X=(%0d,%0d) -> DUT S=(%0d,%0d) expected
(%0d,%0d)",
                    n, seq_r[n], seq_i[n], s_real, s_imag, exp_r, exp_i);
            errors = errors + 1;
        end
    end

    if (errors == 0)
        $display("PASS: tb_dqpsk_encoder -- matches independent
S(n)=X(n)*S(n-4) reference model");
    else
        $display("FAIL: tb_dqpsk_encoder -- %0d mismatches", errors);
    $finish;
end

endmodule

```

```
`timescale 1ns/1ps
```

```

module tb_interleaver_ppdu_top;

    // Configuration: 1Mbps mode (M=4, DATA_RATE=0)
    localparam M                = 4;
    localparam DATA_RATE       = 1'b0;
    localparam PREAMBLE_TOTAL_BITS = 48;
    localparam CLK_PERIOD       = 10; // 100 MHz

    reg        clk;
    reg        reset;
    reg        start;
    reg  [M-1:0] i_codeword_in;
    reg        i_codeword_valid;
    reg  [M-1:0] q_codeword_in;
    reg        q_codeword_valid;
    reg        last_codeword;

    wire        req_next_symbol;
    wire        i_bit;
    wire        q_bit;
    wire        chip_valid;
    wire        frame_done;

    // DUT Instance
    interleaver_ppdu_top #(
        .M                (M),
        .DATA_RATE        (DATA_RATE),
        .PREAMBLE_TOTAL_BITS(PREAMBLE_TOTAL_BITS)
    ) u_dut (
        .clk                (clk),
        .reset              (reset),
        .start              (start),
        .i_codeword_in      (i_codeword_in),
        .i_codeword_valid   (i_codeword_valid),
        .q_codeword_in      (q_codeword_in),
        .q_codeword_valid   (q_codeword_valid),
        .last_codeword      (last_codeword),
        .req_next_symbol    (req_next_symbol),
        .i_bit              (i_bit),
        .q_bit              (q_bit),
        .chip_valid         (chip_valid),
        .frame_done         (frame_done)
    );

```

```

// Clock Generator
always #(CLK_PERIOD/2) clk = ~clk;

integer symbol_cnt;

initial begin
    // Initialize Signals
    clk          = 0;
    reset        = 1;
    start        = 0;
    i_codeword_in = 0;
    i_codeword_valid = 0;
    q_codeword_in = 0;
    q_codeword_valid = 0;
    last_codeword = 0;
    symbol_cnt    = 0;

    // Reset Sequence
    #(CLK_PERIOD * 4);
    reset = 0;
    #(CLK_PERIOD * 2);

    $display("--- Starting PPDU Transmission Test ---");
    @(posedge clk);
    start <= 1'b1;
    @(posedge clk);
    start <= 1'b0;

    // Dynamic Handshake Loop
    while (!frame_done) begin
        @(posedge clk);

        // Default valid signals low
        i_codeword_valid <= 1'b0;
        q_codeword_valid <= 1'b0;

        if (req_next_symbol) begin
            symbol_cnt = symbol_cnt + 1;
            $display("[TB @ %0t ps] Received req_next_symbol for Symbol
#%0d", $time, symbol_cnt);

            if (DATA_RATE == 1'b0) begin
                // 1Mbps: Supply 1 4-bit codeword pair per symbol request
                i_codeword_in  <= me_val(symbol_cnt);
                q_codeword_in  <= ~ me_val(symbol_cnt);
            end
        end
    end
end

```

```

        i_codeword_valid <= 1'b1;
        q_codeword_valid <= 1'b1;

        if (symbol_cnt == 4) begin
            last_codeword <= 1'b1;
            $display("[TB @ %0t ps] Asserted LAST codeword flag.",
$time);
        end
    end else begin
        // 250kbps: Supply 2 consecutive 32-bit codewords per request
        // Codeword N
        i_codeword_in    <= 32'hA5A5A5A5;
        q_codeword_in    <= 32'h5A5A5A5A;
        i_codeword_valid <= 1'b1;
        q_codeword_valid <= 1'b1;
        @(posedge clk);
        // Codeword N+1
        i_codeword_in    <= 32'hFFFF0000;
        q_codeword_in    <= 32'h0000FFFF;
        i_codeword_valid <= 1'b1;
        q_codeword_valid <= 1'b1;

        if (symbol_cnt == 2) begin
            last_codeword <= 1'b1;
            $display("[TB @ %0t ps] Asserted LAST codeword flag.",
$time);
        end
    end
end
end
end

    $display("[TB @ %0t ps] Transmission Finished Successfully! (frame_done
asserted)", $time);
    #(CLK_PERIOD * 5);
    $finish;
end

// Value Generator Helper Function for Test Data
function [3:0] me_val; input integer idx;
begin
    case (idx)
        1: me_val = 4'b1010;
        2: me_val = 4'b1100;
        3: me_val = 4'b0011;
        default: me_val = 4'b1111;
    endcase
end

```

```

        endcase
    end
endfunction

// Monitor Output Serial Stream
always @(posedge clk) begin
    if (chip_valid) begin
        $display("    [CHIP OUTPUT @ %0t ps] I = %b | Q = %b", $time, i_bit,
q_bit);
    end
end

endmodule

```

```

`timescale 1ns/1ps

module tb_preamble_sfd;

    reg [7:0] addr1m, addr250;
    wire b1m, b250;

    preamble_sfd_rom #(.TOTAL_BITS(48),
.MEMFILE("vectors/preamble_sfd_1mbps.mem"))
        dut1m (.addr(addr1m), .bit_out(b1m));

    preamble_sfd_rom #(.TOTAL_BITS(96),
.MEMFILE("vectors/preamble_sfd_250kbps.mem"))
        dut250 (.addr(addr250), .bit_out(b250));

    integer i, errors;

    initial begin
        errors = 0;

        // ---- 1 Mbps: 32 ones then SFD = 0111010010011100 ----
        for (i = 0; i < 32; i = i + 1) begin
            addr1m = i; #1;
            if (b1m !== 1'b1) begin
                $display("FAIL 1Mbps preamble bit %0d = %b (expected 1)", i,
b1m);
                errors = errors + 1;
            end
        end
        begin : sfd1m_check

```

```

        reg [15:0] sfd_pat;
        sfd_pat = 16'b01111010010011100;
        for (i = 0; i < 16; i = i + 1) begin
            addr1m = 32 + i; #1;
            if (b1m != sfd_pat[15-i]) begin
                $display("FAIL 1Mbps SFD bit %0d = %b expected %b", i, b1m,
sfd_pat[15-i]);
                errors = errors + 1;
            end
        end
    end
end

    // ---- 250 kbps: 80 ones then SFD (from bipolar -1,1,1,1,1,-1,1,-1,-1,-
1,1,-1,-1,-1,1,1) ----
    for (i = 0; i < 80; i = i + 1) begin
        addr250 = i; #1;
        if (b250 != 1'b1) begin
            $display("FAIL 250kbps preamble bit %0d = %b (expected 1)", i,
b250);
            errors = errors + 1;
        end
    end
end
begin : sfd250_check
    reg [15:0] sfd_pat;
    sfd_pat = 16'b0111101000100011;
    for (i = 0; i < 16; i = i + 1) begin
        addr250 = 80 + i; #1;
        if (b250 != sfd_pat[15-i]) begin
            $display("FAIL 250kbps SFD bit %0d = %b expected %b", i,
b250, sfd_pat[15-i]);
            errors = errors + 1;
        end
    end
end
end

    if (errors == 0)
        $display("PASS: tb_preamble_sfd -- both data rates match reference
SFD/preamble patterns");
    else
        $display("FAIL: tb_preamble_sfd -- %0d mismatches", errors);
    $finish;
end

endmodule

```

```

`timescale 1ns/1ps

module tb_qpsk_mapper;

    reg i_bit, q_bit;
    wire signed [1:0] qpsk_real, qpsk_imag;
    integer errors;

    qpsk_mapper dut ( .i_bit(i_bit), .q_bit(q_bit),
                      .qpsk_real(qpsk_real), .qpsk_imag(qpsk_imag) );

    task check;
        input i, q;
        input signed [1:0] exp_r, exp_i;
        begin
            i_bit = i; q_bit = q; #1;
            if (qpsk_real != exp_r || qpsk_imag != exp_i) begin
                $display("FAIL I=%0d Q=%0d -> real=%0d imag=%0d (expected
real=%0d imag=%0d)",
                        i, q, qpsk_real, qpsk_imag, exp_r, exp_i);
                errors = errors + 1;
            end
        end
    endtask

    initial begin
        errors = 0;
        // From Section 8.2 of the deep-dive doc:
        // I=+1,Q=+1 -> (+1,0) ; I=+1,Q=-1 -> (0,-1) ; I=-1,Q=+1 -> (0,+1) ; I=-
1,Q=-1 -> (-1,0)
        check(1'b1, 1'b1, 2'sd1, 2'sd0);
        check(1'b1, 1'b0, 2'sd0, -2'sd1);
        check(1'b0, 1'b1, 2'sd0, 2'sd1);
        check(1'b0, 1'b0, -2'sd1, 2'sd0);

        if (errors == 0)
            $display("PASS: tb_qpsk_mapper -- all 4 constellation points match
Section 8.2");
        else
            $display("FAIL: tb_qpsk_mapper -- %0d mismatches", errors);
        $finish;
    end
end

```

```
endmodule
```

```
// Self-checking testbench for tx_datapath_top (controller external).
// Models the Controller as a simple symbol source: responds to symbol_req
// with one (i_bit,q_bit) pair + qpsk_valid pulse.
`timescale 1ns/1ps
module tb_tx_datapath_top;
    localparam NSW = 12, ROMW = 5, OUTW = 6;

    logic clk = 0; always #5 clk = ~clk;
    logic reset;
    logic start;
    logic [1:0] chirp_index;
    logic [NSW-1:0] num_symbols;
    logic tx_done;
    logic i_bit, q_bit, qpsk_valid;
    logic symbol_req;
    logic signed [OUTW-1:0] tx_real, tx_imag;
    logic tx_valid;

    tx_datapath_top #(.NUM_SYMBOLS_WIDTH(NSW), .ROM_WIDTH(ROMW),
.OUT_WIDTH(OUTW)) uut (
        .clk(clk), .reset(reset), .start(start), .chirp_index(chirp_index),
        .num_symbols(num_symbols), .tx_done(tx_done),
        .i_bit(i_bit), .q_bit(q_bit), .qpsk_valid(qpsk_valid),
.symbol_req(symbol_req),
        .tx_real(tx_real), .tx_imag(tx_imag), .tx_valid(tx_valid)
    );

    int unsigned error_count = 0, check_count = 0;
    task automatic check(input bit c, input string m);
        check_count++;
        if (!c) begin error_count++; $display("[%0t] ERROR: %s", $time, m); end
    endtask

    // ---- Controller model: one symbol per request, configurable latency ----
    int resp_min = 0, resp_max = 0, dly;
    int symbols_sent = 0;
    initial begin
        i_bit = 0; q_bit = 0; qpsk_valid = 0;
        forever begin
            @(posedge clk);
            if (symbol_req) begin
```

```

        dly = (resp_max > resp_min) ? $urandom_range(resp_max, resp_min)
: resp_min;

        repeat (dly) @(posedge clk);
        i_bit      <= $urandom_range(1,0);
        q_bit      <= $urandom_range(1,0);
        qpsk_valid <= 1'b1;
        symbols_sent++;
        @(posedge clk);
        qpsk_valid <= 1'b0;
    end
end
end

// ---- continuous checks ----
int unsigned tx_valid_count = 0;
always @(negedge clk) if (!reset) begin
    if (tx_valid) begin
        tx_valid_count++;
        // output must always fit the derived 6-bit signed range
        check(tx_real >= -32 && tx_real <= 31, "tx_real out of 6-bit signed
range");
        check(tx_imag >= -32 && tx_imag <= 31, "tx_imag out of 6-bit signed
range");
    end
end

task automatic do_reset();
    reset <= 1; start <= 0; chirp_index <= 0; num_symbols <= 0;
    repeat (3) @(posedge clk);
    reset <= 0; @(posedge clk);
endtask

task automatic run_packet(input [1:0] ci, input [NSW-1:0] n, output int
unsigned vc, output int unsigned sent);
    int to;
    tx_valid_count = 0; symbols_sent = 0;
    @(posedge clk);
    chirp_index <= ci; num_symbols <= n; start <= 1;
    @(posedge clk); start <= 0;
    to = 0;
    while (!tx_done && to < 200000) begin @(posedge clk); to++; end
    check(to < 200000, $sformatf("TIMEOUT (ci=%0d n=%0d)", ci, n));
    repeat (4) @(posedge clk);
    vc = tx_valid_count; sent = symbols_sent;
endtask

```

```

int unsigned vc, sent;
initial begin
    $display("=== tb_tx_datapath_top ===");
    do_reset();

    // TC1: reset state
    check(tx_valid === 1'b0, "tx_valid not 0 after reset");
    check(tx_done === 1'b0, "tx_done not 0 after reset");

    // TC2: single chirp sequence, all chirp_index values
    for (int ci = 0; ci < 4; ci++) begin
        run_packet(ci[1:0], 12'd4, vc, sent);
        check(vc == 4*38, $sformatf("ci=%0d: expected %0d tx_valid, got %0d",
ci, 4*38, vc));
        check(sent == 4, $sformatf("ci=%0d: expected 4 symbols consumed, got
%0d", ci, sent));
        do_reset();
    end

    // TC3: multi-sequence
    for (int ci = 0; ci < 4; ci++) begin
        run_packet(ci[1:0], 12'd12, vc, sent);
        check(vc == 12*38, $sformatf("ci=%0d: expected %0d tx_valid, got
%0d", ci, 12*38, vc));
        check(sent == 12, $sformatf("ci=%0d: expected 12 symbols consumed,
got %0d", ci, sent));
        do_reset();
    end

    // TC4: larger packet (full preamble+SFD+payload scale, 1 Mbps: 48+8=56)
    run_packet(2'd0, 12'd56, vc, sent);
    check(vc == 56*38, $sformatf("expected %0d tx_valid, got %0d", 56*38,
vc));
    check(sent == 56, $sformatf("expected 56 symbols consumed, got %0d",
sent));
    do_reset();

    // TC5: 250 kbps scale (96+12=108)
    run_packet(2'd2, 12'd108, vc, sent);
    check(vc == 108*38, $sformatf("expected %0d tx_valid, got %0d", 108*38,
vc));
    check(sent == 108, $sformatf("expected 108 symbols consumed, got %0d",
sent));
    do_reset();

```

```

        // TC6: variable controller response latency
        resp_min = 0; resp_max = 4;
        run_packet(2'd1, 12'd8, vc, sent);
        check(vc == 8*38, $sformatf("variable latency: expected %0d tx_valid, got %0d", 8*38, vc));
        check(sent == 8, $sformatf("variable latency: expected 8 symbols, got %0d", sent));
        resp_min = 0; resp_max = 0;
        do_reset();

        // TC7: back-to-back packets
        run_packet(2'd0, 12'd4, vc, sent);
        check(vc == 4*38 && sent == 4, "back-to-back packet 1 stats wrong");
        run_packet(2'd3, 12'd8, vc, sent);
        check(vc == 8*38 && sent == 8, "back-to-back packet 2 stats wrong -- stale state?");

        $display("=====");
        if (error_count == 0) $display(" ALL TESTS PASSED (%0d checks, 0 errors)", check_count);
        else $display(" TESTS FAILED (%0d checks, %0d errors)", check_count, error_count);
        $display("=====");
        $finish;
    end

    initial begin #10_000_000; $display("WATCHDOG TIMEOUT"); $finish; end
endmodule

```

```

`timescale 1ns / 1ps

module tb_zero_padding;

    // Parameters
    parameter GROUP_SIZE = 6;
    parameter DATA_WIDTH = 8;
    parameter ADDR_WIDTH = 7;

    // DUT Inputs
    reg clk;
    reg reset;
    reg load;

```

```

reg enable;
reg [6:0] payload_length_reg;
reg [DATA_WIDTH-1:0] payload_rd_data;

// DUT Outputs
wire [4:0] pad_bits;
wire [15:0] padded_total_bits;
wire [ADDR_WIDTH-1:0] payload_rd_addr;
wire [15:0] bit_index;
wire bit_out;
wire frame_done;

// Simulated RAM for payload data
reg [DATA_WIDTH-1:0] ram_memory [0:(1<<ADDR_WIDTH)-1];

// Self-checking variables
reg [15:0] expected_padded_total_bits;
reg [15:0] expected_total_bits;
reg expected_stream [0:2047];
integer check_idx = 0;
integer err_count = 0;
reg data_valid; // Tracks the 1-cycle latency of bit_out

// Instantiate the Unit Under Test (UUT)
zero_padding #(
    .GROUP_SIZE(GROUP_SIZE),
    .DATA_WIDTH(DATA_WIDTH),
    .ADDR_WIDTH(ADDR_WIDTH)
) uut (
    .clk(clk),
    .reset(reset),
    .load(load),
    .enable(enable),
    .payload_length_reg(payload_length_reg),
    .pad_bits(pad_bits),
    .padded_total_bits(padded_total_bits),
    .payload_rd_data(payload_rd_data),
    .payload_rd_addr(payload_rd_addr),
    .bit_index(bit_index),
    .bit_out(bit_out),
    .frame_done(frame_done)
);

// Clock generation (100MHz / 10ns period)
initial begin

```

```

        clk = 0;
        forever #5 clk = ~clk;
    end

    // RAM read logic (combinational read)
    always @(*) begin
        payload_rd_data = ram_memory[payload_rd_addr];
    end

    // Task to generate the expected output sequence
    task generate_expected;
        input [6:0] p_len;
        integer i, b, idx;
        reg [11:0] phr;
        begin
            // 1. Calculate bit counts
            expected_total_bits = 12 + (p_len * 8);
            expected_padded_total_bits = expected_total_bits + ((GROUP_SIZE -
(expected_total_bits % GROUP_SIZE)) % GROUP_SIZE);
            phr = {3'b000, 2'b00, p_len};
            idx = 0;

            // 2. Populate PHY Header (12 bits)
            for (i = 0; i < 12; i = i + 1) begin
                expected_stream[idx] = phr[i];
                idx = idx + 1;
            end

            // 3. Populate PSDU Payload (p_len bytes)
            for (i = 0; i < p_len; i = i + 1) begin
                for (b = 0; b < 8; b = b + 1) begin
                    expected_stream[idx] = ram_memory[i][b];
                    idx = idx + 1;
                end
            end

            // 4. Populate Padding Bits (Zeroes)
            while (idx < expected_padded_total_bits) begin
                expected_stream[idx] = 1'b0;
                idx = idx + 1;
            end
        end
    endtask

    // DUT Valid output tracker

```

```

    // The DUT updates payload_bit_in synchronously when enable is high and it's
not IDLE.
    // Therefore, bit_out is valid exactly 1 cycle later.
    always @(posedge clk) begin
        if (reset)
            data_valid <= 1'b0;
        else if (enable && (uut.state != 2'b00))
            data_valid <= 1'b1;
        else
            data_valid <= 1'b0;
    end

    // Self-checking block
    always @(posedge clk) begin
        if (data_valid) begin
            if (bit_out != expected_stream[check_idx]) begin
                $display("[FAIL] Time: %0t | Index: %4d | Expected: %b | Got:
%b",
                        $time, check_idx, expected_stream[check_idx], bit_out);
                err_count = err_count + 1;
            end
            check_idx = check_idx + 1;
        end
    end

    // Stimulus
    initial begin
        // Initialize Signals
        reset = 1'b1;
        load = 1'b0;
        enable = 1'b0;
        payload_length_reg = 7'd4; // 4 bytes payload to test padding logic
properly

        // Initialize RAM with test data
        ram_memory[0] = 8'hA5; // 10100101
        ram_memory[1] = 8'h3C; // 00111100
        ram_memory[2] = 8'hF0; // 11110000
        ram_memory[3] = 8'h45;

        // Generate expected golden data
        generate_expected(payload_length_reg);

        // Reset sequence
        #20;

```

```

    @(posedge clk);
    reset = 1'b0;

    // Trigger Load
    @(posedge clk);
    load = 1'b1;

    // Start shifting bits
    @(posedge clk);
    load = 1'b0;
    enable = 1'b1;

    // Wait for the frame to finish from the DUT
    wait(frame_done == 1'b1);

    // Let it run a few more cycles to clock out the final bit and return to
IDLE
    repeat(2) @(posedge clk);
    enable = 1'b0;

    // Final Report
    $display("\n=====");
    $display("                SIMULATION COMPLETE                ");
    $display("=====");
    $display(" Total Bits Checked : %0d", check_idx);
    $display(" Expected Bits      : %0d", expected_padded_total_bits);
    $display(" Errors Found       : %0d", err_count);

    if (err_count == 0 && check_idx == expected_padded_total_bits) begin
        $display(" STATUS                : PASSED SUCCESS \n");
    end else begin
        $display(" STATUS                : FAILED ERROR \n");
    end
    $display("=====");

    $finish;
end

endmodule

```